Agentic CCP: Target Architecture for AI-Native Compliance

Nov 9, 2026 · @Claude

Bottom line. Keep CCP's core, because it is what no incumbent documents and what auditors and buyers will test: write-once raw evidence, append-only observations and verdicts, bitemporal state, extract-then-rule and forced row-level security. Build the target around one rule: code decides every status from complete, versioned facts and abstains when the facts are incomplete; models read, extract, investigate and draft only through one permission-checked tool registry; and every AI claim is a pointer that code checks before a person, or later a measured policy, accepts it. The agent stack follows what frontier teams do now (6.4 to 6.11): harness changes ship as releases behind evals, each prompt is rebuilt from the run log, agents search through layered, per-tenant retrieval that is evaluated on its own, and guardrails and authorization sit outside the model. Run it as one Python service on one Postgres per cell, with S3 Object Lock, a per-tenant tamper-evident record and the Anthropic API, and add no workflow engine, broker, second database or sandbox until a named trigger fires. Get there in three steps: ten weeks of fixes inside today's architecture that close the verified false-pass and fail-open paths and take the first real-model measurements (the first report's Q1 to Q13, plus tenant-first keys); by month 8, a production cell with a connector contract that proves completeness, an authenticated approval service, one tool registry and evals that gate every release; then the differentiators, which are pull-request fixes rendered by code, evidence review and questionnaires with machine-checked citations, packages auditors can verify, and autonomy that reversible actions earn from measured false-pass rates. CCP will not out-integrate Vanta soon, but it can be the platform whose every PASS is complete, checkable and accountable, which is what buyers and auditors now scrutinize.

How to read this report. Linked statements were checked against their source on 9 Nov 2026. "Inferred" or "in our reading" marks reasoning across sources, and "proposed" or "Proposed for CCP" marks a design choice. Code references are to the `agentic` repository at `3386899`, relative to `ccp/` unless a path says otherwise.

1. Current architecture assessment

CCP is a 21,515-line Python build of the whole compliance loop, written from `spec.md` v1.3 and run against two fixture tenants. Its core already has the shape an audit-grade product needs: a write-once raw zone, append-only observations, a bitemporal state table, and "models extract, code decides". Its edges are teaching-scale by design: three connectors with one live client, ten tests, a command line, one polling worker, and AI quality that has

never been measured against a real model. The target keeps the core and rebuilds the edges. The first report (Section A) holds the line-level evidence. Code was read on 9 Nov 2026 at `agentic` `3386899`.

Dimension	CCP today
Tenants	2 fixture tenants in `config/tenants.yaml`
Connectors	GitHub (fixtures and a live REST and GraphQL client), a mock HRIS, manual uploads (Markdown, text, text-layer PDF)
Tests and controls	10 tests (7 deterministic, 2 extract-then-rule, 1 fixed workflow) behind 9 controls, mapped to a 6-criterion SOC 2 slice and an 8-control ISO 27001 slice
Agents	Tier-3 evaluator, Investigator, Policy Agent and Copilot, all LangGraph, sharing 13 read-only tools
Stack	Python 3.11; Postgres 17 with pgvector and forced row-level security; LangGraph with PostgresSaver; Anthropic API (`claude-haiku-4-5`, `claude-sonnet-5`); Voyage embeddings; Langfuse Cloud
Size	21,515 lines in `ccp/`, 15 SQL migrations, 383 test functions
Interface	Command line and a terminal Copilot; no API, UI or user identity

1.1 One workflow end to end

The flagship case: a person terminated in the HR system still has GitHub access.

Step	What CCP does	What holds up	What is missing
Connect	Reads a token from `GITHUB_TOKEN`; tenants come from a YAML file	One `pull()` contract for fixture and live sources	OAuth, stored credentials, token refresh, connection health and schedules (`connectors/base.py:49-54`)

Step	What CCP does	What holds up	What is missing
Collect	A full pull every run: 7 REST endpoints and one GraphQL query	Provider ids and timestamps are kept; the payload is untouched	Retries, backoff and rate limits; any non-404 error aborts the pull mid-stream (`github/client.py:55-60`)
Store	The raw file is written once; `raw_events` dedups on the provider's event id; only new rows reach `event_log`	A replayable record, append-only by database grant	Local disk only; no completeness signal per pull
Normalize and link	Pure mappers append `Person` and `AccessGrant` observations; HR and GitHub accounts link into one person, and unclear pairs go to review	Every identity comparison is logged with its features and thresholds	Live mode cannot see a revoked grant (`github_mapper.py:303,321`); unchanged member, branch-protection, `SECURITY.md` and file-listing snapshots re-land on every pull (`github/client.py:129`, `:179`, `:189`, `:222`)
Evaluate	Each change maps to (test, subject) pairs, coalesced for 60 s; one verdict per `evaluation_key`	The tier is fixed when a test is written; an infrastructure failure parks or dead-letters, never becomes a verdict	The mapping is hard-coded (`evaluators/impact.py:149-209`); only full runs resolve identities and refresh the index, so change-driven runs skip both (`compliance_state/run.py:65-67`)
Record state	One bitemporal `compliance_state` row per verdict, under a per-subject lock	Separates "what was true" from "what we believed"	A verdict is committed, then announced in a second transaction; a crash in between loses it (`evaluators/runner.py:93-106`)
Investigate	Each fail or `needs_review` goes to the Investigator graph	Read-only tools; budgets on tokens, dollars, time and tool calls	Tool results reach the model unfenced and unscanned (`llm/client.py:126-140`)

Step	What CCP does	What holds up	What is missing
Approve and fix	One append-only approvals table; dry run by default; one live action behind five gates; a re-pull closes the loop	A payload hash pins what was approved	`reject` or a typo approves (`approvals/cli.py:88-90`); nothing checks the hash on resume; rollback skips the gates (`remediation/runner.py:243-289`); reviewers are typed names

1.2 Assessment by area

Area	CCP today	Verdict	Why
Connectors	Three connectors; `pull()` is the whole contract	Rebuild the contract	Breadth needs a connection lifecycle, cursors, completeness, rate limits and a conformance harness first
Raw zone and ingestion	Write-once files on local disk; dedup on provider ids; an event log	Keep; move to object storage with a retention lock	The replayable record is what replay and auditors rely on; the stored hash covers CCP's re-serialization, not the bytes received (`ingest/raw_zone.py:62`), so the move should hash the response bytes and keep the provider's request id
Background work	A Postgres table as a queue: ordered and parallel topics, 5 attempts, dead-letter, parking; worker processes that each poll one consumer (`cli.py:552-563`)	Harden, or move to a durable engine (Section 3)	Safe only as one process; one bad record stalls an ordered topic (`ingest/consumers.py:61-72`)
Normalization and identity	Pure mappers; append-only observations; logged identity scoring with a review queue	Keep	Add an identity provider as a source and a review screen

Area	CCP today	Verdict	Why
Evidence and state	Insert-only grants on history tables; bitemporal `compliance_state`	Keep	Answers "what did we believe on audit day"
Knowledge	9 controls and two framework slices in YAML	Grow	Needs a versioned common-control catalogue and mapping review
Evaluation	Three tiers fixed at authoring time; pure rules decide	Keep the tiers; harden citations	Citation checks confirm presence, not support (`evaluators/citations.py:42-47`)
Agents	Four graphs, per-graph tool allowlists, the tenant bound in code, budgets	Keep the contract	Injection handling only detects; quality is unmeasured
Approvals and remediation	One append-only table, a payload hash, dry run, five gates	Keep the design; fix the defects	Fails open on an unknown decision; no authenticated reviewer
Evals and observability	Golden, judge and adversarial suites; a JSONL step log; Langfuse; replay	Keep; run them for real	All 27 recorded eval runs are local, git-ignored files in which every model-backed case was skipped for lack of an API key; CI has never run because the repo has no remote; replay breaks once a cited document changes
Security and tenancy	`TenantContext`, `ScopedDB` and forced row-level security on every tenant table, vectors and checkpoints included; 21 guardrails, each with a test	Keep	Upload paths are not contained (`manual/uploads.py:88`); secrets are redacted only in prompts
Product surface	Command line and terminal Copilot; Postgres in Docker; tenants in YAML	Build	No API, UI, user accounts, RBAC, onboarding or cloud deployment

1.3 What to keep, and what limits growth

Keep: raw capture with provider ids and timestamps; append-only observations and verdicts keyed by `evaluation_key`; the bitemporal state table; extract-then-rule with pure rules; "graphs read, runners write"; per-graph tool allowlists with the tenant bound in code; budgets on every run; failure handling in which an outage parks or dead-letters work instead of producing a verdict; approvals pinned by a payload hash; three layers of tenant isolation; and guardrail and eval suites tied to tests.

What limits growth, most severe first:

1. **Live evidence is not trustworthy yet, and CCP cannot see what it has not assessed.** The connector contract has no cursor, completeness signal or snapshot semantics, so revocations are invisible and unchanged data re-lands on every pull. The live client also reads any 404 as "nothing there" (`connectors/github/client.py:55-60`), and GitHub answers 404 rather than 403 to a request not authorized for a private resource (GitHub), so a terminated person whose grants CCP could not see passes GH-1 with "no GitHub grants on record" (`evaluators/deterministic/gh1.py:42-49`). The rollup counts a test as pending only when it has no state rows at all (`knowledge/rollup.py:76-78`), so a subject never evaluated, or whose re-evaluation parked or dead-lettered, drops out of view while its old PASS keeps counting.
2. **Citation checks confirm presence, not support.** On the repo's own fixtures, a training record marked "In progress" verified as completed and passed, and tier 3 fixes retrieval quality at 1.0 (`agent/graphs/tier3.py:189`).
3. **The approval path can fail open.** An unknown decision approves, no resume checks the payload hash, rollback bypasses the five gates, and reviewers are typed names.
4. **Background work is safe only as one process.** A verdict can be lost between two transactions, one bad record stalls an ordered topic, and parallel consumers could skip rows or replace a fresh verdict with a stale one.
5. **AI quality is unmeasured.** No eval has run against a real model, and the nightly gate would block on noise (5.2).
6. **Injection handling only detects.** Tool results reach the model unfenced and are cut at 20,000 characters, possibly mid-JSON, without the model being told (`llm/client.py:136`); detection is 15 fixed English phrases (`security/prompt_guard.py:28-45`).
7. **Replay and change propagation have gaps.** Replay breaks once a cited document changes, and tool loops cannot be replayed because only their first request and last response are kept (`agent/core.py:328-330`). Evaluators read the latest observations at processing time and cite entity ids, so a verdict cannot be re-derived from what it saw (`evaluators/context.py:116-129`). Verdicts store no rule version, and because

the evaluation key omits it, a corrected rule run over unchanged data is dropped as a duplicate until the data changes or the 30-day revalidation runs (`evaluators/verdict.py:72-90`, `:107`; `evaluators/runner.py:200`; `evaluators/clock.py:61`). The impact mapping is code, and one daily model budget also parks deterministic tests.

8. **There is no product surface.** No API, UI, authenticated users, connection onboarding or cloud deployment.

2. Competitive technical research

The six incumbents share one architecture: a current-state store fed mostly by scheduled syncs, deterministic tests over it, and models around the pass or fail decision rather than inside it. In 2026 all of them shipped agents, and the best-documented teams converged on rules CCP already follows: tools designed around information, verifiers inside the loop, and proposals that wait for approval. They differ on evidence integrity and on how far agents may write. None publicly documents an immutable event ledger, bitemporal state or citations checked in code, and the open-source code of the leading AI-native entrant shows how an agent product gets approvals and result integrity wrong.

Linked statements were checked against the source on 9 Nov 2026. "In our reading" marks inference. Vendors' internal systems are private, and nothing here states one as fact unless the vendor published it.

2.1 How the incumbents are built

Company	Data, sync and checks	Agents and autonomy	Quality and evals
Vanta	400+ integrations by its count. Tests read Vanta's stored copy of synced resources (tests); test snapshots keep raw JSON and, for API tests, the requests made (snapshot API). MongoDB Atlas is the primary database (Vanta)	One general agent with 280+ tools (75k+ tokens) on a model-agnostic harness that runs on Gemini; loading tools by domain raised task completion 5% and cut cost per run 48% (Vanta, Sep 2026). Every product action is one authorized service method exposed to REST, CLI, agent and MCP (Vanta, Aug 2026). Custom agents stop at "Needs approval" (help); Auto Mode never pre-approves destructive actions (updates)	An evidence verifier checks admissibility and adequacy (Vanta, Aug 2026); adding it to the browser agent doubled task completion (Vanta, Jun 2026). Mapping precision rose from below 35% to 78%, and judge agreement with experts from about 65-70% to 85-90% (Vanta, Jun 2026)
Drata	Tests run daily, and each run is stored with raw JSON (monitoring). Custom tests are JSON rules; AI summaries of failures are "assistive, not authoritative" (AI summaries). A 2025 post describes a database partition per customer (Drata)	The vendor-risk agent shows source excerpts and asks before every write (TPRM Agent). Autonomy presets exist, and it withholds a recommendation when too many criteria are inconclusive (autonomy). The mapping agent proposes with confidence; nothing applies until a user maps (mapping)	An offline LLM pipeline wrote 1,000+ deterministic cloud tests, each validated against Drata's own test API (up to 3 attempts) and reviewed by auditors, at about $20-30 per resource (Drata, Dec 2025). Braintrust is used for offline evaluation only (trust page)

Company	Data, sync and checks	Agents and autonomy	Quality and evals
Secureframe	Daily syncs. Tests evaluate only the latest sync, and no integration test history is kept in the app (connections). Changes to built-in tests ship as versioned updates after a countdown (updates)	An MCP server with 100+ tools including create, update and delete, and no read-only mode (MCP). Cloud fixes are copy-paste IaC and CLI (remediation)	AI evidence validation is advisory only (validation)
Sprinto	Ingestion "plans" (transform, map, insert, evaluate) into customer-definable entity types (ingestion); most checks refresh every 4 to 12 hours (FAQ)	Playbooks give each workflow a charter (goal, scope, guardrails) and ordered agent steps that can pause at approval forms (Jul 2026). A browser extension changes customer consoles through the user's session after confirmation (Sprinto Assist)	Questionnaires run in passes: verbatim fact extraction, answer direction, then writing, with a grounding check that can retry once; 0.995 faithfulness on an internal 30-question benchmark (Aug 2026)
Scrut	Direct API connectors with 24-hour syncs (example); vendor-authored tests from a CIS-based library (tests); every REST write needs an `Idempotency-Key` (API)	Agents propose rather than act (Teammates); MCP has 19 read tools and 3 narrow writes (MCP)	A validation agent reproduces each vulnerability finding before publishing it as likely true or false positive (Sep 2026)
Thoropass	Evidence belongs to one audit, and evidence from previous audits "can never be used" (help)	MCP writes only comments, attachments and status on evidence requests (MCP)	Its own CPA firm audits; LLMs draft first-pass test results that auditors review and own (Thoropass, Sep 2026)

What makes each effective, in our reading: Vanta's discipline of verifiers before autonomy and tools shaped around information; Drata's use of AI offline to write deterministic tests, plus a rule to withhold judgement when evidence is thin; Sprinto's separation of fact extraction from answer writing; Scrut's reproduce-before-publish; and Thoropass's evidence frozen per audit.

2.2 What open-source code shows

Two AI-native GRC products and one established open-source platform publish their code, which shows what docs do not.

Project	What the code shows	Lesson for CCP
Comp AI (`150fed5`, 4 Nov 2026)	No agent runtime: each feature calls the AI SDK directly with a hard-coded model (assistant). AWS remediation runs whatever SDK command the model names, guarded by a 13-command denylist that rollback skips (executor). Approval is the string "acknowledged", after which another AI call regenerates the fix steps (remediation). Failing checks are held as "inconclusive" and hidden from the customer until a self-heal agent decides (evaluation). Findings are reconciled only from complete, successful scans (reconciliation)	What not to do: no denylists, no regeneration after approval, and no AI deciding whether a customer sees a failure. The complete-scan rule matches the sync contract CCP needs
Probo (`7bbe06a`, 9 Nov 2026)	Its own Go agent runtime: Postgres `SKIP LOCKED` claims with leases and heartbeats, a checkpoint after every tool turn, and approval interrupts whose decisions must cover exactly the pending tool calls (service). No production agent enables approvals or guardrails (Slack agent); MCP tools take `organization_id` from the model (resolver); there is no row-level security (scope)	Copy the interrupt design. Make approval mandatory per tool class, not opt-in; bind the tenant on the server and keep row-level security underneath

Project	What the code shows	Lesson for CCP
CISO Assistant (`a867086`, 9 Nov 2026)	"Proposals, not actions": the assistant never writes to the database (architecture). Its MCP read-only profile does not register write tools at all (45 of 102), and an action ledger tracks proposed, approved, rejected and expired actions. It ships evals for tool-selection accuracy against tool-surface size and gold sets per framework (MCP server)	The open design closest to CCP. Copy the hard read-only MCP profile and the tool-surface eval

2.3 AI-native entrants and adjacent agent companies

AI-native GRC vendors document their agents loosely. The clearest engineering comes from agent companies in law, customer service, finance and healthcare, and it converges on CCP's own design. State, side effects and final decisions stay in code or an approved policy; models read, extract and draft. These companies add three things CCP lacks: a policy layer for consequential actions, versioned releases gated by evals, and calibrated monitors in production.

Company	What is technically different	What it shows for CCP
Anecdotes (GRC)	Deterministic "analysis rules" run over evidence normalized from 230+ systems; agents react to rule outcomes and re-run the rule after a fix; a recipe drafts new rules. Approvals, permissions and evals are not documented (Agent Studio)	Models can draft rules that code then runs; a fix is confirmed by re-running the test, not by the agent's report
Trustero (GRC)	Publishes "p90 consistency across repeated control evaluations" of 92-93%, with no dataset or method (a vendor claim) (Trustero)	Verdict stability is a quality buyers can see; CCP should define it (pass^k) and publish the method

Company	What is technically different	What it shows for CCP
Complyance, Mycroft (GRC)	Complyance's pages contradict each other: agents "execute actions autonomously" versus "All actions are surfaced for human approval". Mycroft's agents change customer systems, with approval stated only for configuration drift (Complyance)	Autonomy in AI-native GRC is mostly undocumented; a precise approval model is a credibility advantage
Conveyor (questionnaires)	MCP write tools limited by the capabilities granted to each key; low-confidence or high-risk answers routed to reviewers by question type (May 2026)	If CCP ever offers MCP writes, scope them per credential
Hyperproof	Deterministic tests for structured data and AI for documents; its help page warns that an unresolved confirmation "might be applied to the next work item" (help)	Approvals must bind to the exact payload, as CCP's hash does
Fieldguide, Optro (audit side)	Auditor-side agents run engagement phases; Fieldguide's reviewer "confirms every conclusion traces to its evidence" (Fieldguide). Optro (formerly AuditBoard) bought Midship, whose agents test raw client evidence (May 2026)	Auditors' agents will check traceability, so CCP's exports need stable ids, hashes and citations from each conclusion to raw evidence
Harvey (legal)	A tool-call policy engine outside the model: permit, deny or ask the user; it fails closed, workspace settings can only tighten it, and tool definitions are pinned (Oct 2026). Citations point to sentence indices instead of fuzzy-matched text, and attorneys preferred the result four times as often (Apr 2026)	One deterministic policy point for all tools; citations as pointers that code renders and checks

Company	What is technically different	What it shows for CCP
Sierra (customer service)	Each release is an immutable snapshot of code, prompts, model versions and knowledge, with rollback (Sierra). Supervisors switch from observe to intercept on high-risk topics (monitoring). On its τ-knowledge benchmark the best model scored 37.4% on one try and 20.6% across four (May 2026)	Verdicts should name a release bundle; blocking checks belong only where output is irreversible; agents that must search documents and act still fail often
Ramp (finance)	Agents answer approve, reject or needs review instead of giving a confidence number; deterministic limits sit on top; the total exposure an agent can create is capped by a budget (trust, risk)	Cap automated status changes per tenant; never treat users' acceptance of a verdict as ground truth
Decagon, Anterior, Parcha	Decagon keeps critical actions in code beneath natural-language procedures (Decagon). Anterior orders human review by risk and compares judge and human decisions continuously (talk). Parcha found that separating extraction from judgement improved document checks (Parcha)	Independent confirmation of extract-then-rule, plus a production review loop CCP lacks

A new standard is also forming: AIUC-1 certifies AI agents after 1,000 to 5,000 red-team scenarios and an accredited audit, with quarterly retests.

2.4 Where CCP stands

Capability	CCP today	Best public practice	CCP's position
Integration breadth	3 connectors, 1 live	400+ (Vanta); 590 in Comp AI's catalogue, with long-tail checks kept as a versioned JSON DSL in the database	Far behind. Breadth needs a connector SDK and declarative checks
Sync semantics	Snapshots only; a full pull each run	Push APIs that replace per type or per session (Vanta, Drata, Sprinto); reconcile only from complete scans (Comp AI)	Behind. Needs a completeness contract
Evidence history	Write-once raw zone, append-only verdicts, bitemporal state	Run logs and snapshots (Vanta, Drata); evidence frozen per audit (Thoropass)	Ahead in design, unproven at scale
Check logic	Pure rules over typed entities; tier fixed per test	Deterministic rules over stored data; AI-written tests validated offline (Drata)	At par. Adopt offline generation
AI output checks	Citations checked in code, for presence only	Verifiers scored against experts and auditors (Vanta); grounding checks (Sprinto); no machine-checked citations documented	A lead once support checks land
Agent autonomy	Graphs read, runners write; one approval stage that fails open today	Approval before every write (Drata); destructive actions never pre-approved (Vanta); proposals only (CISO Assistant)	Strongest contract on paper; defects to fix

Capability	CCP today	Best public practice	CCP's position
Agent interface	Internal tools only	Hosted MCP, from 3 narrow writes (Scrut) to 100+ full-write tools (Secureframe); a hard read-only profile (CISO Assistant)	Behind. Generate MCP from the tool registry, read-only first
Evals	Golden, judge and adversarial suites, never run on a real model	Published numbers only from Vanta and Sprinto	Behind until run. Published results would be rare in the category
Tenant isolation	Forced row-level security on every tenant table, vectors and checkpoints included	A partition per customer (Drata); Vanta reported a cross-tenant leak after a filter was removed (RCA)	Ahead in design
Background work	Postgres queue with `SKIP LOCKED`, dead-letter, parking and budgets; safe for only one process per topic today	Temporal in a Drata job posting; Hatchet on Vanta's subprocessor list	Behind on durability and scale
Remediation	Dry run; one live action behind five gates	Draft PRs through the customer's coding agent (Vanta, docs); copy-paste fixes (Secureframe); console changes through a browser extension (Sprinto); model-chosen AWS commands under a denylist (Comp AI)	Safest design, least shipped

Incumbents lead on breadth and product surface. CCP's possible lead is evidence integrity and agent contracts, which matter more as buyers and auditors scrutinize AI after the 2026 allegations against Delve, which Delve denies.

3. AI agent architecture research

The teams that publish most about production agents (Anthropic, OpenAI, Google, Cognition, LangChain and HumanLayer) agree on four points. Code owns the control flow. Models get a few well-designed tools. The durable record lives outside the model's context. Consequential actions pause for a person. CCP's design already follows all four, so the research mostly confirms it and shows where it must harden. Much of the evidence comes from coding and research agents on Claude models, the family CCP uses, but most of it is the vendors' own internal measurement. Evals and guardrails are covered in Section 5, and 6.4 to 6.11 turn the newest practice from frontier labs and agent companies into a decision for each layer.

In the tables, the middle column is checked against the linked page (read 9 Nov 2026), and the last column is our proposal for CCP.

3.1 Workflows first; agents only where the path is unknown

Lesson	What the source shows	Proposed for CCP
Start with the simplest design	Anthropic separates workflows, where code fixes the path, from agents, where the model directs its own steps. It recommends "the simplest solution possible" and stopping conditions such as an iteration cap (Anthropic, Dec 2024)	Keep CCP's split. Tiers 1 and 2 and the tier-3 workflow stay fixed paths; only the Investigator, Policy Agent and Copilot loop, inside the budgets CCP already enforces
Use agents for judgement and unstructured input	OpenAI reserves agents for complex judgement, rules that are hard to maintain, or heavy unstructured data, and names vendor security reviews as an example (OpenAI guide, undated)	Vendor reviews, questionnaires and review of uploaded evidence qualify. Checks over structured API data stay deterministic

Lesson	What the source shows	Proposed for CCP
Orchestrate in code	Orchestration by code is "more deterministic and predictable, in terms of speed, cost and performance" (OpenAI Agents SDK docs). Production agents are "mostly deterministic code, with LLM steps sprinkled in" (HumanLayer 12-factor agents)	CCP already does this: a runner, not the graph, persists every output. Keep it as a hard rule
Mix code and model steps in one graph	Google's ADK 2.0 added graph workflows that combine agent nodes with deterministic nodes (ADK)	CCP's LangGraph graphs already mix the two, so no framework change is needed for this

3.2 One writer: where several agents help

Finding	What the source shows	Proposed for CCP
Parallel readers help breadth, at a price	An orchestrator with worker agents beat a single agent by 90.2% on Anthropic's internal research eval, using about 15 times the tokens of a chat. It fits poorly when every agent needs shared context (Anthropic, Jun 2025)	Consider it only for read-heavy breadth such as audit prep or vendor research, and only after measuring the single Investigator on the same task. The eval and its data are not public
Parallel writers conflict	"Actions carry implicit decisions, and conflicting decisions carry bad results" (Cognition, Jun 2025). In 2026 Cognition found that in the multi-agent patterns that work, "writes stay single-threaded" (Cognition, Apr 2026)	Keep "graphs read, runners write": code is the only writer of verdicts, state and approvals

Finding	What the source shows	Proposed for CCP
Generators grade themselves too kindly	"agents reliably skew positive when grading their own work". A separate, sceptical evaluator was easier to tune. The full setup cost $200 against $9 for a solo run, and only the full setup produced a working app (Anthropic, Mar 2026)	Check AI conclusions in code first (citations, rules), then with a reviewer that starts from a clean context, for high-value outputs only. CCP's judge runs on the same model as tier 2 (`claude-haiku-4-5`); give it a different model: a different Claude model, inside the zero-retention organization, for anything that reads customer evidence, and a different model family only on fixture and synthetic suites (Section 5). Anthropic calls it "best practice to evaluate with a different model from the one used to generate the evaluated output" (Anthropic)
Autonomy needs a strong verifier	Sixteen parallel agents wrote a C compiler of about 100,000 lines over about 2,000 sessions for about $20,000. It worked because the test suite was a near-perfect check (Anthropic, Feb 2026)	Compliance has only partial verifiers (rules and citation checks), so autonomy stays low. CCP's five remediation gates are the right default

3.3 Tools are the contract between code and model

Anthropic describes tools as "a contract between deterministic systems and non-deterministic agents" (Anthropic, Sep 2025). The published practice:

- **Few, task-level tools.** Not one wrapper per endpoint. Use namespaced names, meaningful identifiers instead of UUIDs, concise or detailed response modes, pagination and actionable errors (same source).
- **Arguments that make mistakes hard.** Requiring absolute file paths removed a whole class of errors on SWE-bench (Anthropic, Dec 2024).
- **Expect wrong tools and wrong parameters.** These are the most common failures. Loading tool definitions on demand cut their context by 85% and raised Opus 4 from 49% to 74% on MCP evals; usage examples raised accuracy on complex parameters

from 72% to 90% (Anthropic, Nov 2025). These are Anthropic's internal figures on Claude.

- **Code execution saves tokens but needs a sandbox.** Having the agent write code that calls tools cut one workflow from about 150,000 tokens to about 2,000, at the cost of a sandbox, resource limits and monitoring (Anthropic, Nov 2025).
- **Rate every tool's risk.** Low, medium or high, by write access, reversibility, permissions and financial impact (OpenAI guide).
- **MCP is now stateless and stricter.** The 2026-07-28 revision removes sessions (changelog). Servers "MUST NOT accept or transit any other tokens" (authorization), and clients must treat annotations such as `readOnlyHint` as untrusted (tools). MCP moved to the Linux Foundation's Agentic AI Foundation in Dec 2025 (MCP blog), and the official registry is still a preview for public servers only (registry).

Proposed for CCP. CCP's tool module is already close: the tenant comes from a closure, extra arguments are rejected, and off-list calls get an error result. Grow it into one typed registry that is the source of truth. Each tool declares Pydantic input and output schemas, a class (read, propose or act), scopes, a rate limit and a timeout. Tools return evidence by id, version and hash, require explicit ids and as-of times, and report missing data as an error, so an empty list is never read as compliant. The same registry generates the agents' tool specs and a customer-facing MCP server. Read-only behaviour is enforced in the gateway, never by annotations. Credentials stay in a broker that neither the model nor any sandbox can reach, as in Anthropic's Managed Agents, which fetches them from a vault through a proxy (Anthropic, Apr 2026).

3.4 Context and memory: the record is not the prompt

- Anthropic treats context as "a finite resource with diminishing marginal returns". It loads data just in time by identifier, clears old tool results first when compacting, keeps notes outside the window, and has sub-agents return summaries of 1,000-2,000 tokens (Anthropic, Sep 2025).
- Google's ADK rebuilds every prompt from durable stores: "Context is a compiled view over a richer stateful system" (Google, Dec 2025).
- LangChain's Deep Agents writes the original messages to files as "a canonical record" before it summarizes them (LangChain, Jan 2026).
- Anthropic's memory guidance warns that "an incorrect note can be carried into every session". It keeps injected memory small and re-checks notes against the source transcripts; it does not cover memory poisoning or multi-tenant isolation (Anthropic, Nov 2026).

Proposed for CCP. CCP's step log already records each step with OpenTelemetry GenAI attribute names, and replay rebuilds a verdict from it. Promote that log from a JSONL file to the system of record: an append-only run log in Postgres under row-level security, holding the inputs, each tool call and result, the exact model request and response, and the prompt, model, tool and harness versions. The prompt is a view rebuilt from that log on each call, so compaction can never destroy what the agent saw. Stable content (instructions, the control catalogue) comes first so it can be cached; tenant evidence comes last, by reference. When the Copilot gains memory, keep it per tenant under row-level security, with its source and a checked or unconfirmed flag. Memory may steer retrieval but never counts as evidence.

3.5 Durable execution and approvals

Agent frameworks leave durability and approval security to the application:

- LangGraph re-runs the whole interrupted node on resume ("the runtime restarts the entire node from the beginning"), so a model call in the same node as an interrupt is sampled again (LangGraph).
- The OpenAI Agents SDK warns that "Approval permits a tool call to execute; it does not guarantee exactly-once execution" and that "Possession of a run ID or decision ID is not authorization" (OpenAI Agents SDK).
- Anthropic's Managed Agents keeps no state in the harness. Every event goes to an append-only session log, and a crashed harness resumes from it (Anthropic, Apr 2026).

CCP's own queue already gets the hard parts right: idempotent verdicts keyed by `evaluation_key`, dead-letter, parking, and model errors classified so an outage never becomes a verdict. Four gaps sit beneath any engine choice, and each is verified at `3386899`:

- The verdict and its announcement commit in two transactions, so a crash between them strands the verdict (`evaluators/runner.py:93-106`).
- A tier-3 approval is recorded only after the resumed graph and the verdict commit succeed, so a model outage at that moment loses the reviewer's decision (`approvals/actions.py:51-53`).
- A failed record becomes claimable again at once, with no backoff, so a burst of rate-limit errors can use up all five attempts in seconds (`ingest/event_log.py:110-115`).
- Tier 3 starts a fresh thread on every attempt, so a re-delivery repeats the whole run and its model spend (`agent/tier3.py:157-158`).

Fixing these four gaps keeps CCP on its own queue, which is the recommendation; an engine comes in only when a named trigger fires. The engines that could take over, for a small Python team on Postgres:

Option	What the source shows	Proposed for CCP
DBOS Transact	An MIT library with no server: checkpoints, queues and schedules live in Postgres (architecture). A transaction can commit an application write and DBOS's record of it together (transactions). Queues can be partitioned by a key such as the tenant (queues), and schedules fire once across processes in a named time zone (schedules). It recovers only workflows that match the running code version (upgrading), and its FAQ says recovery across several processes needs its paid Conductor, which its recovery page contradicts (FAQ)	The leading candidate if the hardened queue falls short. Run its spike in the medium term (kill a worker mid-step, overlap two deploys) and adopt it only if the spike passes and a trigger fires: a workflow with more than three durable steps across external systems, two or more queue incidents a quarter, or more than about 20 worker processes (Section 9, Decision 4). If adopted, use its portable JSON serializer rather than pickle, pass only ids as inputs, because its tables carry no row-level security, and give its system database a direct connection, because its FAQ says PgBouncer works "only in session mode" (FAQ). LangGraph is not among its documented integrations (AI quickstart), so CCP's graphs would keep their own checkpointer

Option	What the source shows	Proposed for CCP
LangGraph with PostgresSaver	Checkpoints save asynchronously by default, so a crash can lose the last step (source). An encrypting serializer exists, and checkpoints pile up unless pruned (persistence). Self-hosting LangGraph's server needs an Enterprise licence (LangSmith)	Keep the MIT library for the agent graphs, with synchronous checkpoints, encryption, pruning on the evidence-retention schedule and a graph version on every thread. Skip the hosted server
Temporal	Workflow code runs in a sandbox that forbids I/O, so database and model calls must move into activities (sandbox). Version pinning does not work with rolling deploys (versioning). The LangGraph plugin is a 0.1.0 Public Preview that keeps checkpoints in memory (Temporal), and the generally available agent plugin targets OpenAI's SDK (Temporal). Cloud starts at $50 per million actions (pricing)	The upgrade path if agent work becomes long-running or multi-agent. Today it would mean rewriting CCP's agent core into activities and sending evidence into Temporal's history unless a payload codec encrypts it
Pydantic AI	Deferred tools end a run with pending approvals, and the application resumes it later with typed approve or deny decisions; the docs warn that "Approval is not an authorization boundary" (deferred tools). Its durable API is mid-migration, with the original wrappers due to be removed in v3 (DBOS page)	Borrow its approval semantics now; reassess it as a runtime after v3

Option	What the source shows	Proposed for CCP
Hatchet, Procrastinate, Restate, Claude Agent SDK	Hatchet is a separate engine with per-tenant fairness, but its cron runs in UTC only and skips ticks missed while it is down (cron). Procrastinate is an MIT Postgres task queue with heartbeats and periodic tasks, but no multi-step durability (docs). Restate advises against handlers that run for days (versioning). The Claude Agent SDK spawns a CLI process per session whose local files can leak between tenants in a shared container (hosting)	If the hardened queue falls short and DBOS fails its spike, Hatchet is the second choice and Procrastinate the fallback. Restate and the Claude Agent SDK do not fit CCP's evaluator and agent graphs

Inferred across these sources: every engine pushes towards short runs. DBOS counts a waiting workflow against its queue's concurrency limit, and recovers it only on the same code version; Restate advises against handlers that run for days; and Temporal can pin a long workflow to its code version only with rainbow deployments. So a review that takes days should be a row in a table, not a paused workflow.

Proposed for CCP: approvals as data, whatever the engine.

1. An agent run ends when it stores a proposal.
2. An approval is an authenticated, append-only row. The server checks the decision value, the reviewer's rights, the stored request and its payload hash, and rejects replays.
3. The approval starts a separate, idempotent apply step keyed by proposal id and payload hash, which checks the target system before acting.
4. Each model call is its own recorded step, so a resume never re-samples a model whose output fed a verdict.
5. A run is pinned to its harness version (prompts, graph, tools, budgets). A deploy that changes the version cancels and restarts any run still in progress, with the reason recorded. A stored proposal waiting for a person is not a run: it stays bound to its stored artifacts and their hashes and is never sampled again. Anthropic notes that scaffolding goes stale as models improve (Anthropic, Apr 2026), so the version is stored on every verdict.

This replaces CCP's tier-3 interrupt, where a LangGraph checkpoint waits for a CLI command and nothing re-checks the payload hash on resume.

3.6 Human approval that adds assurance

Anthropic found that users approve 93% of permission prompts. Claude Code's auto mode therefore lets read-only actions through, sends the rest to a classifier, and escalates to a person after 3 consecutive or 20 total denials; the classifier still misses 17% of real over-eager actions (Anthropic, Mar 2026). OpenAI brings in a person when failure thresholds are exceeded and for sensitive, irreversible or high-stakes actions, until the agent's reliability is established (OpenAI guide).

Proposed for CCP. Tier approvals by risk. Deterministic results publish without review; AI drafts that pass the verifiers get sampled review; any write to customer-visible state or to a customer's systems needs a per-item approval until it graduates under 4.4; status changes, mappings, external publication, merges and changes in a customer's cloud never graduate. Track how often reviewers approve without changes: a gate that approves everything adds no assurance and should be redesigned. Before any customer relies on CCP's approval stage, it needs authenticated reviewers and server-side validation of the decision, because today a typo approves.

3.7 Models, data terms and observability

CCP is Anthropic-first: tier 2 and the judge run `claude-haiku-4-5`, and tier 3 and the agents run `claude-sonnet-5` (`spec.md`, Stack). The platform has moved under both pins, and five of the changes below alter how CCP must treat models.

Topic	What the source shows	Proposed for CCP
Model lifecycle	The current models are Fable 5.2, Opus 5.5, Sonnet 5.5 and Haiku 5.5; CCP's Haiku 4.5 and Sonnet 5 are listed as legacy but still available (models). Haiku 4.5's commitment ("Not sooner than October 15, 2026") has passed, so it can now retire on at least 60 days' notice; Sonnet 5 is committed to at least 30 Jun 2027 (deprecations). Older aliases such as `claude-haiku-4-5` resolve to the latest snapshot; only dated ids are pinned (model ids)	Pin dated ids (`claude-haiku-4-5-20251001`) and store the model each response reports. Start the Haiku 4.5 to Haiku 5.5 migration now, through the eval gates. Budget about one forced migration per model tier per year

Topic	What the source shows	Proposed for CCP
Determinism	API version 2026-10-21 removes `temperature`, `top_p` and `top_k`; requests then sample at 1.0, and "No other setting replaces it" (migration). Infrastructure updates can shift behaviour even under a fixed model id (model ids). Python SDK 2.0.0 sends the new version by default (release notes)	Reproducible means rebuilt from stored artifacts, not re-run; consistency becomes a measured gate (pass^k, Section 5). CCP's lock file holds SDK 1.5.0 (`uv.lock:29-30`), but `pyproject.toml:17` accepts any later version, and CCP sends `temperature: 0` to Haiku 4.5 (`llm/anthropic_client.py:62-73`), so a routine lock upgrade to SDK 2.x would make every tier-2 and judge call fail with a 400 and dead-letter. Remove the parameter and pin the API version in code first
Typed output and citations	Structured outputs are generally available on every model and platform. Enabling the Citations feature together with structured outputs returns a 400 (citations)	Keep structured outputs plus quotes verified in code (5.4). Test native citations only as a separate quote-selection pass
Refusals	The 5.5, Opus 5 and Fable models run safety classifiers that end a response with `stop_reason: "refusal"`, and "Benign cybersecurity work can also trigger this category" (refusals)	Compliance evidence is security material. A refusal becomes needs_review with its category, never a verdict. Measure the refusal rate per test before migrating
Caching and batches	The minimum cacheable prompt is 4,096 tokens on Haiku 4.5 and 512 on the 5.5 models (caching). Batches cost half price but keep results for 29 days, so they are not eligible for zero data retention (batches)	CCP's spec records that tier-2 prompts never reach Haiku 4.5's minimum, so they are never cached (`spec.md` A-62); caching becomes a real cost lever after the migration. Use batches only for evals and other non-customer data
Cost tracking	CCP's own code: `price_for()` returns $0 for any model it does not know, which today includes `claude-haiku-5-5` (`observability/cost.py:43-50`)	Fix the price table before any model switch, or the per-run dollar caps never trip

Topic	What the source shows	Proposed for CCP
Data retention	By default Anthropic deletes API inputs and outputs within 30 days, and flagged content can be kept up to 2 years (privacy). Zero data retention excludes batches, the Files API, code execution, the MCP connector and Managed Agents, and the Fable models require 30-day retention (retention)	Run production evidence through a zero-retention organization with online calls only; keep evals and batches in a separate organization; state the terms in the customer agreement
Residency and failover	`inference_geo` offers only global or US (at 1.1x), and Haiku 4.5 rejects it (residency). Bedrock serves the same models but not Message Batches, and Amazon becomes the processor, with its own lifecycle (Bedrock). Priority Tier is no longer sold (service tiers)	Use Bedrock as the one secondary platform, for online calls; pause batch work during a failover. Log `usage.inference_geo` per call as location evidence. Treat the spend-cap 429, which carries no retry-after, as a reason to park work, not to retry
Embeddings	`voyage-3-large` has been legacy since 15 Jan 2026; `voyage-4-large` costs $0.12 per million tokens against $0.18 and supports 1,024 dimensions (lifecycle, pricing). Self-serve Voyage data may be used for training unless the organization opts out (MongoDB)	Re-embed with `voyage-4-large`, keeping the `vector(1024)` schema; opt out of training; list MongoDB as a subprocessor
Tracing	OpenTelemetry's GenAI conventions are still in Development and capture no message content by default (conventions). Self-hosting Langfuse means running ClickHouse; its cloud plans cost $29 to $2,499 a month (Langfuse, Terraform module)	Keep Langfuse Cloud as a view, sending production tenants' metadata only (ids, hashes, token counts, latency, cost and scores); full content goes only from the eval organization's synthetic runs. The run log stays the system of record (3.4). Pin the conventions version CCP emits

3.8 What the evidence does not settle

- Most of the agent evidence comes from the vendors' own internal evals, much of it on Claude models. That transfers to CCP better than to other model families, but none of it was measured on compliance tasks, so CCP must measure on its own.
- There are no public results for multi-agent designs on compliance tasks.
- No source says which reproducibility standard auditors will accept for AI-produced verdicts: replaying the stored output, or agreement across re-runs (pass^k), and at what threshold.
- No source publishes how often Claude verdicts flip across re-runs now that sampling runs at temperature 1.0, or how often the 5.5 models refuse benign security evidence. CCP must measure both on its golden set before it migrates.
- Three sources could not be opened: Google's agent whitepapers (Sep 2024 and Nov 2025) and OpenAI's "Harness engineering" post (Feb 2026).

4. Deterministic vs agentic decision framework

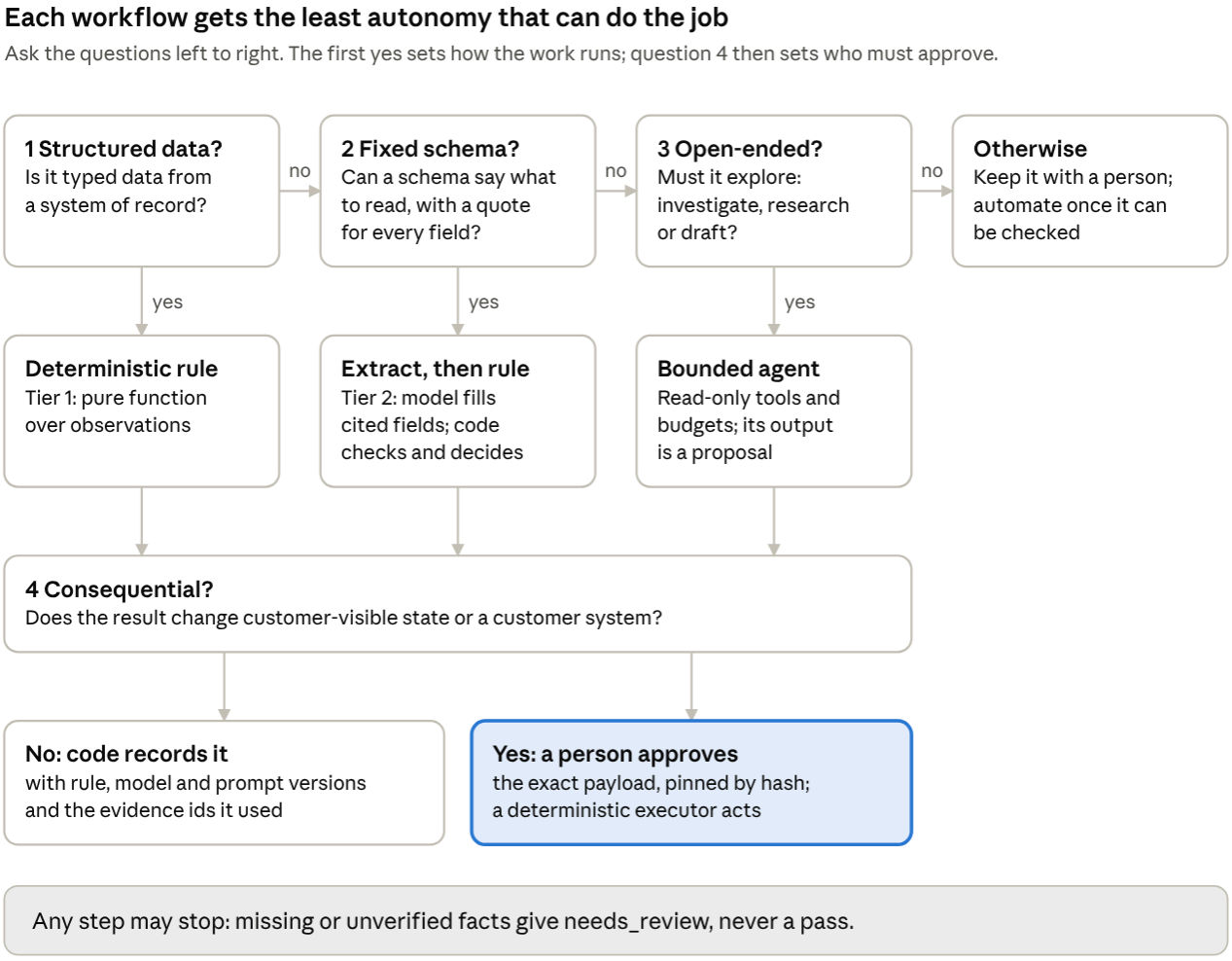
The thesis holds, with four refinements. Deterministic code should establish the facts, and it must also make the decisions. Facts a model reads out of a document are candidates until code verifies them. Agents should propose, not act. And every decision must record its method and be allowed to abstain. Refined: **code decides from verified facts; models read, extract, draft and investigate; people approve anything consequential; every decision records its method, and any step may abstain.**

4.1 The thesis, tested

Part of the thesis	Holds?	Refinement	Evidence
"Deterministic systems establish verifiable facts"	Partly	Code must also decide: a pass or fail is a versioned function over facts, so the rule version is part of the record	CCP's pure rules (1.2); Drata labels AI summaries "assistive, not authoritative" (2.1)
Facts come from systems	Not always	A field a model reads from a document is a candidate fact until code checks that its quote supports it	CCP's presence-only citation check (1.3); no incumbent documents machine-checked citations (2.1)

Part of the thesis	Holds?	Refinement	Evidence
"Agents interpret, investigate"	Yes, if bounded	Investigation and research are where agents earn their cost. Interpretation that sets a status ends in a rule	Anthropic and OpenAI guidance (3.1); Sprinto's staged questionnaire pipeline (2.1)
"and act on those facts within controlled boundaries"	No	Boundaries inside an agent are not enough. Agents propose; a deterministic executor acts only on an approved payload	Comp AI's denylist and regeneration after approval (2.2); "writes stay single-threaded" (3.2)
Not stated: method and abstention	Add	Every decision records its rule, model, prompt and evidence versions; insufficient facts give needs_review	Drata withholds a recommendation when evidence is inconclusive (2.1); pass^k (Section 5)

4.2 Choosing the execution model



Decision path · four questions, five outcomes, one abstain rule

Tiers 1 and 2 are CCP's existing evaluator tiers. Question 4 applies to every outcome, including deterministic ones: a rule that would publish to a customer still goes through the approval stage when its effect is consequential. Autonomy is earned later, by measurement (4.4).

4.3 Workflow by workflow

Workflow	Execution model	Why	Who decides
Integration setup	Deterministic	Connecting an account is a fixed protocol (OAuth or a cross-account role); a model adds risk and nothing else	The customer authorizes; code verifies the connection

Workflow	Execution model	Why	Who decides
API evidence collection	Deterministic	Structured data that must be complete, timestamped and reproducible	Code, with a completeness record per sync
Test execution	Deterministic (tier 1)	A rule over stored observations is cheap, explainable and replayable	A versioned rule
Identity matching	Deterministic scoring; people for the unclear band	The features are explainable, and a wrong merge can hide a terminated user	Code within thresholds; a reviewer between them
Policy interpretation	Extract-then-rule (tier 2)	The question fits a schema, and quotes can be checked	A rule over cited fields; a failed citation gives needs_review
Evidence review of uploads	Extract-then-rule plus a verifier	Admissibility breaks into checkable fields: dates, scope, issuer	A rule; a reviewer below the confidence threshold
Questionnaires	Extract-then-rule for answers; a person sends	Answers must cite evidence, and they go to a customer	Code checks citations; a person approves what is sent
Control mapping	Agent proposes, person approves	Judgement over catalogues; one wrong mapping affects every verdict that uses it	A mapping reviewer
Risk assessment	Agent drafts, person decides	Accepting a risk is a management decision with a named owner	The risk owner
Exception handling	Person decides; code enforces	An exception is an accountable decision with an expiry	A person; code enforces the expiry

Workflow	Execution model	Why	Who decides
Investigation	Bounded read-only agent	The path is unknown in advance; the output is a cited proposal	No decision: it changes no status
Audit prep	Code assembles; an agent lists gaps	The package must be reproducible as of a date; finding gaps is open-ended	Code builds the package; a person submits it
Remediation	Agent proposes; a person approves the payload; code executes	It writes to customer systems	A person approves; a deterministic executor acts
Connector drafting	Agent drafts offline; tests and a reviewer gate it	Generated code can be checked, as Drata's 1,000 generated tests show (2.1)	The conformance harness and a reviewer

4.4 Earning more autonomy

A workflow moves to lighter human review only on measured evidence, never by default:

- **Accuracy.** The false-pass rate on the expert-labelled set is below the agreed limit, with confidence intervals (5.2).
- **Consistency.** pass^k meets the target over repeated runs on the same evidence (Section 5).
- **Reviewer signal.** Blind, sampled re-review by an expert agrees with the workflow at the agreed false-accept bound. The rate of approvals without changes is tracked only to detect rubber-stamping, never as evidence of correctness (Ramp, 2.3).
- **Reversibility.** Only reversible actions graduate. Irreversible ones, such as revoking access or publishing to customers, keep per-item approval, as Vanta's Auto Mode does (2.1).

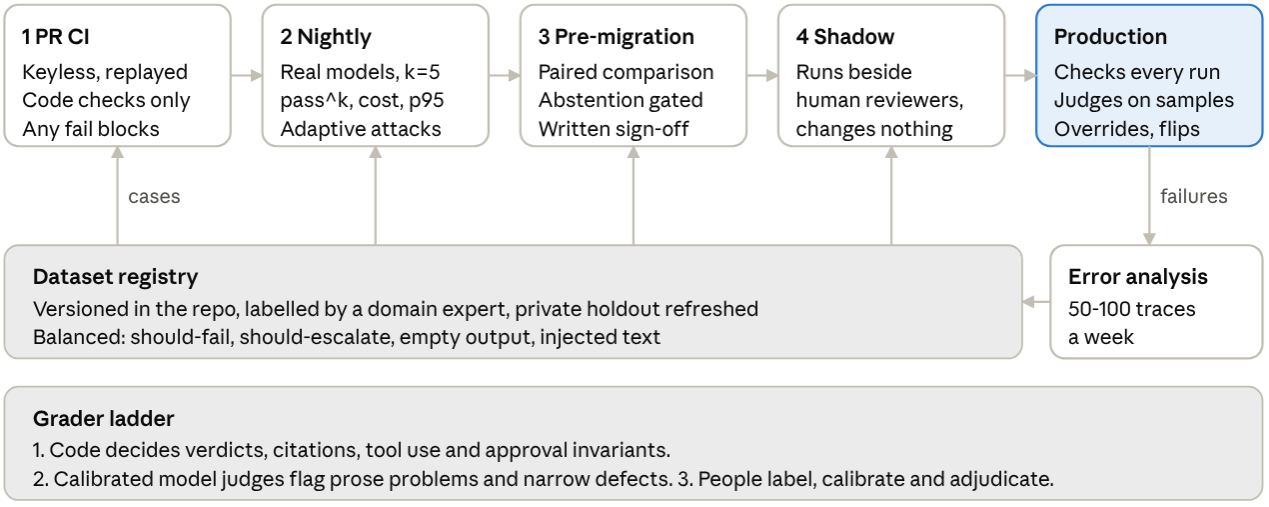
A regression demotes the workflow automatically: when a metric falls below its gate, review returns to the previous level.

5. Evals and guardrails

Both are built before any new agent ships, and both are enforced in code. Graders decide what passes, a policy gateway decides what an agent may do, and model-based defenses only raise flags. The research is consistent on why: published injection defenses fail against adaptive attackers, and model judges miss many defects, so neither can be the control that prevents a false pass. 6.6 adds what frontier practice in 2026 changes: harness edits treated as releases, judges that never see the agent's reasoning, evidence kept out of the user turn, and traces that carry no evidence text.

Every AI change passes four gates; production failures become new cases

Gates run left to right. The registry feeds every gate, and error analysis keeps it current.



Eval pipeline · four gates, production monitoring, and the loop back to datasets

A change to a prompt, model, tool schema or retrieval setting must clear all four gates. Production checks every run cheaply, judges a sample nightly, and sends confirmed failures back into the registry as new cases.

5.1 Graders and where each can be trusted

Grader	Use it for	Do not use it for	Evidence
Code assertions	Verdict against the expert label, final state, schema, citation span and typed value checks, tool allowlist, tenant scoping, no write before approval	Judging prose quality	Grade "what the agent produced, not the path it took" (Anthropic, Jan 2026)

Grader	Use it for	Do not use it for	Evidence
Model judges	Prose quality and narrow binary defect detectors, each with a reference answer, an "unknown" option and a judge from a different model (a different family offline; a different Claude model on customer data)	Verdicts and close calls	GPT-4 agreed with experts 85% of the time but held its answer under position swaps only 65% of the time (Zheng et al.); judges caught only 30-60% of inconsistent summaries (Yan)
Entailment checkers	A second line on narrative answers: contradicted claims are rejected, "not enough information" goes to review	Sole acceptance gate	The best grounded fact-checkers average about 75-80% on mixed benchmarks (LLM-AggreFact); NLI citation checks reach kappa 0.53-0.70 against humans (ALCE)
Human labels	Gold sets, judge calibration (100-200 labels per failure mode, TPR and TNR reported separately), weekly error analysis	Volume checks a computer can do	Below 60 labels the confidence intervals are too wide (Husain); trained annotators agree only moderately on attribution (AIS)

5.2 What to measure

- **Correctness against an expert label,** split into false PASS and false FAIL. A false PASS that reaches an auditor or a customer's trust page is the most expensive error, so it carries the highest cost weight.
- **Consistency.** pass^k, the chance that all k runs of a case succeed, plus verdict flips on unchanged evidence. In τ-bench, a model with over 60% single-run success fell below 25% at k = 8, and auditors would see that as flapping.
- **Escalation as a graded behaviour:** coverage, error among automatic decisions, over-escalation and under-escalation. Reasoning-tuned models abstain 24% less (AbstentionBench), so model upgrades must gate on under-escalation.
- **Grounding:** share of citations whose span verifies, share passing the typed value check, and entailment results.
- **Tool use:** right tool, right arguments (tenant and date ranges included), and invariants such as approval-skipped = 0.

- **Security:** attack success and utility under attack, per model, for static and adaptive attacks.
- **Cost and speed:** tokens, cost per completed task, p95 latency, and cost per tenant.

Statistics matter at this scale. Compare versions on the same cases with paired differences and standard errors clustered by tenant or source document, which can be over three times naive errors (Anthropic). Count errored and unscored runs as failures. CCP's current gate, which blocks on any single flip, would fire about 92% of the time on noise alone (1 − 0.95^50) with 50 cases that are each 95% reliable.

5.3 Gates through the lifecycle

Stage	What runs	Gate rule
Development	Capability suites that start with a low pass rate, plus error analysis	None; this is where quality is built
PR CI	Keyless deterministic and replayed cases; citation, schema and safety invariants	Any failure blocks
Nightly	Real models, 5 runs per case, regression suites, static and adaptive attacks	Block a release when the lower confidence bound drops below the agreed margin
Pre-migration	Full paired comparison whenever a model, judge, prompt, tool schema or retrieval setting changes	Written sign-off; under-escalation gated. Anthropic commits each model for about a year and then gives at least 60 days' notice (deprecations), so expect about one forced migration per model tier per year
Shadow	A new workflow or model runs beside human reviewers and changes nothing	Launch only at the agreed false-accept rate
Production	Cheap checks on every run, model judges on a nightly sample, 50-100 traces reviewed weekly	Alerts on confidence bounds, not point estimates

Seed each suite with 20-50 cases from real failures, then grow to 100-200 per failure mode, labelled by one named domain expert with a second annotator on about 20% for a human ceiling. Every suite needs negative controls: an empty, refused or do-nothing

output must fail. Flawed grading changed one benchmark score from 42% to about 95% (Anthropic), so graders are tested too. Vanta's pattern is worth copying: score the evidence verifier by agreement with the auditor's accept-or-flag decision (Vanta).

Keep datasets, graders, rubrics and thresholds in the repo in a vendor-neutral format. Hosted eval products are churning: OpenAI's Evals platform shuts down on 30 Nov 2026 (deprecations), and Google's recommended Gen AI evaluation SDK is still in Preview.

5.4 Citation verification, hardened

CCP's checker confirms that a quote exists, not that it supports the value: on the repo's own fixtures, a training record marked "In progress" verified as completed and passed. The target is a layered, fail-closed check:

1. **Provenance at ingest.** Every document version is hashed, and each citation is stored as a span selector: document version, hash, offsets and the exact text with context (W3C annotation model).
2. **Span match.** Exact match after normalization, or the same tokens in the same order. No bag-of-words fallback, and no ellipsis splicing across sentences.
3. **Typed value check, mandatory for every cited field.** Status values match a lexicon with an exclusivity rule and a label anchor; dates parse to ISO and must be equal; numbers and frequencies are compared in canonical units; negation cues route to review (negation is underrepresented in NLI data).
4. **Entailment as a second line** for narrative claims, judged by a different Claude model inside the zero-retention organization. A contradiction rejects the field; not enough information goes to review.
5. **Rules decide.** An unverified field becomes "needs review", never PASS and never "Not applicable".
6. **People review** with the spans highlighted, measured on 200-300 labelled field-span pairs that include splices, swapped dates and negations. CI allows zero false accepts on the deterministic classes.

5.5 Guardrails enforced outside the model

Control	Where it is enforced	What it stops	Basis
Tenant binding	The tool gateway takes the tenant from the authenticated session; agent tools use a read-only database role under row-level security	An injected or guessed id reading another customer's data	OWASP LLM06: authorize downstream, not in the model
Rule of Two per session	A session that has read untrusted content and holds private data loses act-class tools; a taint flag switches them off. Propose-class tools stay, because every proposal waits for a person, and the rule allows all three properties when the agent "requires supervision"	A vendor PDF steering an agent to flip a verdict or leak data	Meta, Willison, Google
Tool classes	Every tool is labelled read, propose or act by us, never by trusting a server's hints; strict schemas, budgets, rate limits	Tool misuse and runaway loops	MCP tools spec: annotations from untrusted servers are untrusted

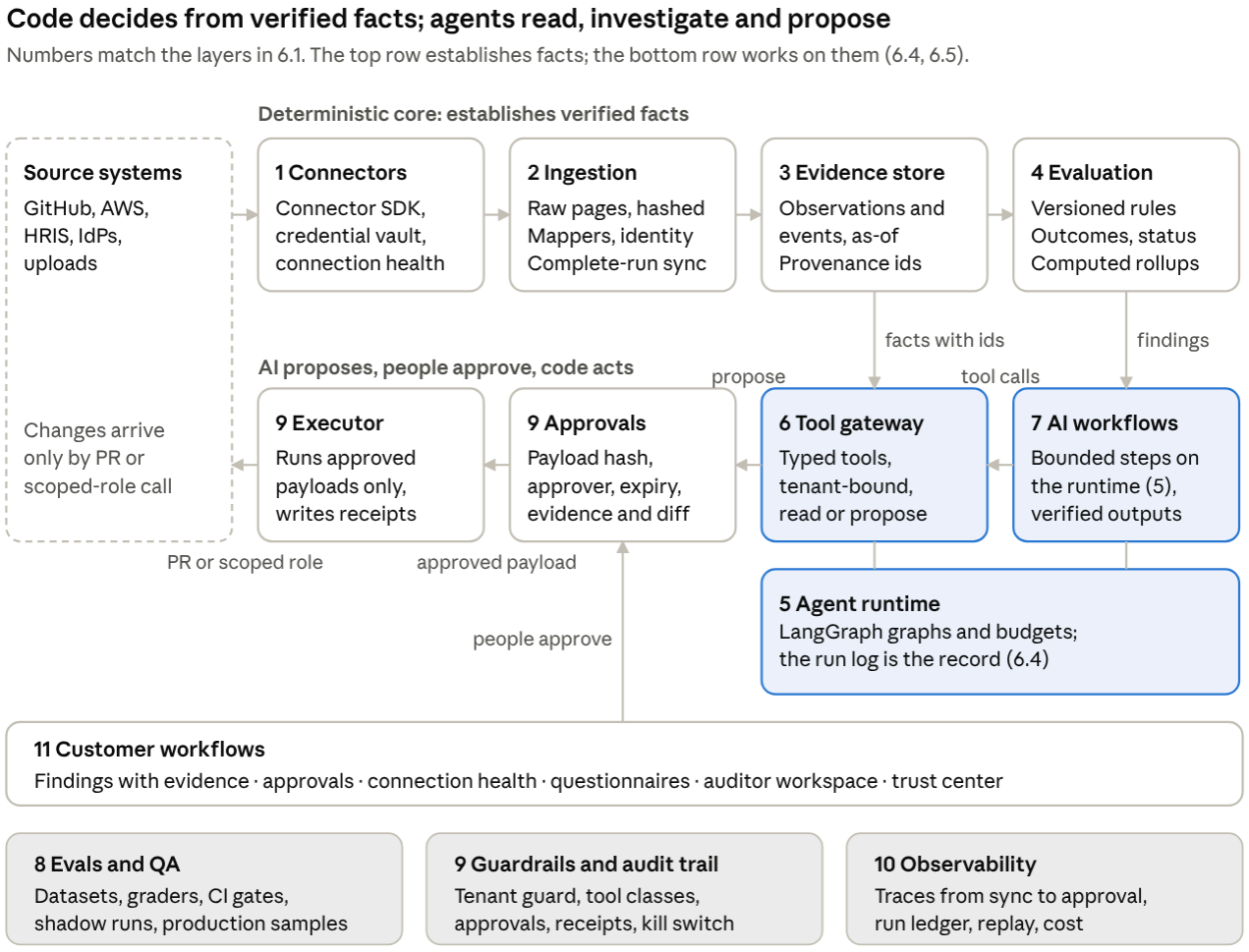
Control	Where it is enforced	What it stops	Basis
Models extract, code decides	Extraction calls have no tools and run per chunk; rules decide; every verdict is recorded inside CCP with its method, and a tier-3 draft awaiting review shows `needs_review`; an AI-derived PASS is never asserted outside CCP (a trust page, an auditor package, a sent questionnaire) without review until the test graduates (4.4)	A poisoned chunk corrupting more than its own fields	Design patterns for securing agents
Approvals	Code writes the diff and risk summary; the approval binds a payload hash and a real approver, who must differ from the requester for high-impact actions; approvals are capped per session	Approval fatigue and model-written justifications	OWASP agentic top 10, Microsoft failure taxonomy
Credential broker	Agents and runners never hold real credentials; a broker checks the approved operation and issues a short-lived, single-action credential; pull requests preferred	A compromised workflow acting with standing access	Anthropic sandboxing

Control	Where it is enforced	What it stops	Basis
Egress proxy	No server-side fetch of model-chosen URLs; one proxy blocks private and metadata ranges, checks redirects and pins DNS	Server-side request forgery, if a future feature fetches URLs that a customer or a model chooses. CCP has none today; until one exists, no tool fetches a URL	MCP security best practices
Render sanitization	AI text is shown with no images, only cited or allowlisted links, and an HTML allowlist	Data leaking through crafted image URLs	Google
Provenance on content	Each chunk carries its source, author permission, uploader, hash and trust tier; GitHub text from users without write access is dropped	Outsiders planting instructions in issues or tags	GitHub Copilot agent
Memory hygiene	Vector namespaces keyed by organization id; AI output never re-ingested as trusted; approved answers labelled, with expiry and rollback	Self-reinforcing poisoning of the answer library	OWASP agentic top 10, ASI06
Audit trail	Every tool call, decision, approval and execution is an append-only record linked to the result	Unexplainable findings	CCP grants only insert on history tables (first report, A.5)

Model-based defenses still run, but as signals: evidence fences with datamarking, an injection classifier on chunks, and a reviewer model that sees only the request and the proposed action. A hit lowers confidence or routes to review; it never grants a PASS. The numbers explain why. Restricting tools before untrusted data arrives cut one model's attack success from 47.7% to 6.8% (AgentDojo). Adaptive attacks broke 12 published defenses at over 90% success (Nasr et al.), and even Anthropic's best browser-agent setup still let 1.4% of attacks through (Anthropic).

6. Target architecture

The target keeps one rule above all others: code decides compliance status from verified facts, models read, extract, draft and investigate, and people approve anything that changes a customer's status or systems. Every result records who decided, by what method, and from which evidence.



Target architecture · 11 layers in two rows plus cross-cutting services

The top row turns source data into facts with provenance; the bottom row reads those facts through one tool gateway and can only propose. Approved changes go back to the customer's systems as a pull request or a narrowly scoped role call, and the next sync re-checks them like any other change. 6.4 to 6.11 settle the agent side layer by layer: the agent stack, retrieval, evals and guardrails, context and long-running work, multi-agent work and the harness, judges and training, and the screens, with a coverage map at the end.

6.1 Layer by layer

"Keep" cells describe code verified at `3386899`; "Change or add" cells are proposals, detailed in the roadmap (Section 8). External facts carry their links.

Layer	Keep	Change or add	Technology and why	Alternatives rejected
1. Integration and connector infrastructure	One `SourceEvent` contract for fixture and live sources, with provider ids, provider time and untouched payloads (`connectors/base.py:26-54`); a pinned GitHub API version (`connectors/github/client.py:45`)	Connections as data, checked daily. Sync runs of declared scopes that end complete, partial, failed or not visible; absence only from a complete scope. A per-endpoint error table and a conformance harness with negative controls. Content-hash snapshots; rate buckets per (tenant, account, region, service). Agent read tools on the same clients	An in-repo connector kit over httpx and boto3 in standard retry mode, which AWS recommends for clients serving many tenants (AWS). A GitHub App, whose tokens "expire after 1 hour" and can be limited to named repositories (GitHub). CloudFormation and StackSets	Unified-API vendors on the core evidence path: they hold credentials and drop provider ids (inferred). Events as evidence: Config emits them "on a best effort basis" (AWS). Third-party MCP servers: their annotations are untrusted (MCP). Treating a 404 as empty
2. Evidence ingestion and normalization	Land before indexing; dedup on provider ids (`connectors/base.py:44-46`); pure mappers; append-only observations; identity scoring with a review queue (`storage/migrations/0004_identity.sql:30-44`)	Identity and indexing on the change path (today full runs only, `compliance_state/run.py:66-67`). A poison record is dead-lettered and marks its sync partial instead of stalling the topic (`ingest/consumers.py:61-72`). Absence only from complete scopes. `mapper_version` , `ts_kind` , `sync_run_id` and `recorded_seq` on observations. Secrets dropped before landing. Upload tenant from the session (`connectors/manual/uploads.py:88`)	The Postgres `event_log` stays the transactional outbox	Kafka or change-data capture: a second log with no throughput need (inferred). Correcting observations in place
3. Evidence storage, history and provenance	Write-once raw capture; insert-only audit grants (`storage/migrations/0005_state.sql:66` , `:68`); bitemporal `compliance_state` ; forced row-level security with a raising tenant guard	An S3 raw zone with Object Lock (governance mode, event holds). Pointers carry the version id, a hash of the bytes received (today a re-serialization, `ingest/raw_zone.py:62`) and the provider's request id. A KMS key per tenant. A per-tenant chain over audit rows with hourly signed checkpoints in a separate account. Tenant-first keys. Insert-only chunks for replay (`replay/rebuild.py:168-173`). Personal identifiers in erasable side tables, so the chain holds only commitments	Event holds, launched 8 Sep 2026 "at no additional charge" (AWS); SSE-KMS with Bucket Keys (AWS); checkpoints modelled on CloudTrail's signed digests (AWS); tenant-first keys because key checks "always bypass row security" (PostgreSQL)	A managed ledger: QLDB ended on 31 Jul 2025 (InfoQ, 26 Jul 2024). Compliance mode for raw data: irreversible short of closing the account (AWS). A transparency-log service before buyers ask. Bitemporal entity tables for now

Layer	Keep	Change or add	Technology and why	Alternatives rejected
4. Deterministic compliance evaluation engine	Tiers fixed at authoring; pure rules; extract-then-rule; one verdict per key (`storage/migrations/0005_state.sql:34`); `needs_review` as a fact; the clock producer; the rollup rule	Verdict and announcement in one transaction (`evaluators/runner.py:93-106`). Rule and parameter versions in the key, then pinned inputs. A required unknown input gives `needs_review`. Coverage and freshness in the rollup. Impact table as data. Tier 1 never parks on model budget. Rule changes with preview and notice. Long-tail checks as versioned YAML	Python rules and a small, total predicate interpreter, shipped in the release bundle behind the gates (proposed)	Model-returned statuses. Rego or CEL now: a second language to audit (inferred). Checks as rows edited at runtime (inferred). AWS conformance packs as the authority: AWS has no SOC 2 equivalent and points customers to partners (AWS)
5. Agent execution and orchestration	The MIT LangGraph library; four graphs over one core with retry and timeout policies (`spec.md` A-52); budgets; checkpoints under row-level security; "graphs read, runners write"	Agent runs as queue jobs with deterministic thread ids (today random, `agent/tier3.py:157-158`) and synchronous checkpoints. Every model and tool call recorded, so resumes never re-sample. The Postgres run log as the record (today tool loops keep the first request and last response only, `agent/core.py:328-330`). Bundles pin runs. Runs end with a proposal. One investigation per evidence state. One writer	LangGraph on the hardened Postgres queue. LangGraph defaults to `async` checkpoints, which can lose the last step (LangGraph). DBOS only after its spike and a trigger	DBOS as the default (Decision 4). Temporal: no I/O in workflow code and a 0.1.0 LangGraph plugin (Temporal). LangGraph's Agent Server: an Enterprise licence to self-host (LangSmith). The Claude Agent SDK for verdicts: a subprocess and local files per session (Anthropic). Parallel writers
6. Shared agent tools and access controls	Tenant bound by a closure; extra arguments rejected (`agent/tools.py:35-36`); per-graph allowlists (`agent/allowlists.py:19-30`); off-list calls get an error (`agent/core.py:339-341`); redaction at the tool seam (`spec.md` A-90)	One typed registry (read, propose or act; schemas; scopes; rate bucket; version). One fail-closed `policy.check` (permit, deny or ask), logged. Explicit truncation (today a silent 20,000-character cut, `llm/client.py:136`); missing data as an error. Fenced, scanned results. Taint removes act-class tools; a Rule-of-Two lint. A read-only database role (today the app role can insert, `storage/db.py:34-36`). A credential module. REST and read-only MCP generated from the registry	Pydantic and a small policy module with a YAML table, after Harvey's engine, which fails closed and lets workspace settings only tighten (Harvey, Oct 2026)	Trusting `readOnlyHint` (MCP). MCP as the internal transport (inferred). OPA or Cedar for a one-page rule table (inferred). Tenant ids chosen by the model

Layer	Keep	Change or add	Technology and why	Alternatives rejected
7. AI-driven compliance workflows	Extract-then-rule; computed confidence; a closed action catalogue; drafts never become evidence on their own; the Copilot's status contract (`spec.md` A-79)	Pointer citations (segment ids, JSON Pointers) that code renders and checks, with typed value checks. A second reader before any AI-derived PASS. A refusal gives `needs_review` (today terminal, `llm/anthropic_client.py:255-256`). Code renders fixes from templates. Agent reads land as evidence. New workflows: evidence review, the AWS Investigator, questionnaires from live verdicts, mapping, audit prep, offline check drafting	Claude through `LLMClient` with structured outputs, generally available on every platform (Anthropic); Haiku 5.5 with thinking off for extraction (Anthropic), Sonnet 5.5 for agents, Opus 5.5 where measured; Claude's PDF input for scans, zero-retention eligible except Covered Models (Anthropic); `voyage-4-large` at $0.12 per million tokens (Voyage)	Native Citations with structured outputs: a 400 (Anthropic). Model-written diffs. Free-text memory as a rule input. Fine-tuning (inferred). Browser agents in customers' consoles. A cross-vendor router (inferred)
8. Evaluation and quality assurance infrastructure	Datasets, graders and thresholds in the repo; golden, judge and adversarial suites; trace capture; calibration with intervals	Run them for real: remote CI on every pull request, keyed nightly runs in an eval organization. Five runs per case, pass^k, paired comparisons, clustered errors; errored runs count as failures; false PASS and false FAIL reported apart; negative controls. 100 to 200 labels per failure mode. Judges as in Decision 9. Bundles promoted only through the gates; shadow diffs; replay sentinels; quality snapshots	`ccp evals` on pytest and GitHub Actions. Borrow Inspect's epochs and `pass_k` reducers, not the framework, since Inspect drops errored samples by default (Inspect)	A hosted eval product as the record: OpenAI's Evals platform ends on 30 Nov 2026 (OpenAI) and Langfuse's trace-level evaluators on 16 Nov 2026 (Langfuse). The single-flip gate (Section 5.2)

Layer	Keep	Change or add	Technology and why	Alternatives rejected
9. Guardrails, human approval and audit trails	One approval stage; the payload hash (`approvals/store.py:22-24`); dry run by default; five live gates (`remediation/gates.py:53-84`); target validation; 21 guardrails, each with a test	Fail closed now (first report, Q2). Then signed-in reviewers with roles; one validated decision per item; payload and precondition hashes; expiry; separation of duties; single-use grants; an append-only journal; rollback as a new proposal; code-written risk summaries. The same service for identity merges, scope changes, break-glass, and exceptions with an expiry, shown as "fail (excepted until D)" and never as PASS. A cap on automated status changes. Passkey-signed approvals for the cloud write path	Cognito, which federates customers' SAML and OIDC providers and redirects "silently" given `idp_identifier` (AWS); later WebAuthn Level 3, whose signed client data carries the server's challenge (W3C, 25 Aug 2026)	Typed names (`storage/migrations/0011_remediation_approvals.sql:50`). Approvals held in engine state. Bulk approval. SAML built in-house (inferred). WorkOS or Auth0: a new subprocessor (inferred). Keycloak: a stateful system to run (inferred)
10. Observability and production monitoring	One step logger with OpenTelemetry GenAI names; the record first, sinks second; replay without a vendor; cost per step	The Postgres run log as the record, with the dated model id, the reported model, API and SDK versions, platform and `usage.inference_geo` on every call. The price table refuses unknown models (today $0, `observability/cost.py:43-50`). Langfuse gets metadata only for production. A fixed failure taxonomy. Correctness objectives. `ccp explain` and `ccp verify`	OpenTelemetry with the conventions version pinned, since they are still in Development (OpenTelemetry); Langfuse Cloud as the view, Pro at $199 a month (Langfuse); CloudWatch	A tracing vendor as the evidence store. Self-hosted Langfuse, which means running ClickHouse (Langfuse). Another APM vendor (inferred)

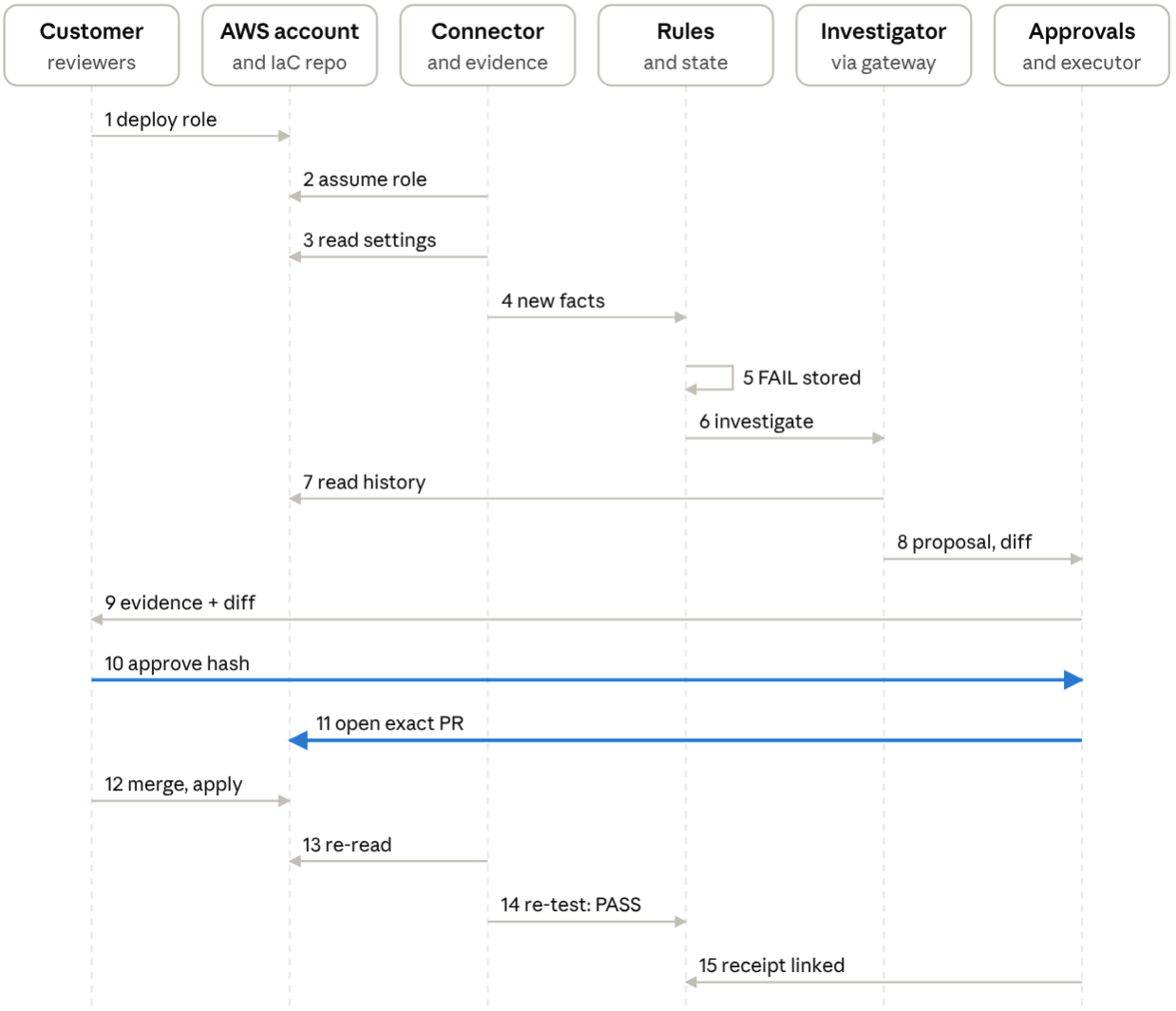
Layer	Keep	Change or add	Technology and why	Alternatives rejected
11. Customer-facing product workflows	The command line for operators; the Copilot's status contract	REST generated from the registry. UI v1: connect GitHub and AWS with the checks visible; the rollup with coverage and freshness; findings with evidence spans and a method card; an approvals inbox with the exact payload and hash; uploads with instant verifier feedback; an as-of export; the Copilot. Five status labels (pass, fail, partial, needs review, not assessed), each with a provenance badge (system-observed, customer-attested, AI-read and code-verified). Rule-change notices. Later: an auditor workspace, attestations, read-only MCP, an autonomy console, published quality results	FastAPI, built on Pydantic and OpenAPI (FastAPI), so CCP's models become the REST contract; Jinja and HTMX for server-rendered pages (proposed); Cognito	A React single-page app for v1: a second toolchain for about six screens (inferred). Django: its ORM and user model beside `ScopedDB` (inferred). Dashboards that compute status apart from the rollup. Auto-publishing every PASS

6.2 The AWS example, end to end

A customer connects AWS, a deterministic check finds a bucket open to the public, the Investigator finds the cause, a person approves an exact fix, and the fix is re-checked like any other change. Every step leaves a record that links back to the evidence.

A public S3 bucket is fixed only through an approved, exact pull request

Numbers match the steps in the table below. On this path CCP holds no AWS write access.



AWS example · 15 steps across six parties, one approval

Step	Who	What happens	What is recorded	Guard
1	Customer	Launches CCP's CloudFormation stack, which creates a read role that trusts CCP under an external ID that CCP generated. An AWS Organization adds a stack in the management account and a StackSet for member accounts	Connection: tenant, account, external ID, stack version	One random external ID per account, generated by the vendor (AWS); StackSets never deploy to the management account (AWS)
2	Connector	Assumes the role with the external ID, which must succeed, and without it, which must fail; lists the role's policies	Onboarding checks, as events	Refuse a role that works without the ID or that holds write actions. Use SecurityAudit plus explicit denies on reads that expose secrets, such as Lambda environment variables and EC2 user data (Datadog)
3	Connector	Reads each bucket's public-access block and policy status, the account's public-access block, and, where the customer records AWS Config, the bucket's configuration items, paging through results with standard retries	Raw pages written once, with a hash, the provider's capture time or CCP's observation time, and a completeness record for the run	AWS API quotas are shared with the customer's own automation (AWS), so a rate limit applies per tenant, account and region. A 404 `NoSuchPublicAccessBlockConfiguration` on the account means "not configured", a real fact (AWS); AccessDenied means "not visible", and the test returns `needs_review`
4	Connector	Maps pages to `Asset` observations, redacting secrets before anything is stored	Append-only observations with evidence ids	A missing resource becomes a deletion only from a complete run

Step	Who	What happens	What is recorded	Guard
5	Rules	The changed bucket re-runs the test "S3 buckets block public access"; the rule takes each flag from the bucket or the account, whichever blocks, reads the policy status, and returns FAIL	A verdict with its rule version, evidence ids and as-of times, and a bitemporal state row	Code decides; no model is involved
6	Rules	A new FAIL, or changed evidence behind a failing verdict, starts at most one bounded Investigator run per evidence state; when the rule's reasons already explain the failure, code writes the investigation instead	A run record with its harness version and budgets	Turn, token, cost and time budgets
7	Investigator	Calls read tools through the gateway: Config history in 7-day windows, a CloudTrail lookup of who changed the setting, and a search of the customer's infrastructure code for the bucket	Every provider response lands in the raw zone through the connector's client (rate limit, redaction, request id) before the model sees it, and every call is in the run log; code, not the agent, can then back-date the FAIL to when CloudTrail or Config shows the change	The policy check allows only the read class; the tenant comes from the session; CloudTrail lookups stay under its limit of 2 per second (AWS); tags and names are treated as untrusted input

Step	Who	What happens	What is recorded	Guard
8	Investigator	Proposes a probable cause with cited evidence and picks a catalogue action with its parameters (repository, file, resource address); code renders the Terraform change from a versioned template, validates it offline and checks that it touches only this bucket	An append-only proposal with a payload hash	Code checks every citation; the agent cannot write anywhere and never writes the diff that is approved
9	Approvals	Shows a reviewer the failing evidence, the cause, the exact change and its hash	None	The reviewer is authenticated on the server and holds rights for this tenant and action; the request carries the payload hash, a precondition hash (the FAIL is still the head, the bucket's state, the base commit) and an expiry, and the approver is not the requester and owns the repository on the customer's side
10	Customer	Approves	A decision: reviewer, decision, payload hash, time	An unknown decision is rejected; a replayed or changed payload is refused; an expired request is refused, and a decision mints one single-use execution grant
11	Executor	Opens a pull request in the customer's repository with exactly the approved change, through CCP's GitHub App	A receipt: pull request link, commit, payload hash	The idempotency key is the proposal id plus the hash. The executor consumes the grant, re-checks the precondition and refuses on drift. It uses an installation token limited to this repository, with contents and pull-request write, which expires after 1 hour (GitHub). CCP holds no AWS write credentials on this path, as with Drata's and Vanta's pull-request fixes (Drata, Vanta)
12	Customer	Reviews and merges the pull request; their pipeline applies it	Nothing in CCP	The customer's own change process

Step	Who	What happens	What is recorded	Guard
13	Connector	Re-reads the bucket after the merge or at the next scheduled sync	A new raw page and observation	An event only triggers a re-read and is never evidence itself; AWS Config events are best-effort (AWS)
14	Rules	The rule returns PASS	A new verdict and state row supersede the FAIL; history keeps both	As-of queries show what was true, and what CCP believed, at any time
15	Approvals	Links the receipt to the PASS verdict	One chain: evidence, FAIL, run, proposal, approval, pull request, PASS	A per-tenant hash chain over the raw objects and every database row in this chain (verdicts, state rows, the run, the proposal, the approval, the receipt), with an hourly signed checkpoint written to a compliance-mode bucket in a separate AWS account, modelled on CloudTrail's digest files (AWS), lets an auditor check the evidence without trusting CCP's database

When the bucket is not managed in code. An opt-in remediator role, deployed by the customer as a separate stack, may run one allowlisted AWS-owned runbook, such as `AWSConfigRemediation-ConfigureS3BucketPublicAccessBlock`, which verifies its own result (AWS). CCP assumes it only after approval, for 15 minutes, with a session policy limited to the target resource and the approval id as `SourceIdentity`, so the customer's CloudTrail shows which approval each action served (AWS). The executor re-reads live state first and refuses if it no longer matches the state recorded in the approval. CCP never uses AWS Config auto-remediation, which can act on stale compliance data (AWS). No competitor documents a vendor-held write role (Drata, Secureframe), so this path ships only when a design partner asks for it, and its approvals are signed with a passkey (Decision 5).

What not to build on. AWS Audit Manager closed to new accounts on 30 Apr 2026 (AWS) and CloudTrail Lake on 31 May 2026 (AWS). Security Hub deletes findings that have not been updated for 90 days, and the IAM credential report keeps only its latest copy (Security Hub, IAM), so CCP snapshots both into its own raw zone when it reads them.

6.3 Cross-cutting concerns

Multi-tenancy.

(proposed) One pooled cell per region by default (a database, a raw bucket, a deployment), isolated by forced row-level security, the raising tenant guard and a non-owner application role. Before real data lands: tenant-first composite keys and tenant-scoped cache keys, because unique and foreign-key checks bypass row security (PostgreSQL) and today a state row can reference any tenant's verdict (`storage/migrations/0005_state.sql:51`) while both caches are keyed without the tenant (`storage/migrations/0008_chunks.sql:36`, `:44`); a catalogue lint with a deliberately permissive policy as its negative control; and the tenant set inside each transaction everywhere, including the checkpoint saver, which today passes it as a connection startup option (`storage/checkpoints.py:64-68`) that PgBouncer's transaction mode rejects or drops (PgBouncer). Use PgBouncer, not RDS Proxy, which pins a session on every `set_config` (AWS). Search each tenant's chunks exactly by default: pgvector filters "after the index is scanned", so small tenants get short results from a shared index (pgvector), and exact search returns the same order every time, which replay needs (inferred). Require pgvector 0.8.2 or later for CVE-2026-3172, a cross-relation leak (EDB, 10 Mar 2026). The tenant is bound in code on every AI path: closures for graphs, the session for the policy check, the signed-in token for MCP. Tenant evidence sits after the prompt-cache breakpoint (`spec.md` A-62); caches are isolated per workspace on the Claude API but only per organization on Bedrock (Anthropic). No standing staff access; break-glass goes through the approval service. A cross-tenant canary runs in production. Silo cells (own database, bucket and key) on contract, because a tenant-specific storage key is possible only in a silo (AWS, Apr 2024). Vanta's 2025 cross-customer exposure came from "a product code change" (TechCrunch, 2 Jun 2025); CCP's raising guard turns that class of bug into an error.

Scaling from 2 to thousands of tenants.

Stage	Tenants	Shape (proposed)	What changes
0	2 to 50	One RDS PostgreSQL 17 instance (Multi-AZ); two `api` and two `worker` tasks; no pooler	The near-term fixes; about 60 database connections (inferred)
1	50 to 500	A larger instance; four to eight workers scaled on queue age; `event_log` partitioned by time, with processed partitions dropped (it is operational, not evidence)	Fair claiming through a work-hint table and LISTEN/NOTIFY wake-ups, already in place from the medium term (today one loop visits every tenant every 2 seconds, `ingest/worker.py:67-103`)

Stage	Tenants	Shape (proposed)	What changes
2	500 to 5,000	PgBouncer in transaction mode; LIST partitions with their own HNSW index for the largest tenants, within the "few thousand partitions" Postgres handles well (PostgreSQL); a read replica for exports	Very large tenants move to their own cell
3	5,000 and up	Several pooled cells per region; a distributed Postgres only if one cell's database is outgrown, which tenant-first keys keep possible	Tooling to place and move tenants between cells

(inferred) At 1,000 tenants with about 2,000 (test, subject) pairs each, a 30-day revalidation is about 67,000 evaluations a day plus changes: a few a second, mostly cheap tier-1 SQL. The first limits are model rate limits and spend caps (the Scale tier caps spend at $200,000 a month, and at the cap requests get a 429 with no retry-after, Anthropic) and customers' own AWS quotas, which CCP shares with their automation (CloudTrail `LookupEvents` allows 2 requests a second per account and region, AWS). Content-hash snapshots keep the steady state cheap: an unchanged re-sync renews freshness and writes no new verdicts.

Fault tolerance.

Failure	Behaviour (proposed)
Worker killed mid-job	The lease expires and another worker claims the job; idempotency keys make the repeat a no-op; an agent run resumes from its last synchronous checkpoint without sampling a model again
Crash between a verdict and its announcement	Impossible after the outbox fix (today it strands the verdict, `evaluators/runner.py:93-106`); a reconciler announces any verdict with no state row and reports the count

Failure	Behaviour (proposed)
Model outage, 429 or 529	Backoff with jitter through `available_at` (today a retry is claimable at once, `ingest/event_log.py:110-115`); then AI work parks and tier 1 continues
Anthropic spend cap	A 429 with no retry-after (Anthropic): park all model work and page; never retry
Refusal	`needs_review` with its category, never a verdict and never a dead letter
Prolonged Anthropic outage	A rehearsed manual switch to Bedrock for online calls, pre-agreed in each tenant's DPA because Amazon becomes the processor and Bedrock has no Message Batches (Anthropic); runs restart on the secondary, because Haiku 5.5 thinking blocks are bound to the account that produced them (Anthropic)
Connector throttling or lost permission	The scope ends partial; dependent tests return `needs_review`; the connection shows degraded; never a false absence
Bad mapper release	Dead-letter, mark the sync partial, alert; after the fix, replay from the immutable raw zone
Executor crash or drift	A single-use grant, an append-only journal and deterministic branch names make retries idempotent; drift from the approved precondition refuses and asks for a new approval
Deploy	Runs pinned to their bundle; stored proposals stay bound to their artifacts and hashes
S3 or KMS unavailable	Landing and decryption fail closed; evaluation waits rather than run on evidence it cannot store or verify
Altered database history or restore	The next `ccp audit verify` against checkpoints in the separate account fails and pages; attestations freeze; a quarterly restore drill ends with a verify

Schema evolution and backward compatibility.

Artifact	Versioned by	Compatibility rule (proposed)
Provider payloads	`SourceEvent.schema_version` (`connectors/base.py:35`); raw objects never change	A mapper change ships as a new `mapper_version`; replay from raw appends observations and never rewrites old ones, so old verdicts still point at what they read
Database schema	Additive migrations (the runner's existing rule, `spec.md` section 2)	Expand, migrate, contract for renames. Audit rows are hashed over a versioned canonical projection, so a new column never breaks an old hash
Rules and declarative checks	Rule version (later the inputs hash) in the evaluation key	A correction re-evaluates history and supersedes with reason `correction`; an amendment applies from its effective date. Preview and notice first; FAIL-to-PASS flips need a named sign-off. Add `rule_change` to the reason check (`storage/migrations/0005_state.sql:57`)
Extraction schemas and prompts	`schema_version` on verdicts; prompt files never edited and hash-checked (`spec.md` A-21)	A change makes a new bundle behind the gates; old artifacts stay re-verifiable because verifiers are versioned too
Models	Dated ids inside bundles; the API version pinned in code; the model each response reports	About one forced migration per tier per year, since each model's commitment runs about a year and retirement needs at least 60 days' notice (Anthropic). Each is a paired comparison gated on pass^k, refusal rate and under-escalation. Verdicts from a retired model stay reconstructible from stored outputs. An audit-period bundle freeze cannot outlast a retirement notice
Connectors and customer roles	Pinned provider API versions (`connectors/github/client.py:45`); recorded responses per version; a template version on each AWS connection	A test that needs a new AWS action returns `needs_review` on an older template instead of failing; daily checks catch AWS's edits to managed policies (SecurityAudit is at v94, edited 30 Sep 2026, AWS)
Tools	`ToolSpec.version`	Output schemas only gain optional fields; a breaking change gets a new name
Agent checkpoints	A graph version on each thread	Threads of a retired version close and re-run; nothing waits inside a checkpoint
Embeddings	The model id on every chunk	Re-embed into new rows and switch the index version; keep old rows for replay
Framework mappings	A catalogue version with effective dates; control ids are already copied onto each state row at write time (`storage/migrations/0005_state.sql:49`)	Add the mapping version to state rows, so the mapping in force on any past date is known

Artifact	Versioned by	Compatibility rule (proposed)
Exports and APIs	A version on each API and export manifest	Readers accept the current and previous versions

Cost.

Model list prices per million tokens, input and output, with the cache-read price in brackets (Anthropic): Haiku 4.5 $1 and $5 ($0.10); Haiku 5.5 $0.10 and $0.50 ($0.01) for prompts up to 100,000 tokens; Sonnet 5.5 $2 and $10 ($0.10); Opus 5.5 $4 and $20 ($0.20). Models from Claude 4.7 on produce about 30% more tokens for the same text. Batches cost half, but keep results 29 days and so fall outside zero retention (Anthropic).

Unit	Assumption	Estimate
One tier-2 extraction today	The spec's logged call: 3,120 tokens in, 410 out (`spec.md:1783-1784`)	$0.0052 on Haiku 4.5
The same on Haiku 5.5	30% more tokens, thinking off	About $0.0007; about $0.0005 with a cached prefix (inferred)
Second reader on an AI-derived PASS	One more extraction, on candidate passes only	About $0.0007 per pass (inferred)
One Investigator run	Sonnet 5.5, about 8 calls, about 100,000 cumulative input and 4,000 output tokens	About $0.24 uncached, about $0.10 with most input cached; capped at $0.50 a run (`config.py:91`) (inferred)
A 200-question questionnaire	Per question: four Haiku 5.5 extractions, one Sonnet 5.5 draft, one entailment check	About $3 to $5 (inferred)
Embeddings	`voyage-4-large`	$0.12 per million tokens (Voyage)

(inferred) A tenant with 600 tier-2 calls (about $0.40), 30 investigations once they are deduplicated by evidence state ($3 to $7; without that, a 30-day revalidation of every persistent failure buys a new run), 100 Copilot answers ($5 to $15) and 50 document reviews (about $0.50) costs roughly $10 to $25 a month in model spend: about $10,000 to $25,000 a month at 1,000 tenants. Investigations and the Copilot dominate, so investigating once per evidence state, writing rule-explained investigations in code, and caching stable prefixes (possible from 512 tokens on the 5.5 models, against 4,096 on Haiku 4.5, Anthropic) are the main levers. Infrastructure: KMS keys cost $1 a month each plus up to $2 for rotations (AWS), so about $1,000 to $3,000 a month at 1,000 tenants

(inferred); Object Lock adds no fee beyond storage; Langfuse Pro is $199 a month (Langfuse). RDS and Fargate prices are not in the notes, so this report gives no figure for them; they grow with instance size and task count, not tenant count (inferred). Avoided: DBOS Conductor at $99 a month for one million checkpoints, then $50 per million (DBOS); Temporal Cloud from $50 per million actions (Temporal); a sandbox (E2B Pro is $150 a month plus usage, E2B); a ClickHouse cluster. Customers pay for AWS Config if they enable it: $0.003 per continuously recorded item and $0.012 per periodic item (AWS), which is why event-driven sync is an opt-in add-on.

Operational complexity and on-call.

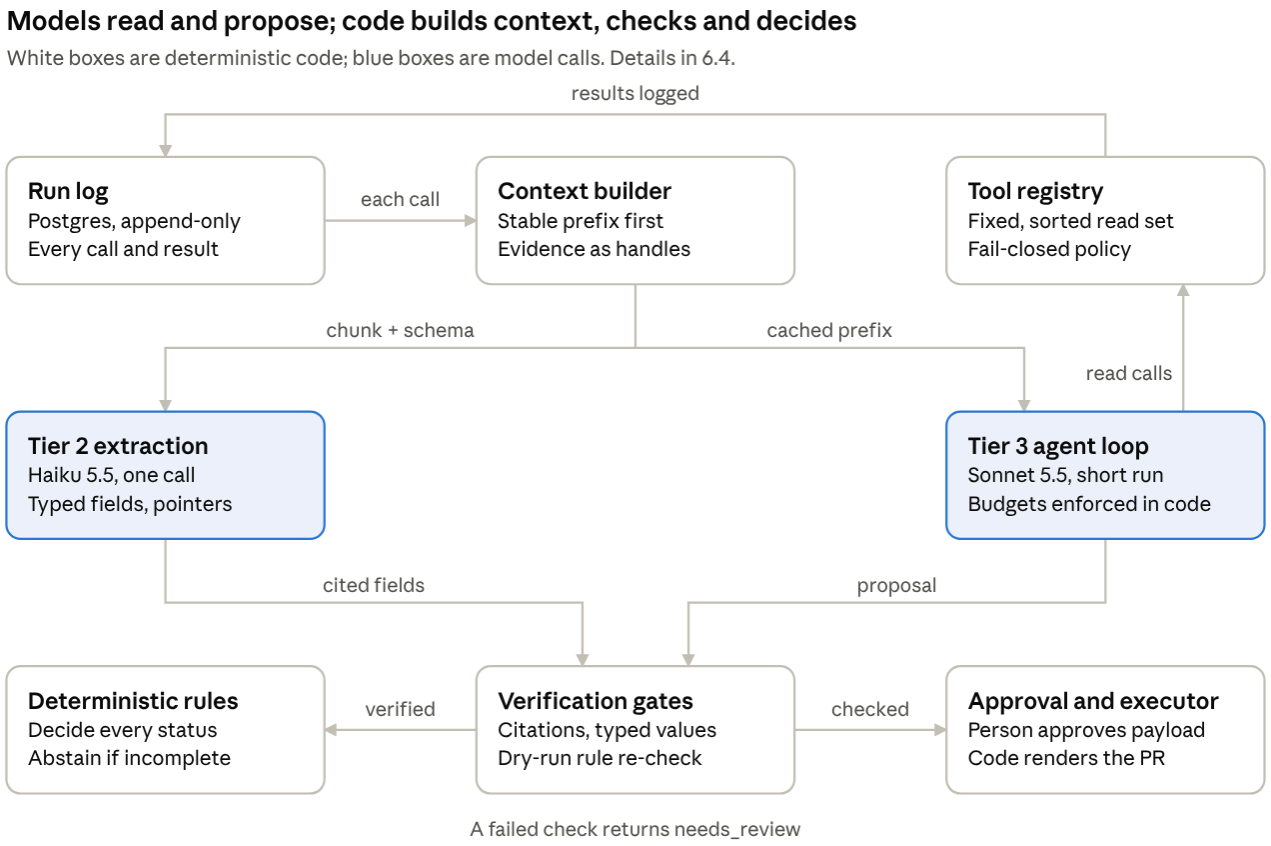
- **Inventory (proposed).** One RDS PostgreSQL 17 (Multi-AZ); S3 buckets in two AWS accounts; KMS keys; one container image run as two ECS services (`api`, `worker`); a load balancer; a Cognito user pool; CloudWatch. Five outside services: Anthropic, Bedrock (standby), Voyage, Langfuse Cloud and GitHub (the App and Actions).
- **Deliberately not added.** Kafka, Redis, Temporal, DBOS (until triggered), Kubernetes, a vector database, self-hosted observability, an MCP gateway, a code sandbox.
- **Pages.** The database is down or failing over; the oldest pending evaluation is older than 30 minutes; dead letters on evaluations or apply jobs pass a threshold; chain verification fails; a replay or executed-versus-approved mismatch; AI work parked for over 30 minutes or the spend cap hit; connection checks failing for many tenants at once; the cross-tenant canary fails. Everything else becomes a ticket.
- **Rotation.** One weekly rotation across the three to five engineers, business hours first; after-hours paging only for database, integrity and apply alerts until a contract requires 24-hour cover. A separate weekly quality review reads 50 to 100 traces, the coverage backlog and monitor alarms.
- **Runbooks.** Replay dead letters; unpark AI work; switch to Bedrock and back; rotate the GitHub App and signing keys; re-verify or revoke a connection; re-map after a mapper bug; roll out a rule correction; migrate a model; offboard a tenant by destroying its key; seal an auditor export; break-glass access.
- **Game days, monthly.** Kill a worker mid-run; fail over the database; simulate an Anthropic outage; corrupt a chain entry in staging and watch the verifier page.
- **Kill switches.** Per workflow to suggestion-only or off; per tool; per tenant AI off; one switch that parks all model work.

Hosting and product stack. The options were AWS (ECS Fargate, RDS, S3, KMS, Cognito), GCP (Cloud Run, Cloud SQL), Kubernetes or a platform-as-a-service, and for the UI FastAPI with server-rendered pages, a React single-page app or Django. The choice is AWS on ECS Fargate with RDS PostgreSQL 17, and FastAPI with server-rendered pages for version 1. The AWS connector needs an AWS principal that customers' roles trust, which on AWS needs no stored keys (inferred); S3 event holds, KMS and Bedrock sit in one

account (6.1); Fargate runs containers "without having to manage servers or clusters of Amazon EC2 instances" (AWS); and FastAPI is built on Pydantic and OpenAPI (FastAPI). The costs are lock-in to AWS, a less rich UI than a single-page app, and an older pgvector: RDS and Aurora ship 0.8.2 (AWS) while Cloud SQL ships 0.8.5 (Google), though exact search avoids the HNSW code paths fixed after 0.8.2, and the HNSW partitions planned for the largest tenants wait for 0.8.3 or later (6.5) (inferred). Early customers mostly on GCP or needing EU processing first (Vertex EU), or a UI that needs heavy client-side interaction, would change it.

6.4 The agent stack: decisions by layer

In our reading, frontier teams now build agents the same way: one writer runs a short loop inside code, with a few stable tools, a prompt rebuilt from a durable record, and harness changes shipped like releases. CCP's design already has this shape (3.1 to 3.5), so the table settles the remaining choices for layers 5 to 7 of 6.1, plus models and context, and names what waits for a trigger. None of the decisions adds a service.



Agent stack · six code parts around two model steps

Only the two blue boxes call a model. Everything that builds context, checks, decides or acts is code, and each model step starts from a context rebuilt from the run log.

Component	Decision for CCP	Frontier precedent	Not now, and the trigger
Orchestration	Each graph is a fixed workflow with at most one agent loop, placed where the path is unknown: the Investigator loops only in `gather`, and four of its six nodes are code (`agent/graphs/investigator.py:291-312`). Code is the one writer; a sub-agent, if added, is read-only, gets its own context and an output schema, and returns ids. Breadth work, such as vendor reviews or audit-prep gap lists, fans out from code, one fresh run per item, with a deterministic check before results merge	A Google Research, Google DeepMind and MIT study of 180 configurations found that a central coordinator gained 80.8% on a finance task that splits cleanly, while every multi-agent design lost 39% to 70% on sequential planning. Independent agents amplified errors 17.2 times, against 4.4 with a coordinator (Kim et al., Dec 2025). Microsoft's Agent Framework 1.0 ships graph workflows "for composing agents and functions into deterministic, repeatable processes", with checkpoints and human approvals (Microsoft, Apr 2026)	Agents that negotiate with each other or write in parallel; Claude Code's agent teams are still "experimental and disabled by default" (Claude Code docs, undated). Trigger: one agent scores below about 45% on a read task that splits into independent parts; above that score, the study found coordination stops paying
Harness releases	Every run and verdict names its bundle (3.5; 6.1, rows 5 and 8), which pins prompts, graph, tool descriptions, budgets, effort and thinking settings, and the context-building code. Today effort is fixed at medium in code (`llm/anthropic_client.py:69-71`). A change ships after per-model evals, a line-by-line ablation of any multi-line prompt edit, and a canary tenant before a staged rollout; a model migration is planned to finish within six weeks of a deprecation notice	Anthropic traced three quality drops that users noticed to harness changes, not the model: default effort cut from high to medium ("the wrong tradeoff"), old thinking cleared on every turn, and a verbosity line that, with other changes, cost 3% on an ablation eval. It now uses per-model evals for every system-prompt change, line-by-line ablations, soak periods and gradual rollouts (Anthropic, Apr 2026). Its 2026 deprecations gave about two months' notice, as with Sonnet 4.5, announced 30 Sep and retiring 30 Nov (Anthropic release notes, Sep 2026)	Never: a vendor-hosted prompt store. OpenAI shuts its reusable-prompts API on 30 Nov 2026 and advises "Move prompt content into application code" (OpenAI, undated). Prompts stay in git, hash-checked (`spec.md` A-21)

Component	Decision for CCP	Frontier precedent	Not now, and the trigger
Context assembly	The prompt view (3.4) keeps one byte-stable order: tool definitions (already sorted, `agent/core.py:292`), then system prompt and schema, then the run's data; nothing that varies per run goes before the cache point. Tool loops turn on automatic caching, so each turn reads the earlier turns from cache; today only system blocks carry a cache point (`llm/anthropic_client.py:48-55`, `:303-312`). A result too large for the context arrives as a handle with a preview, and a read tool returns the rest in pages	Claude Code orders prompts "static content first, dynamic content last" and puts updates in the next message, not the system prompt (Claude blog, Apr 2026). Anthropic's automatic caching moves the cache point forward as a conversation grows (Anthropic release notes, Feb 2026). Cursor writes long tool outputs to files that the agent reads in parts (Cursor, Jan 2026)	Skill-style procedure files per framework, listed by name and description and loaded in full only when relevant, as Agent Skills are (Anthropic, Oct 2025). Trigger: a third framework, or guidance too large for the cached prefix; each file is versioned and hashed and holds no pass or fail logic
Long runs and compaction	Runs stay short: by default a run stops at 8 tool calls, 60,000 tokens, $0.50 or 120 seconds (`llm/budget.py:27`, `config.py:91-93`), and each Copilot turn starts a fresh context (`agent/graphs/copilot.py:256`). Longer work becomes a new run seeded from typed state. If a run must shed context, the view swaps old tool results for handles, never a result that a draft cites, and no model-written summary sits between evidence and a proposal	Anthropic's context editing "automatically clears stale tool calls and results"; Anthropic reports a 29% gain from it alone on an internal eval run before the 5.x models (Anthropic, Sep 2025). Amp replaced compaction with hand-offs to fresh threads because compaction is "lossy" (Amp, Oct 2025). Factory reports that every compression method it tested was weakest at tracking which files changed (Factory, Dec 2025)	Server-side compaction, still in beta (Anthropic docs, undated). Trigger: the Copilot starts carrying earlier turns; then on-demand compaction with CCP's own summary prompt, stored in the run log, for the Copilot only
Memory	Facts stay in the versioned store, and past investigations are read through tools such as `get_investigation` (`agent/tools.py:170-174`), never copied into memory. When memory is added, its only new layer is short per-tenant notes on how a tenant works (owners, where evidence lives), each with source ids, and a person confirms each note before any agent sees it. No rule, extraction or verifier reads memory	Anthropic's memory tool "operates entirely client-side through tool calls", so the builder owns the store (Anthropic, Sep 2025). A June 2026 study found that agents which write and retrieve memory more aggressively are "more exploitable", and that "existing prompt injection defenses fail to cover memory poisoning attacks" (Dash et al., Jun 2026)	Copilot memory, absent today (`spec.md:113-116`), and any memory hosted by a model provider or shared across tenants. Trigger: users keep restating context, and a poisoning suite (notes planted through uploads, connector text and chat) passes before the first write path ships

Component	Decision for CCP	Frontier precedent	Not now, and the trigger
Tool set	Each graph keeps a small, fixed tool list per bundle, in sorted order and without overlaps: today 4 to 13 of CCP's 13 tools (`agent/allowlists.py:19-30`). A blocked or failed call returns a typed error that says what to try next; tools are never added or removed mid-run. Three consecutive errors end the run in `needs_review`, as Claude Code's auto mode escalates after three consecutive denials (3.6)	GitHub cut "the default 40 built-in tools down to 13 core ones" and reports 2 to 5 points more on SWE-bench Verified and SWE-Lancer, and 400 ms lower latency (GitHub, Nov 2025). Cursor reports 46.9% fewer agent tokens, in runs that used MCP tools, from loading only tool names up front (Cursor, Jan 2026). Claude Code: "adding or removing a tool invalidates the cache for the entire conversation" (Claude blog, Apr 2026)	Tool search with deferred loading, which no longer needs a beta header (Anthropic release notes, Feb 2026); see 3.3. Trigger: one graph needs more than about 20 tools. The Copilot gets every read tool (`agent/allowlists.py:29`), so it gets there first as connectors add read tools
Code execution	No model-written code runs in CCP, and none will run on a path that feeds a status. Typed tools cover today's workflows, and code renders remediation diffs from templates (6.2)	Replit's testing sub-agent writes Playwright code in a REPL instead of calling fixed browser tools, and Replit reports a median cost of $0.20 per testing session (Replit, Dec 2025). Anthropic's containment rule: "The weakest layer is the one you built yourself"; standard hypervisors held where its own proxies failed (Anthropic, May 2026)	Anthropic's code execution, which falls outside zero retention (3.7), and programmatic tool calling, which runs inside code execution and so falls outside it too (inferred). Trigger: an offline analysis that typed tools cannot express, such as diffing a large access-review export; then a standard microVM sandbox with no network or credentials, run by CCP, with the generated code kept as a record of what ran
AI workflows (tiers 2 and 3)	Tier 2 stays one tool-less call per chunk or item, and a rule decides from its typed fields; a large batch is a code loop of such calls, never one long context. Tier 3 ends a run by storing a proposal whose actions come from the closed catalogue, with reversibility read from the table, never from the model (`knowledge/actions.py:11`)	Manus found that uniform context makes agents brittle: "don't few-shot yourself into a rut" (Manus, Jul 2025). Stripe's compliance agents split each review into scoped sub-tasks; the reviewer answers every one, and agent output is "supplementary information to the human reviewer" (Stripe on the AWS ML blog, Jun 2026)	Agents that act. Trigger: a reversible, low-impact action earns a measured level under 4.4 (LT8)

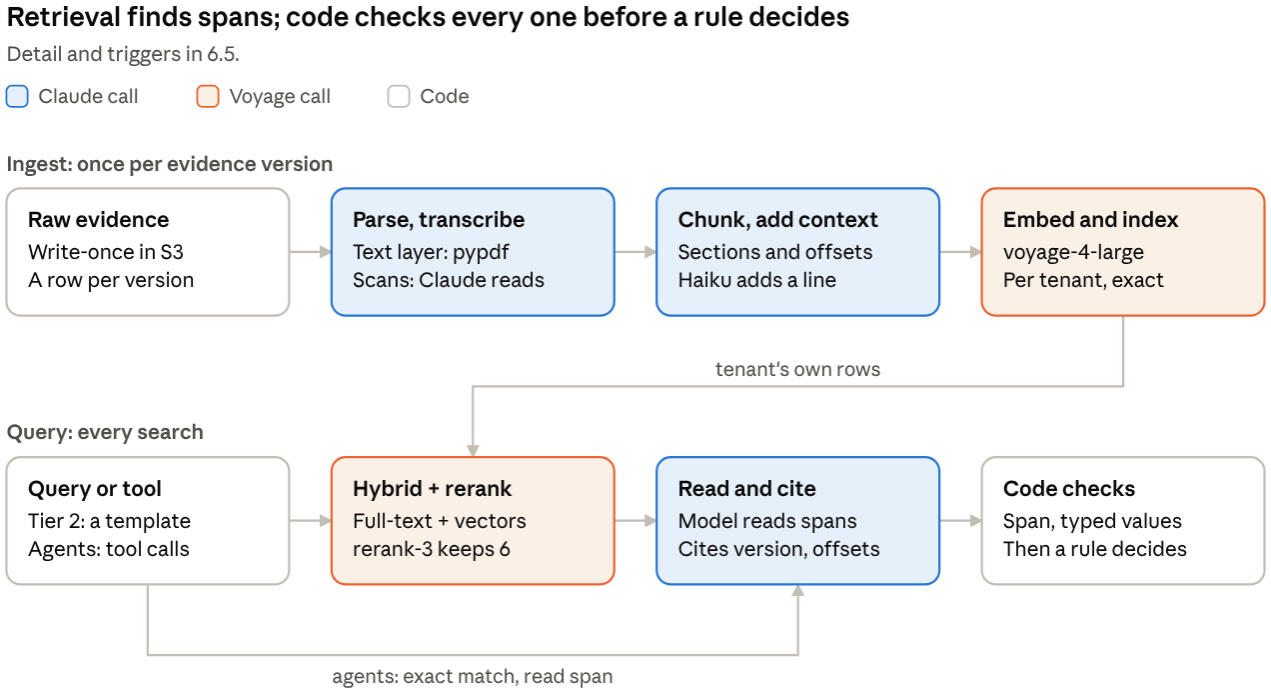
Component	Decision for CCP	Frontier precedent	Not now, and the trigger
Verification before approval	A run reaches "propose" only through code gates: schema, citations with typed values (5.4), catalogue bounds and, where an action changes an observed setting, a dry run that applies its declared effect (a new catalogue field) to a copy of the observations, re-runs the tier-1 rule and shows the reviewer both results. A failure gives `needs_review` with its reason; high-value outputs then get the clean-context reviewer of 3.2, as a signal only. Today the Investigator checks only that quotes exist (`agent/graphs/investigator.py:187-213`), and the rule re-runs only after approval, on fixtures (`remediation/runner.py:205-231`)	Anthropic ranks rule-based checks first and an LLM judge last, as "generally not a very robust method" (Anthropic, Sep 2025). Replit verifies in a separate testing sub-agent, to catch features that look finished but do not work (Replit, Dec 2025). Anthropic found its own adversarial reviewer can be overly defensive and add needless complexity (Anthropic, Oct 2026)	A model reviewer on every output, or judge-driven retry loops. Trigger: the reviewer's flags agree with expert labels at 85% or better on a labelled sample (7.1)
Model routing and budgets	Route by step and never switch models inside a run: Haiku 5.5 for extraction, Sonnet 5.5 for agent loops, and Opus 5.5 only as a fresh run on cases where evals show it helps (Decision 9). Hard caps stay in code (`llm/budget.py:73-100`); an API task budget is added only at a measured p99 and set once per run	Claude Code notes that switching from Opus to Haiku 100,000 tokens into a session costs more than letting Opus answer, because Haiku must rebuild the cache, so it routes cheaper work through sub-agents (Claude blog, Apr 2026). Task budgets are "a soft hint, not a hard cap", have a 20,000-token minimum, can cause "refusal-like behavior" when too small, and break the cache when changed (Anthropic docs, undated)	The advisor tool. On the 5.5 models its advice comes back as "opaque `encrypted_content`" (Anthropic docs, undated), so the run log could not record it, and its zero-retention status is unconfirmed; Anthropic's results, such as Haiku with an Opus advisor at 41.2% against 19.7% alone on BrowseComp, are self-reported (Anthropic, Apr 2026). Trigger: readable advice and confirmed zero retention, then a gain on CCP's evals
Retrieval	Agents search through typed, as-of-aware tools that return ids, versions and hashes; hybrid search is one of those tools. Detail in 6.5	Anthropic advises starting with agentic search and adding semantic search only for speed or varied phrasing (Anthropic, Sep 2025); Cursor reports 12.5% higher answer accuracy on average from adding semantic search to grep (Cursor, Nov 2025)	See 6.5
Evals, guardrails and operations	Every bundle clears the gates in 5.3 and the guardrails in 5.5. The run log adds four agent metrics per graph: cache-hit rate, tokens per completed task, consecutive tool errors and refusals. Detail in 6.6	Factory: "The right optimization target is not tokens per request. It is tokens per task." (Factory, Dec 2025). After infrastructure bugs that its evals missed, Anthropic added "Continuous quality evaluations run on production systems" (Anthropic, Sep 2025)	See 6.6

Not adopted yet

- **A hosted agent runtime.** Every Claude Managed Agents endpoint still needs a beta header (Anthropic release notes, Apr 2026), Anthropic hosts the harness, and the service sits outside zero retention (3.7), which conflicts with replay from CCP's own run log (inferred). Revisit for an isolated, sandboxed analysis task if it becomes eligible.
- **Fine-tuned small models.** Fin reports 12.5% lower cost per transaction, with satisfaction unchanged, after replacing a GPT-4.1 step with a fine-tuned 14B model, and it trained one classifier on a million examples (Fin, Sep 2025). Haiku 5.5 is already 20 times cheaper than Sonnet 5.5 with no training pipeline (6.3, Cost). Revisit for one narrow step with that volume and stable labels.

6.5 Retrieval, layer by layer

Today one SQL statement fuses Postgres full-text and pgvector HNSW results by reciprocal rank fusion, and Voyage `rerank-2.5` reranks the top 20 and passes 6 on (`retrieval/hybrid.py:112-151`, `retrieval/config.py:34-37`, `retrieval/rerank.py:50`). Chunks are 700-character windows, PDFs give only their text layer, and agents get one search tool capped at 8 results plus chunk reads (`retrieval/chunker.py:19-21`, `connectors/manual/pdf_text.py:91-100`, `agent/tools.py:32`). The target keeps this core, reads scans and screenshots, serves each tenant from its own rows as of any date, and gives agents exact tools beside semantic search; retrieval only finds spans, code checks them (5.4), and a rule decides.



Retrieval pipeline · ingest once per version, query per search, code checks every citation

Ingest runs once per evidence version; queries run on every search. Whatever finds a span, code checks the citation before any rule uses it.

Layer	Decision for CCP	Frontier precedent	Not now, and the trigger
Parse and transcribe (PDFs, scans, screenshots)	Keep pypdf for pages with a text layer. Transcribe scans, image-only pages and screenshots page by page with Claude in the zero-retention organization, choosing the model on CCP's fixtures; record each transcript like an extraction and show the page image beside every cited span. Today images are skipped and a PDF is rejected only if every page is empty, so a mixed PDF's scanned pages drop out with no event (`connectors/manual/uploads.py:125-132`, `connectors/manual/pdf_text.py:86-88`, `normalize/manual_mapper.py:28-31`)	Claude reads each PDF page as text plus an image (Anthropic), but a scan without a text layer cannot be cited (Anthropic). Vendor-built benchmarks put Claude Opus 4.8 at 93.0% field accuracy as a parser (Extend reports) and Haiku 4.5 near the bottom (LlamaIndex reports)	A third-party parser such as Reducto or LlamaParse, which adds a processor of evidence. Trigger: Claude misses the field-accuracy target on CCP's fixtures, such as SOC 2 exception tables
Chunk and add context	Split on structure (headings, policy and report sections, whole table rows) within pages, store code-point offsets into the canonical text, which chunks lack today (`storage/migrations/0008_chunks.sql:10-29`), and tune size per evidence type. Keep contextual enrichment, on Haiku 5.5 with thinking off and cached by content hash, and give it the whole document within the 100,000-token price band instead of a 60,000-character cut (`retrieval/contextual.py:17`, `:43`): about $0.003 for an 8,000-token policy and under $0.20 for a report near the limit, once per version (inferred)	Prepended context cut top-20 retrieval failures by 35%, and by 49% with the context also in the lexical index, for $1.02 per million document tokens at 2024 prices (Anthropic, Sep 2024). Vanta measured +16% recall from context-aware chunking on its compliance content (Vanta, Feb 2025). Chunk size is "a hyperparameter to tune" (Husain, Sep 2026)	`voyage-context-4`, a contextualized-chunk embedder at $0.12 per million tokens with no model calls, which gives the lexical arm nothing (MongoDB). Trigger: enrichment cost or ingest time becomes material
Index	Exact search over each tenant's own rows by default; LIST partitions with their own HNSW index only for the largest tenants (6.3, U13), indexed as `embedding::halfvec(1024)` to roughly halve index memory (inferred), on pgvector 0.8.3 or later. Keep `ts_rank_cd`: its GIN index goes unused under row-level security (Decision 8), so the lexical arm scans the tenant's rows, which suits small tenants (inferred). Check hot plans as the app role	pgvector supports half-precision expression indexes and recommends list partitioning for multitenancy (pgvector); 0.8.3 fixed index corruption during HNSW vacuuming (pgvector, Jun 2026). pg_textsearch warns that "BM25 corpus statistics include all indexed rows, including rows hidden by RLS", while "BM25 statistics are local to each partition" (pg_textsearch)	BM25 with `pg_textsearch`, only ever on a tenant's own partition: on a shared table one tenant's vocabulary would shift every tenant's scores (inferred). Trigger: identifier recall or a large tenant's lexical latency misses target, and RDS allows the extension, which needs `shared_preload_libraries`. Binary quantization waits for a halfvec index that outgrows memory

Layer	Decision for CCP	Frontier precedent	Not now, and the trigger
Query understanding	Tier 2 queries stay templates rendered from the test definition (`evaluators/tier2.py:99-116`): deterministic, stored, never rewritten by a model. Evidence type, subject, period and as-of are typed filters, not query words. Agents rewrite and split questions themselves with several narrow searches. Which tests apply is decided by code from scoping facts, never by retrieval	OpenAI's file search offers query rewriting and attribute filters (OpenAI). Multi-hop and open-ended queries are the hardest for retrievers (ViDoRe V3, Nov 2025). Fin warns that "a missed rule can alter the response" (Fin, Sep 2026)	A query-rewriting model in tier 2. Trigger: the golden set's paraphrase slice misses its recall target after reranking
Hybrid search and fusion	Keep one statement, both arms on every query, RRF at k = 60. Add per-call arm weights only where the golden set shows a gap, such as identifier queries that need the lexical arm, fitted from the ranks the step log already keeps (`evaluators/tier2.py:178-185`)	Linear runs full-text and vector queries in parallel, then rank fusion and reranking (turbopuffer, Jun 2025); OpenAI exposes fusion weights per arm (OpenAI). Vanta measured +9% recall from BM25 plus embeddings with RRF (Vanta, Feb 2025)	Learned fusion. Trigger: fitted weights still leave a measured gap on a query slice
Reranking	Move to `rerank-3`, a cross-encoder at the same price, and keep its top score as the input to the distractor-calibrated confidence cap (`evaluators/tier2.py:305-308`). Rerank 50 candidates instead of 20 only if Recall@20 is the gap; that costs under a tenth of a cent a query (inferred)	Reranking cut Anthropic's top-20 failures from 2.9% to 1.9% (Anthropic, Sep 2024), and on 50 candidates added 21% MRR at Vanta (Vanta, Feb 2025). LLM rerankers lead on familiar queries but generalize unevenly and take minutes where cross-encoders take seconds (arXiv, Aug 2025)	An LLM or fine-tuned reranker; Intercom reports that one distilled from an LLM beat Cohere Rerank 3.5 on MAP (0.612 against 0.521) at 80% lower cost. Trigger: thousands of consented labels (last row) and a way to calibrate its score
Agentic search tools	The Investigator, Policy Agent and Copilot get, through the registry: `search_evidence` with the subject, period and as-of filters it lacks (`agent/tools.py:39-42`); `find_exact` for control ids, CVEs, ARNs, emails and dates; `outline` and `read_span` to navigate a document version, then read it by page or offsets; and typed fact tools. Results carry version ids and offsets; every call is logged and budgeted. Agents get both kinds of search because the measured gains come from combining them (inferred)	Anthropic calls semantic search faster but "less accurate" than agentic search and suggests starting agentic (Anthropic, Sep 2025). Cursor reports that adding semantic search to its agent's grep raised answer accuracy by 12.5% on average. A stronger retriever lifted a GPT-5 research agent from 55.9% to 70.1% with fewer searches (BrowseComp-Plus, Aug 2025)	A trained retrieval sub-agent or embeddings trained on agent traces, as Cognition and Cursor built. Trigger: searches per verified citation stay high after these tools ship, and enough consented labels exist

Layer	Decision for CCP	Frontier precedent	Not now, and the trigger
Structured facts instead of a document graph	Build no graph from documents. The links already exist as typed, versioned facts (person, account, grant, asset, vendor, report period), so a multi-hop question such as "which leavers still hold GitHub admin?" is a join inside a typed tool, with as-of dates and explicit truncation (7.1). Chunks keep their subject entity for entity-scoped search (`storage/migrations/0008_chunks.sql:15`)	Building a knowledge graph kept only about two thirds of answer entities, graph-triplet retrieval reached 39.2% accuracy against 88.6% for plain retrieval, and graph methods won only on multi-hop questions, by about 3 points (Han et al., Nov 2026)	A document-derived graph or model-written SQL: see **Not now**
Long-context reads	Read one whole document when the task is exhaustive: a vendor SOC 2 report's exceptions, complementary user entity controls, carve-outs and opinion, or a policy against a clause list. Send canonical text rather than the PDF where a text layer exists, extract verbatim quotes first and check them (5.4). Keep retrieval for questions across a tenant's documents	Claude takes up to 600 PDF pages per request, but dense PDFs can fill the window first (Anthropic). Accuracy fell with input length across 18 models (Chroma, Jul 2025), and 11 of 13 models fell below half their short-context score at 32K tokens (NoLiMa, Feb 2025). Anthropic advises extracting word-for-word quotes first beyond 20K tokens (Anthropic)	Long context as the default across documents: 400K tokens cost about $0.80 per uncached Sonnet 5.5 question, against half a cent for six chunks (inferred). Trigger: a current model shows no loss at 200K tokens on CCP's own set
Citations	CCP's pointer citations stay the contract: document version, offsets and exact text, checked by code (5.4). Agent and Copilot tools return `search_result` blocks whose `source` is a stable `ccp://` URI for the version and chunk and whose content is one text block per sentence, so code maps each cited block range to offsets and runs the same checks. Tier 2 keeps structured outputs with quoted spans (3.7)	Anthropic guarantees only that citations "contain valid pointers" (Anthropic). `search_result` blocks hold text only, citations are all-or-nothing per request, a tool result holding one may hold nothing else, and JSON-only prompts may get few citations (Anthropic). Harvey cites sentence indices instead of fuzzy-matching model text (Harvey, Apr 2026)	Native citations in tier 2, which return a 400 alongside structured outputs. Trigger: the API accepts both together

Layer	Decision for CCP	Frontier precedent	Not now, and the trigger
Permissions, as-of filters and erasure	Tenant scope stays in SQL (forced row-level security, later the tenant's partition), never a filter after search; when user roles arrive, a visibility class on sensitive evidence such as HR files joins the same policy. Chunks become insert-only rows per evidence version (6.1, row 3) with validity dates, so search and replay can ask "as of" any date; HNSW covers current rows only and past dates search exactly (inferred). Today the index keeps current evidence only and deletes the rest (`retrieval/index.py:76`, `:188-192`). Erasure removes a version's chunks, cache rows and index entries in one job (U10)	Harvey enforces access "at the database layer so that unauthorized vectors are never retrieved", and its embeddings follow the source document's encryption and retention (Harvey, Apr 2026). Glean checks source-derived permissions as it serves each result and removes content the source no longer returns (Glean)	Live per-user permission checks against each source, as Glean does. Trigger: customers expect per-user source permissions mirrored inside a tenant
Embedding and reranker models	Move to `voyage-4-large` at 1,024 dimensions and `rerank-3`: embed into new rows under a new index version, keep old rows for replay (6.3), recalibrate the thresholds kept per model pair (`retrieval/config.py:38-45`), and switch after a paired golden-set comparison. Today a model change overwrites vectors in place (`retrieval/index.py:160-169`). Keep an open-weight embedder as a tested fallback, not a running service	`voyage-3-large` has been Legacy since 15 Jan 2026 and `rerank-2.5` since 29 Sep 2026 (MongoDB, Oct 2026). All 4-series embeddings "are compatible with each other" (Voyage, Aug 2026), so a cheaper query model could share the index (inferred). Public scores "can become inflated", so RTEB keeps private sets (Hugging Face, Oct 2025)	Running an open model such as Qwen3-Embedding-8B, the retriever behind the BrowseComp-Plus gain. Trigger: a contract that excludes MongoDB as a processor, or a Voyage outage longer than the lexical-only fallback can carry (`retrieval/hybrid.py:176-180`)
Retrieval evals and gates	Grow the 30 golden queries over six evidence types (`spec.md:1645`) to 25 to 50 per type, sliced by query kind (identifier, paraphrase, multi-hop, negation) and tenant size. Report Recall@20 before reranking, Recall@6 after it, MRR, and the share of queries with a distractor in the top 6; a model, parser, chunker or prompt change ships only if recall holds and that share does not rise (5.3). Add free positives from spans that passed the 5.4 checks and a reviewer accepted, keep a private holdout, and calibrate, so the flat 0.60 retrieval factor every tier-2 verdict gets today becomes measured (`evaluators/confidence.py:23`, `:58-60`; `config/retrieval.yaml:8`)	Evaluate retrieval separately and first, with IR metrics (Husain, Sep 2026). RTEB reads a large gap between public and private results as overfitting (Hugging Face, Oct 2025). Cursor reports pairing an offline benchmark with an online test of code retention, and labelling embedding training data from agent traces	Online tests per retrieval setting. Trigger: enough volume per workflow to detect a difference

Tier by tier.

Path	Retrieval it uses	Pinned for replay
Tier 2 extraction	One templated hybrid search per test and subject, filtered by type, subject and as-of, reranked to six; no tools and no loop	The query, the index version (parser, chunker, embedding and rerank models), candidate ids with ranks, chunk ids with text hashes, and the fenced request and response. Replay rebuilds from stored text and never searches again; today candidates sit only in the step log (`evaluators/tier2.py:178-185`)
Tier 3 agents (Investigator, Policy Agent)	A budgeted loop over search, exact match, outline, read span and typed fact tools, after any fixed fetches the graph declares; the Policy Agent reads policy documents only (`agent/graphs/policy.py:4-5`)	Every tool call and result in the run log (3.4), with the version ids and offsets each returned
Copilot	The same read tools, as of the date asked; status answers come from rule outputs and the rollup, never from documents	The turn's tool calls, results and cited spans
Questionnaires	Code first maps each question to tested controls and their current verdicts; search runs only for supporting prose and unmapped questions (LT3)	The mapping version, chunk ids per answer and the verdicts relied on; an answer goes stale when any of them changes
Evidence review	No search: the document under review is read whole, or by outline and span when long, quotes first	The document version and text hash, the transcription method if any, and the extraction request and response

Not now.

- **GraphRAG.** Typed facts already hold the links, and document-built graphs lose entities and single-hop accuracy (see the structured-facts row). Trigger: a corpus-wide question that summaries over typed facts cannot answer, such as recurring

exceptions across every vendor SOC 2 report; then trial a LazyGraphRAG-style index, whose indexing costs the same as vector search (Microsoft, Nov 2024), in a drafting feature only.

- **Visual (ColPali-style) retrieval.** The best visual retrievers reach about 0.65 NDCG@10 on enterprise pages (ViDoRe V3, Nov 2025), page-image vectors cannot carry text citations that code checks, and screenshots are mostly found by control, system and date. Trigger: transcription measurably loses what retrieval needs on CCP's fixtures, such as architecture diagrams.
- **Text-to-SQL for any status.** Never: a status is a versioned rule over facts (Decision 1), so no trigger applies. Read-only model-written SQL may later serve Copilot exploration over curated views under row-level security, with timeouts and row caps, labelled exploratory, once Copilot logs show questions the typed tools cannot answer.
- **A dedicated vector engine.** Linear left Elasticsearch and pgvector at 2B+ documents in 4M+ namespaces to cut operations and keep tail latency flat (turbopuffer, Jun 2025); Harvey's dual writes show the cost: "In a dual write system drifts are inevitable" (Harvey, Oct 2026). Trigger: p99 latency that partitions cannot hold flat, per-tenant keys required on the index itself, or millions of chunk writes a day for one tenant.

6.6 Evals, guardrails and operations: decisions by layer

Section 5 fixes the graders, metrics, gates and guardrails, and rows 8 to 10 of 6.1 name what to build; this subsection records only what frontier practice in 2026 changes or adds. The labs' own postmortems and studies show quality and safety slipping at the seams around the model: a harness or prompt edit, drift in serving, where untrusted text sits in a prompt, which model actually answered, and what a trace carries out. Each row gives the decision, the precedent behind it, and what waits for a trigger.

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
8. Prompt and harness changes are releases	A change to any prompt line, tool description, effort setting, model id or graph edge makes a new bundle; today the effort level is fixed in client code (`llm/anthropic_client.py:69-71`). The bundle runs the nightly suite on every model that uses the prompt, and a change of several prompt lines that moves a metric is retested one line at a time. A bundle change demotes a workflow's autonomy level (4.4) until a shadow soak passes	In April 2026 Anthropic traced a Claude Code quality drop to three product changes, among them a lower default reasoning effort and a system-prompt line that had passed weeks of internal evals; a broader test later showed that line cost 3%. Anthropic now runs a per-model suite on every system-prompt change, tests prompt lines one at a time, and uses soak periods and gradual rollout (Anthropic, Apr 2026)	A/B tests on customer traffic. Trigger: a drafting surface where a person approves every output and volume supports a 95% interval, read as Intercom does: significant only when the whole interval is above zero (Intercom Help). Status-deciding steps keep shadow (5.3)
8. Drift and infrastructure noise	A fixed canary set of synthetic evidence with known answers runs daily through the production path (same organization, platform and bundle), and an alert fires when the lower bound of its accuracy drops below baseline. Timeouts, rate limits and connector errors are logged as an infrastructure class: they still fail the gate (5.2), but a run with many of them is re-run, not read as a quality change. No bundle or autonomy level is promoted on a gain under about 3 points unless it repeats across days (inferred)	Anthropic's 2025 quality bugs came from serving infrastructure under unchanged model ids, and its evals "didn't capture" them; it now runs quality evals continuously on production systems (Anthropic, Sep 2025). Resource settings alone moved one coding benchmark by 6 points, and Anthropic treats gaps under 3 points with skepticism (Anthropic, Feb 2026)	Canaries per cell and platform. Trigger: a second cell, or an EU cell on Bedrock (LT9)

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
8. Gold sets sized to the false-PASS bound	Weight each workflow's gold set towards FAIL cases, since only they measure false PASS: about 300 expert-labelled FAIL cases with none passed show a rate of at most 1% at 95% confidence, and one false pass raises the need to about 475 (inferred arithmetic). Today's labelled verdict set has four FAIL cases (repository root `evals/labeled/README.md:23`). A FAIL case counts as a false PASS if any of its five runs passes it; labels come from named experts, never from approvals given in CCP; retrieval keeps its own labelled queries (6.5)	HealthBench reports worst-at-k, the worst score across k samples (HealthBench, May 2025). Ramp does not take users' past decisions as ground truth, because finance teams are lenient (Ramp). "Evaluate the two components separately, starting with retrieval" (Husain, Sep 2026)	Certifying a workflow before its FAIL set reaches the size its bound needs; until then it keeps per-item review. Trigger: U3 sets the bound per test
8. What judges see	Judges get the source spans that code verified, the tool outputs and the final artifact. The agent's reasoning is graded only as prose, never taken as proof of what it did; today the judge reads the agent's whole output, reasoning included, and only the case input besides (`evals/runner.py:252-257`, `evals/judge.py:128-133`). Empty, truncated, punctuation-only and "Thought process:" outputs join the negative controls (5.3), and the judge must fail them all	Rewriting an agent's reasoning while keeping its actions fixed raised web-agent judges' false-positive rates by up to 90% (Gaming the Judge, Jan 2026). Content-free replies such as ":" or "Thought process:" won false positives from judges including Claude-4 (One Token to Fool, Jul 2025)	Several judges voting. Anthropic found one judge prompt more consistent than several (Anthropic, Jun 2025). Trigger: a judge misses the 85% expert-agreement bar (7.1)

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
8. Drafts: pairwise comparison and simulated users	Choose drafting models and prompts (questionnaire answers, policy drafts, Copilot answers) by blinded side-by-side comparison, three experts per pair. Simulated auditors and customers, scripted to be vague and uncooperative, serve only as regression tests for the Copilot and questionnaires, never for quality or autonomy claims. Neither gates a status-deciding step	Pairwise expert judgments recovered legal quality rankings far better than rubrics (per-task Spearman 0.908 against 0.150) in under half the annotation time (JudgmentBench, May 2026); Harvey has 3 to 10 lawyers review each pair (Harvey, Nov 2025). Simulated users are "excessively cooperative": with them, agent success reached up to 77.8%, against 63.6% with real people (Sim2Real, Mar 2026)	Checking simulators against real sessions each quarter. Trigger: the Copilot reaches customers (UI v1, MT4)
9. Untrusted text in the prompt	Evidence reaches the model only as tool results or document blocks, never pasted into the instruction text, and the attack suite below tests each placement before it ships. Strip invisible Unicode from evidence and from pasted Copilot questions, and mark pasted text as quoted. Today tier 2, the structured agent nodes and the Copilot's answer step paste fenced evidence or tool output into the text of the user message (`evaluators/tier2.py:257-260`, `agent/core.py:233-236`, `agent/graphs/copilot.py:291-296`)	Anthropic reports that early Opus 5.5 snapshots followed instructions planted in pasted user text in 52% of attempts but never in tool results (0 of 105); the released model still did about 2% of the time at default effort, and never once invisible characters were stripped and pasted text marked (Opus 5.5 system card, Sep 2026, self-reported)	Scans and screenshots as model input (LT2). Trigger: the suite covers instructions hidden in images

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
9. Which model answers	Never opt in to API safety fallbacks on a tool-using graph; a refusal still becomes needs_review (3.7). Check on every turn that the model the response reports is the bundle's model, and route the run to review if not. CCP sends no fallback setting today and records the model each response names (`llm/anthropic_client.py:202-208`, `:232`)	On the API, automatic fallbacks are opt-in; on a cyber-classifier block, Sonnet 5.5 falls back to Sonnet 5 (Anthropic, Sep 2026). Anthropic reports that none of the 2,872 adaptive-attack requests Opus 5.5 answered itself were compromised; its attack success came from turns its fallback, Opus 4.8, served (Opus 5.5 system card, Sep 2026, self-reported)	Fallbacks on tool-less extraction calls. Trigger: U5 finds refusals too frequent for review to absorb, and the fallback model passes the same gates
9. An attack suite built from CCP's evidence	Plant instructions in every evidence type CCP reads (vendor PDFs, tickets, cloud tags, GitHub text, pasted questions) and in the approved-answer library, aimed at changing an extracted field. Count a success when a field changes and nothing surfaces it, per attempt and per scenario at k tries, and refresh the attacks each quarter and on every model change. Today's suite plants text in one policy chunk and counts only status flips through tier 2 (`evals/adversarial.py:1-5`)	In a contest of 272,000 attack attempts by 464 people on 13 frontier models, every model was vulnerable (attack success 0.5% to 8.5%), and an attack counted only if the agent hid it from the user (IPI Arena, Mar 2026). Existing injection defenses "fail to cover" memory poisoning (Dash et al., Jun 2026). Anthropic's system cards report Opus 5's adaptive attack success rising from 0.56% (Jul 2026) to 88.92% (Sep 2026) once the attacker improved (self-reported; fallback routing also changed)	A third-party adaptive red team, and a bug bounty that pays for a false PASS or a cross-tenant read. Trigger: the first production customer (Gate 2) for the red team; open sign-up for the bounty

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
9. Authorization outside the model	Make `policy.check` default-deny: new permit rules first log what they would allow, denied tools leave the graph's tool list, and three denials in a row stop the run for review, as in Claude Code's auto mode (3.6). Each graph type runs as its own principal, recorded with tenant, user and run on every registry call, and never exceeds the rights of the person who started it; scheduled runs use a read-only tenant principal. Only the executor fetches a credential from the broker (5.5), one per approved operation	AWS's agent policy service blocks every tool call by default, opens calls only on conditions over arguments and identity claims the model cannot alter, and has a log-only mode (AWS, May 2026, AWS, Mar 2026). MCP's enterprise-managed authorization became stable in June 2026, so a company's identity provider can grant agent access centrally (MCP, Jun 2026); Auth0's Token Vault keeps third-party tokens out of agent code (Auth0, Nov 2025)	Cedar or a managed policy service, enterprise-managed authorization on CCP's MCP server, and agent principals in customers' own directories. Trigger: customers' agents call CCP's tools (MT3), or the rule table outgrows a page (6.1, row 6)
9. Review that catches errors	Reviewers first mark each AI-read field right or wrong; correcting is a separate step. Seed known-wrong items into CCP's internal expert review at a low rate, report each reviewer's catch rate and accept-all streaks, and show rule-aware hints for a wrong scope, period or entity	In a preregistered study of 2,784 people checking AI-extracted figures from corporate reports, errors that needed a grasp of the rules were corrected only 31% of the time, and time spent did not predict accuracy; the authors advise separating judging from correcting and flagging reviewers who accept or reject everything (Beck et al., Sep 2025)	Seeded items in customers' approval queues. Trigger: a customer opts in; a seeded item can never execute

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
9. Standards buyers will ask about	Map CCP's agent controls to the OWASP Agentic Top 10 and AIUC-1's six domains, and refresh the map each quarter. Keep every AI output labelled as such (the provenance badges, 6.1 row 11), and ask counsel before 2 Dec 2026 whether the AI Act's Article 50(2) duties reach CCP's drafts	The AI Act omnibus moved Annex III high-risk duties to 2 Dec 2027 and Annex I to 2 Aug 2028, while some Article 50(2) transparency duties apply from 2 Dec 2026 (FPF, Jul 2026). AIUC-1 certifies agents after typically 1,000 to 5,000 red-team scenarios with quarterly retests, and crosswalks to ISO/IEC 42001 (AIUC-1, Oct 2026). NIST's planned agent control overlays have no public draft (NIST, Oct 2026)	AIUC-1 or ISO/IEC 42001 certification. Trigger: a buyer's procurement requires one
10. Trace export and replay	A deterministic hook replaces evidence text, quotes and free text with chunk ids and hashes before any span leaves CCP's account, and a CI test fails if one carries evidence text. Engineers see content only through an access-controlled, audit-logged replay from the run log. Today the tier-2 step hands the trace sink its extracted fields, verbatim quotes included, masked only for secret patterns (`evaluators/tier2.py:293-295`, `evaluators/extraction/base.py:8-14`, `security/secrets_patterns.py:65-69`)	Langfuse's masking hook runs in the application at export time, and Langfuse does no masking on its servers (Langfuse docs). Anthropic's limits on viewing user data slowed diagnosis of its 2025 quality bugs; it responded with privacy-preserving debugging tools (Anthropic, Sep 2025)	Anthropic's OpenTelemetry export, in beta since October 2026 (Anthropic, Oct 2026). Trigger: its documentation shows it can carry metadata only

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
10. Cost and capacity	Alert when the cached share of input tokens drops after a deploy, and use the API's cache diagnostics to find the break; cache tokens are already logged per step (`observability/steps.py:148-153`). Report cost per accepted result, counting failed, retried and refused runs, with reviewer minutes beside it. CCP's token buckets keep about half the rate limit for interactive work, and background jobs yield first when 429s rise (inferred)	Claude Code alerts on its cache hit rate and declares incidents when it falls (Claude blog, Apr 2026); the API's `cache_miss_reason` left beta in September 2026 (Anthropic, Sep 2026). In Datadog's 2026 data only 28% of calls to caching models read any cached tokens, and in February rate limits caused 60% of call errors (Datadog). Counting review and rework cut GPT-5's naive 474-fold cost saving to 1.63-fold (GDPval, Oct 2025)	Cache alerts before the Haiku 5.5 move, since tier-2 prompts never reach Haiku 4.5's 4,096-token minimum (3.7). Trigger: MT7's migration
10. Model lifecycle and the Bedrock path	Run each new Claude release in a tier against the incumbent on the frozen gold set and a hard-case suite within a month, even when no switch is planned, so a deprecation notice never starts the work. Keep the migration runbook short enough to finish in six weeks, and re-certify every autonomy level after any model or platform change. Run the gold suite on Bedrock each quarter and before a failover, and track Bedrock's retirement dates apart from Anthropic's	Anthropic shipped about a dozen models between February and November 2026, and three 2026 deprecation notices set retirement about two months out (Anthropic, Nov 2026). In its 2025 incidents, misrouting hit 16% of first-party Sonnet 4 requests at the worst hour against 0.18% on Bedrock, so one model id can behave differently by platform (Anthropic, Sep 2025)	Automatic failover to Bedrock, or sending it load. Trigger: an availability commitment that a manual switch cannot meet, or sustained 429s at the top tier, once Bedrock passes the same gates

Layer and topic	Decision for CCP	Frontier precedent	Not now, and the trigger
10. Publishing the false-PASS rate	Publish per workflow and release: the false-PASS upper bound with its count of FAIL-labelled cases, pass^5, the bundle's model and prompt versions, the date of the last re-certification, and the method (dataset version, labeller qualifications, run settings). Publish only bounds that rest on gold sets of the size above (LT6)	Intercom lets customers test its agent inside the product on simulations built from their own conversations (Intercom, Aug 2026). Anthropic asks that eval results name their resource settings and generator seeds (Anthropic, Feb 2026, Anthropic, Dec 2025)	Customers or auditors running CCP's suites themselves. Trigger: an auditor or enterprise buyer asks (LT4's auditor workspace)

6.7 Context engineering and long-running work

Sections 3.4 and 6.4 already rebuild every prompt from the run log and keep runs short, and 6.5 decides when to read a whole document. This subsection sets the numbers and rules inside that frame: how much each call may hold and in what order, how thinking is set and carried, when a 1M-token window pays, and how work that spans days or waits on a person becomes short runs that code can resume and check. Where labs and agent companies differ, CCP takes the option that leaves a typed record a reviewer can replay.

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
A context budget per call type	Each call type declares an input ceiling in its bundle; the view builder counts tokens before sending and drops the lowest-priority items first, never instructions or a span a draft cites. Starting ceilings, tuned on evals: tier-2 chunk extraction 20K tokens; whole-document extraction 90K on Haiku 5.5, else Sonnet 5.5 up to 200K or split by section (6.5); an agent's first call 15K and any later call 50K; a Copilot turn 30K (inferred)	Anthropic says models have an "attention budget" that they draw on as context grows, since n tokens form "n² pairwise relationships" (Anthropic, Sep 2025). Manus reports an input-to-output ratio "around 100:1" and about 50 tool calls per task, so input is where cost sits (Manus, Jul 2025)	A model that chooses what enters its own context. Trigger: evals show the fixed priority order dropping items that answers needed
Prompt composition and order	After the cached prefix of 6.4, the variable part runs: typed task and item state, then documents and tool results, then the step's question. Code appends a one-line reminder of the item's question and open checks after the newest tool results on every turn. When several questions go to one document, the document joins the cached prefix and only the question changes (inferred)	Anthropic advises placing long documents "near the top of your prompt, above your query", and reports that "Queries at the end can improve response quality by up to 30 percent in tests" (Anthropic docs, undated). Manus has its agent rewrite a todo.md as it works, to avoid "lost-in-the-middle" issues (Manus, Jul 2025)	A model that edits its own plan, as Manus's agent does. Never for status work: code renders the checklist from the item row

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Preload or fetch just in time	Preload only what every step needs and is small: the test or question definition, the item's typed facts and prior verdict ids. Fetch evidence text, past investigations and long results by handle (6.4); a tool result over about 5,000 tokens is stored and replaced by a preview, tighter than LangChain's 20,000 because CCP's runs are short (inferred). Tier 2 stays fully preloaded, with no tools	Anthropic recommends a "hybrid strategy, retrieving some data up front for speed", as Claude Code loads CLAUDE.md up front and uses glob and grep for the rest (Anthropic, Sep 2025). Deep Agents swaps results over 20,000 tokens for "a file path reference and a preview of the first 10 lines" (LangChain, Jan 2026)	Anthropic's server-side tool-result clearing, which by default fires at 100,000 input tokens and keeps the last three tool uses (Anthropic docs, undated). CCP's view does the swap itself, so the run log shows exactly what the model saw (inferred). Trigger: an agent loop whose ceiling must pass 100K

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Progress records for multi-run tasks	Each long task keeps its progress as typed Postgres rows: a task row and one row per item (status, attempts, evidence ids, open question, last run id). Only the runner changes an item's status, after the 5.4 gates pass. A run may leave a short hand-off note in the run log, which the next run reads as a hint, never as evidence (inferred)	Anthropic's long-running harness tracks over 200 features in a JSON list whose pass flag is all a coding session may change, since the model "is less likely to inappropriately change or overwrite JSON files compared to Markdown files" (Anthropic, Nov 2025). Anthropic's docs advise "structured formats for state data" and git as the log (Anthropic docs, undated)	Free-form notes, such as the NOTES.md file in Anthropic's post (Anthropic, Sep 2025), as the record. Never; per-tenant notes wait for the memory trigger in 6.4

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Sub-agent context isolation	A fanned-out item (6.4, Orchestration) runs in a fresh context holding only its question, typed facts and handles, never the parent's history. It returns a typed result of at most about 1,500 tokens (ids, offsets, fields, a one-line reason); the parent sees only that, and the full sub-run stays in the run log (inferred)	Anthropic's sub-agents may each use "tens of thousands of tokens or more" yet hand back a short summary (3.4) (Anthropic, Sep 2025). Its long-running app harness linked planner, generator and evaluator through files, not shared context: "Communication was handled via files" (Anthropic, Mar 2026)	Giving a sub-agent the parent's whole context. Trigger: an item that cannot be stated without the parent's reasoning, which usually means the item list is wrong (inferred)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Thinking and effort settings	Each bundle sets thinking and effort per call type; nothing relies on defaults, which differ by model. Tier 2 on Haiku 5.5 runs with thinking off, checked on fixtures against low effort; tier 3 on Sonnet 5.5 stays at medium, with high only where a sweep shows a gain. Today only Sonnet, Opus, Fable and Mythos ids get thinking settings, so tier 2 moved to Haiku 5.5 unchanged would think by default inside a 2,048-token cap and could stop mid-JSON (`llm/anthropic_client.py:58-79`, `:198`; `evaluators/tier2.py:278`; inferred)	Defaults are medium on Opus 5.5 and Haiku 5.5 and high on Sonnet 5.5, whose levels "are recalibrated", so Anthropic says to run "a fresh effort sweep" and, for multistep tool use, "start with `medium` for well-specified tasks". In long agent prompts at `low`, Haiku 5.5 "is more likely to skip a search, stop early, or skip a check" (Anthropic docs, undated). Thinking is "On by default" and counts toward `max_tokens` (Anthropic docs, undated)	`xhigh` and `max`, aimed at runs "over 30 minutes" with "token budgets in the millions" (Anthropic docs, undated). Trigger: an Opus 5.5 fresh-run tier (6.4, Model routing) measures a gain at them

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Thinking blocks across tool turns	Within a run, the view passes every thinking block back byte for byte, as CCP's loop already does (`llm/anthropic_client.py:121-122`, `:328-329`; `agent/core.py:333`), and the run log keeps them so a resumed step rebuilds the same turn. Thinking and effort never change mid-run. With `display` left at "omitted", CCP holds only signed, encrypted blocks, and no reviewer or judge sees reasoning as justification (6.6)	Anthropic: "within a tool-use turn, pass thinking blocks back"; "Modified thinking blocks are rejected with a 400 error"; on the 5.5 models earlier thinking stays and counts "as input like any other conversation history". A thinking or effort change "starts a new cache prefix", and visible thinking "is never the raw chain of thought" (Anthropic docs, undated)	Thinking-block clearing, which invalidates the cache "at the point where clearing occurs" (Anthropic docs, undated). Trigger: Copilot sessions that carry turns (6.4) and whose kept thinking fills over about a quarter of the turn's ceiling (inferred)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
1M-token windows	All four current models take 1M tokens, but CCP keeps the ceilings of the first row. The whole-document ceiling rises past 200K only after a position test: CCP documents padded to 100K, 200K and 400K tokens, with known facts placed early, middle and late, scored by checked citations (inferred). Above the ceiling, code splits by report section and merges, as 6.5 does across documents	Fable 5.2, Opus 5.5, Sonnet 5.5 and Haiku 5.5 each list a 1M window (Anthropic docs, undated). There "1M is the default: you don't need a beta header", yet "As token count grows, accuracy and recall degrade" (Anthropic docs, undated); Anthropic saw models "lose coherence on lengthy tasks as the context window fills" (Anthropic, Mar 2026)	Fable 5.2 at $8/$40 per million tokens, which Anthropic suggests "when your evals on Claude Opus 5.5 at higher effort still fall short" (Anthropic docs, undated). Trigger: exactly that, on CCP's evals

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Long-context cost and caching	Haiku 5.5 prompts stay under 100,000 tokens, cache reads included, hence the 90K ceiling. A document asked many questions sits in the cached prefix: 20 questions over a 150,000-token report on Sonnet 5.5 cost about $0.66 of input with a 5-minute cache, against $6.00 uncached (inferred arithmetic). The 1-hour cache, at twice base input to write, serves only read sessions with gaps over 5 minutes (inferred)	From Claude 4.6 on, models other than Haiku 5.5 bill the 1M window at standard rates: "A 900k-token request is billed at the same per-token rate as a 9k-token request". A Haiku 5.5 request over 100,000 tokens pays five times the rate "even when part of its prompt is a cache hit"; Sonnet 5.5 cache hits cost 0.05 times base input (Anthropic docs, undated)	The Batches API at 50% off (Anthropic docs, undated) for overnight audit-prep extraction. Trigger: its zero-retention eligibility is confirmed (not checked here) and audit prep can wait for batch results

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Multi-day tasks as resumable runs	A long task starts with a code step, not a model, that writes the item list from scoping facts: tests in scope for audit prep, questions mapped to controls for a questionnaire, requested documents and checklist items for a vendor review. Each item runs as short queued runs, each in a fresh context seeded from the task and item rows; resuming means taking the next open item. A model may propose new items, which code adds after checks (inferred)	Anthropic's harness has an initializer session write the feature list, an `init.sh` and a progress file; each later session reads "the git logs and progress files", picks "the highest-priority feature that's not yet done" and first runs a basic test (Anthropic, Nov 2025). Cursor's week-long runs still need "periodic fresh starts to combat drift and tunnel vision" (Cursor, Jan 2026)	One agent working for hours across an audit. Anthropic dropped context resets from one harness once Opus 4.5 "largely removed" context anxiety, and reports a run of about 4 hours and $124 (Anthropic, Mar 2026); CCP keeps short runs for replay and budgets, not model limits (inferred). Trigger: a task whose items code cannot list; a planning run then proposes the list and code stores it

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Drift, loops and premature completion	Code watches each run: a repeated identical tool call gets the stored result with a note, and a third repeat ends the run in `needs_review`. A task is done only when code finds every item final with its gates passed; no model marks anything done. Per task, code tracks items done, retries per item and runs without progress, and parks the task for a person after three runs that change nothing (inferred)	Anthropic saw later sessions "see that progress had been made, and declare the job done", and Claude "mark a feature as complete without proper testing" (Anthropic, Nov 2025). Cursor reports that Opus 4.5 "tends to stop earlier and take shortcuts when convenient" (Cursor, Jan 2026). LangChain calls goal drift "the most insidious failure mode" (LangChain, Jan 2026)	A model judge that decides whether work continues, as Cursor's does after each cycle (Cursor, Jan 2026). Trigger: a task whose completion code cannot check, such as a readiness narrative; the judge then flags and a person decides

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Resuming after a human wait	A wait holds no context (3.5). When the person answers, code refreshes facts and re-runs the rule, which may settle the item with no model call. Otherwise a new run starts from the stored question, the answer as quoted, untrusted text (6.6), the typed state and handles to the evidence cited before, re-read as of now; old messages, thinking and cache are not reused (inferred)	Anthropic's docs: "design your state artifacts so that context recovery is fast when a new session starts" (Anthropic docs, undated). Cache entries last 5 minutes or 1 hour (Anthropic docs, undated), so after a wait of hours nothing cached survives anyway (inferred)	A model-written hand-off summary as the seed, in the style of Amp's Handoff (6.4). Trigger: answers that only make sense with the earlier conversation, such as multi-turn auditor clarifications; the summary is then stored in the run log and a person can edit it before use

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Per-task token and time budgets	Keep per-run caps, but count tokens as the peak context of any one call plus dollars, not a running sum. Today cache reads count in full against the 60,000-token cap and cache writes not at all (`llm/anthropic_client.py:316`, `llm/budget.py:74`, `config.py:91-93`), so an eight-turn loop can trip the cap while costing about ten cents (inferred). Each task row adds caps: dollars forecast from item count times the p95 cost per item, a deadline, at most three runs per item, and an alert at 1.5 times the forecast (inferred)	Anthropic: "Effort is soft guidance. `max_tokens` is a strict limit." (Anthropic docs, undated). The 5.5 models get no injected token-budget tags; Anthropic points to task budgets, in beta, instead (Anthropic docs, undated). Anthropic reports a 6-hour, $200 harness run against 20 minutes and $9 for a solo agent (Anthropic, Mar 2026)	Telling the model its remaining budget through task budgets; 6.4 (Model routing) sets that trigger

How CCP's long tasks run. The plans below are inferred from the rows above; only the cited facts are sourced.

- **Audit preparation, over weeks.** Items are the in-scope tests for the audit period, written from scoping facts. Most settle in tier 1 with no model; the gap list is a query over item rows, not a model summary.
- **Vendor review, over days.** Items are the requested documents and the review checklist. Each document that arrives gets one whole-document read, quotes first (6.5), cached for its checklist questions; a missing document is a wait row.
- **An investigation that waits on a person.** The run ends by storing its question as a row, and the answer starts a new run; after three runs a person takes the item.
- **A questionnaire of hundreds of questions.** Code maps questions to controls and verdicts first (6.5, Questionnaires). Each remaining question runs in its own call, calls on one topic share a cached prefix of the same policies, and an answer goes stale when a verdict it relied on changes.
- **Why item runs stay minutes long.** METR's horizon is the task length at which an agent "is predicted to succeed half the time", on tasks "designed to be self-contained and well-specified"; METR calls measurements "above 16 hrs" unreliable and noted in

September 2026 that the page is no longer actively updated (METR, Sep 2026). Its 2025 paper found the horizon doubling about every seven months since 2019 (METR, Mar 2025). A coin flip at hours is no basis for audit work, so items stay short and code checks each one (inferred).

- **Context probes join the eval suite.** LangChain plants a fact early, forces summarization and checks that the agent recovers it, and stress-tests by compressing at 10 to 20% of the window (LangChain, Jan 2026). CCP adds four probes per graph: a fact planted early in a long run, re-reading a swapped-out result by handle, resuming an item without redoing finished work, and a cited span placed early, middle and late in a long document.

6.8 Multi-agent systems and harness engineering

3.2 and the Orchestration row of 6.4 set the rule: one writer, and fan-out only from code. This subsection decides how CCP fans out, which multi-agent failures it guards against, where A2A fits, and how it builds, tests and measures the harness around each agent loop. In the 2025-2026 evidence, quality comes more from the harness (prompts, tool descriptions, stop rules and checks) than from adding agents, so that is where a team of three to five should spend its effort (inferred).

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
When to fan out	Fan out only reading work that splits into independent items: vendor reviews (one run per vendor), the questionnaire questions that code could not answer from mapped verdicts (6.5), and audit-prep gap lists (one run per control family). Investigating one failure, reviewing one policy, Copilot turns, control mapping and remediation stay single runs, because each step depends on the last. Code lists the items; no model decides the split	Anthropic's research system scales effort to the query: "Simple fact-finding requires just 1 agent with 3-10 tool calls", while complex research may use more than 10 subagents (Anthropic, Jun 2025). Claude Code's docs say parallel subagents work best "when the research paths don't depend on each other", and keep work whose phases "share significant context" in the main conversation (Claude Code docs, undated)	A model that decides how many workers to start. Trigger: a reading task whose items code cannot list, on which one run scores below the 45% mark in 6.4

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
The fan-out contract	Each item is its own queued run, keyed by parent run, item id and bundle (3.5), with typed inputs (tenant, item, as-of date), a fixed read-only tool list, its own budget, and a Pydantic output whose claims carry pointer citations. Code checks each result (schema, 5.4) and merges by key; a failed item becomes `needs_review` alone and never blocks the rest. Only code starts workers: no model gets a spawn tool	Anthropic's subagent briefs need "an objective, an output format, guidance on the tools and sources to use, and clear task boundaries"; vaguer ones made subagents run "the exact same searches". Its subagents store their outputs and "pass lightweight references back to the coordinator" (Anthropic, Jun 2025). Claude Code, by contrast, lets subagents spawn their own, "up to three layers below the main conversation" (Claude Code docs, undated)	Workers that message each other or share a scratchpad, and fan-out as branches inside one graph thread. Trigger: none; items that need each other's results are one item (inferred)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Planner, worker and judge	The planner is code: scoping facts and the control catalogue produce one checklist row per item, and only code marks a row done. The worker is a model run. The judge is code first (schema, citations, catalogue bounds, the dry run of 6.4), then, for high-value drafts only, a clean-context model reviewer whose verdict is a signal (3.2)	Cursor reports that its weeks-long runs worked only with planners, workers and a judge after each cycle: twenty equal agents with locks ran at "the effective throughput of two or three", and without a hierarchy "agents became risk-averse" (Cursor, Jan 2026). Anthropic's long-running harness starts from a JSON list of over 200 features marked failing, and agents may change only each feature's `passes` field (Anthropic, Nov 2025). Cognition's reviewer shares no context with the coder and, Cognition reports, finds "an average of 2 bugs per PR" (Cognition, Apr 2026)	A model planner. Trigger: an open-ended sweep, such as readiness across every framework, whose items code cannot list; the planner then proposes items, and code checks them against the catalogue before any worker starts (inferred)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Hand-offs between agents	No conversational hand-offs. Work moves between graphs as stored, typed records read by id: a Copilot request for an investigation starts a fresh Investigator run with typed inputs, and its draft is a row that a person or the Policy Agent reads. No run inherits another's transcript or rights	In the OpenAI Agents SDK a hand-off is a tool call, after which the new agent by default "gets to see the entire previous conversation history"; nested history "does not redact sensitive data", and input guardrails run only on the first agent (OpenAI Agents SDK docs, undated). Claude Code subagents get "only this system prompt plus basic environment details" (Claude Code docs, undated)	Routing Copilot questions to specialist agents. Trigger: Copilot logs show intents that one tool list cannot serve (the 20-tool trigger in 6.4); then code routes each intent to a fresh run
Multi-agent failures and their guards	Guard each MAST class in code: specification failures with typed briefs, fixed tools, caps and loop detection (rows below); misalignment between agents by giving them no channel to each other; verification failures with code checks before merge and before approval, never a model reviewer alone	MAST labelled over 1,600 traces from 7 multi-agent frameworks into 14 failure modes in 3 classes, with inter-annotator kappa of 0.88 (Cemri et al., Oct 2025). The commonest were step repetition (15.7%), reasoning-action mismatch (13.2%) and unawareness of termination conditions (12.4%). The authors found that "many existing verifiers perform only superficial checks"; a task-level verification step raised ChatDev's success by 15.6%, yet "not all failure modes are resolved" (Cemri et al., Oct 2025)	A model that watches other agents' runs. Trigger: labelled traces show a failure class that code checks miss

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
A2A beside MCP	Neither expose nor consume A2A now. CCP's outside surface stays the MCP server generated from the registry (3.3): read tools for customers' and auditors' agents, under their own credentials. A message from an outside agent is never an approval or evidence; an A2A endpoint, if added, wraps the same tools and stored proposals with no new rights (inferred)	A2A 1.0.0 shipped on 12 Mar 2026 with breaking changes, including OAuth flows that "remove implicit/password, add device code / pkce"; 1.0.1 followed in May (A2A, May 2026). A steering committee from AWS, Cisco, Google, IBM Research, Microsoft, Salesforce, SAP and ServiceNow maintains it, and the project calls the protocols complementary: "MCP is for agent-to-tool communication", A2A "for agent-to-agent communication" (A2A, undated). Adoption counts appear only secondhand, so none is given	An A2A endpoint. Trigger: two or more enterprise buyers or audit firms ask to exchange evidence requests agent to agent; then map A2A tasks onto registry tools, using OAuth and the signed Agent Cards added in 0.3.0

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
System prompt and starting brief	One prompt file per graph role, in fixed sections: role and scope, what the run must not decide, tool guidance, stop and abstain rules, and the output contract, with a few examples from the labelled set. Code builds the starting brief, as the Investigator's `load_context` node does (`agent/graphs/investigator.py:291-312`), so the model spends no calls finding its bearings. Start small, and add a line only for a failure seen in evals, naming that case (inferred)	Anthropic aims for the "right altitude", between "hardcoding complex, brittle logic" and "vague, high-level guidance", in sections such as `<instructions>`, `## Tool guidance` and `## Output description`, starting from "a minimal prompt" (Anthropic, Sep 2025). Factory layers tool descriptions, system prompts and mid-run "System Notifications", and reports that giving start-up system information as shell output cut redundant commands (Factory, Sep 2025). Cursor: "the prompts matter more" (Cursor, Jan 2026)	Mid-run reminders, such as Factory's notifications. Trigger: evals show runs drifting from the brief in late turns, which 8-call caps make unlikely (inferred)
Tool descriptions are harness code	Tool descriptions, parameter notes and error texts belong to the bundle: versioned, hashed and gated like prompts (6.6, row 8). Each tool gets a contract test that calls it with good, wrong and missing arguments and checks that the typed result says what to try next. A model may draft better descriptions offline from failed traces; a person accepts them, and the eval gate decides	Anthropic's tool-testing agent "rewrites the tool description to avoid failures", which, Anthropic reports, cut task completion time for later agents by 40% (Anthropic, Jun 2025). Factory treats tool descriptions as one of its three prompt layers and limits tools "to essential operations" (Factory, Sep 2025)	Description rewrites applied without review. Trigger: none on any path that feeds a status

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Verification and self-correction	Allow one correction round, driven by code: when a check fails (a citation that does not resolve, a schema error, an action outside the catalogue), the run gets one turn with the typed failure, the check runs again, and a second failure gives `needs_review`. Today a schema mismatch is only logged (`agent/core.py:256-259`), and the verify node drops an item whose citation fails, with no chance to repair it (repository root `tests/unit/test_agent_core.py:33`)	Anthropic found that "letting the agent know when a tool is failing and letting it adapt works surprisingly well" (Anthropic, Jun 2025). In its long-running harness, agents marked features done after unit tests or curl calls missed end-to-end failures, until prompted to test end to end (Anthropic, Nov 2025). Claude Code's agent teams can block a task from completing with a `TaskCompleted` hook (Claude Code docs, undated)	Critique or debate between models, and judge-driven retries (6.4). Trigger: a drafting workflow where the critic meets the 85% expert-agreement bar (7.1) and one round shows a measured gain
Stopping conditions and loop detection	One stop function returns a typed reason: normal end, cap (6.4), three tool errors in a row (6.4), refusal or truncation, or a loop (the same tool and arguments three times, or the same result three times running). Any stop but a normal end gives `needs_review` with its reason. Today the loop only counts calls, a refusal or truncation is only logged, and the budget enforces its cap on a separate path (`agent/core.py:311`, `:331-338`; `llm/budget.py:88-90`)	Step repetition and unawareness of termination conditions are two of MAST's three commonest failures (row above). Pydantic AI flags the same call 5 times within 10, the same result 3 times in a row, or A-B-A-B alternation, and warns "A nudge doesn't stop a model that ignores it" (Pydantic AI docs, undated)	Nudging a looping run and letting it go on. Trigger: evals show nudged runs end correctly more often than stopped ones

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Error recovery and retries	One policy per error class. Transient errors (429, 529, timeouts) retry in code with backoff and never reach the model; tool errors return to the model as typed results and count toward the three-error stop; bad model output gets the one correction round, then `needs_review`. Today a node retries up to 3 times with jittered backoff (`agent/core.py:52-63`, `:154`), and since the whole tool loop is one node (`:311`), a retry restarts it and samples earlier turns again (inferred); record each model call as its own step (3.5)	Anthropic: "Agents are stateful and errors compound", so it built systems that "resume from where the agent was when the errors occurred" (Anthropic, Jun 2025). Claude Code's agent-team docs warn that "Teammates may stop after encountering errors instead of recovering" (Claude Code docs, undated)	Retrying a failed call on another model. Trigger: as for fallbacks in 6.6 (row 9, which model answers)
Testing the harness	Four test layers run in CI on the stub model. Structural tests (graph shape, allowlists, no write imports) exist (`agent/core.py:89-107`; repository root `tests/unit/test_layering.py:1-10`); add single-step tests on a fixed history, fuzzed tool results fed through the stub's chat handler (`llm/client.py:242-244`), and replay of recorded run-log trajectories against each candidate bundle. A fuzzed result (empty, truncated, malformed, oversized, an error, a planted instruction) must end in `needs_review` or a cited proposal, never a status	LangChain says "about half of our test cases looked like single step evals", stopped before the tools node; it records HTTP requests and replays them in tests, and gives each run "a fresh, clean environment" (LangChain, Dec 2025). Anthropic began its research evals with "about 20 queries representing real usage patterns" and grades end states rather than each turn (Anthropic, Jun 2025)	Fuzzing against the live model in CI. Trigger: fan-out ships; then a nightly fuzz run on the live model in the eval organization (3.7)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Harness metrics	Per graph and bundle, from the run log: stops by reason, tool-error and repeated-call rates per tool, correction rounds and how often they succeed, the share of runs that pass code checks first time, and tokens per accepted result (6.4). For fan-out, add items per parent run, failed items and merge rejections. A bundle that raises loop or cap stops does not ship (inferred)	On BrowseComp, Anthropic reports that "token usage by itself explains 80% of the variance", with tool calls and model choice the other two factors (Anthropic, Jun 2025). AWS's AgentCore Evaluations ship built-in "tool selection accuracy" and "tool parameter accuracy" evaluators (AWS, Mar 2026)	Per-tenant harness dashboards. Trigger: a second bundle runs in production beside the first, as in a canary (6.6, row 8)

How CCP's graphs should be built and tested

- **Shape.** One graph per role, at most one loop node, and every other node code, as in the Investigator's six nodes (`agent/graphs/investigator.py:291-312`). The stop function and the correction round become nodes or edges, so they show in the node path.
- **Declared paths.** Each graph declares its node paths, as the Investigator's `TRAJECTORY` does (`agent/graphs/investigator.py:41-45` ; repository root `tests/unit/test_agent_core.py:156`), and the run log flags any run whose path was not declared (inferred).
- **Fan-out outside the graph.** The launcher, the per-item runs and the merge live in the queue and the runners (row 2), so no graph holds another run's state.
- **Not checked.** OpenAI's harness-engineering post still returned HTTP 403 on 9 Nov 2026, so its figures are not used. There are still no public multi-agent results on compliance work (3.8).

6.9 Judges, feedback loops and training

Judges, reviewer feedback and training form one loop: experts label, judges stretch those labels across more runs, and the labels decide whether training ever pays. Section 5 fixed the graders and 6.6 fixed what judges see; this table settles how judges are calibrated, chosen and re-checked, how reviewer decisions become labelled data within consent, and when to train rather than prompt. In our reading the answer on training is "not yet", and the work that matters now is recording labels that CCP does not keep today.

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Judge verdict format	Keep one yes-or-no verdict per criterion, with an overall pass only when every criterion is met (`evals/judge.py:74-86` , `:156`). Mark each criterion as a blocker or not; a failed blocker scores the case zero. The reference that 5.1 requires arrives as facts (code-verified spans, the expert's key points), never as a model answer to copy (`evals/judge.py:1-3`)	Husain: "Binary evaluations force clearer thinking and more consistent labeling" (Husain, Sep 2026). Cognition grades each criterion as a blocker, where a fail scores zero, or a non-blocker (Cognition, Jun 2026). In Intercom's simulations, "if any single criterion fails, the Simulation fails" (Intercom, Aug 2026)	Scores from 1 to 5 or 0 to 1. Never for anything that gates; drafts are ranked pairwise instead (6.6)
Calibration protocol	Split each failure mode's 100 to 200 expert labels (5.1) into a dev half for prompt work and a held-out test half, which alone reports TPR and TNR with intervals. Read the result against the expert ceiling from the double-labelled 20% (5.3), and rewrite any criterion two experts cannot agree on before tuning a judge. Today calibration runs on one 20-case set whose intervals its own header calls wide (`evals/calibration.py:1-8` ; repository root `spec.md:2236`)	OpenAI's GDPval grader agreed with experts 65.7% of the time, against 70.8% between experts (GDPval, Oct 2025). In HealthBench, physician-physician and model-physician agreement both ran "from 55% to 75%" (HealthBench, May 2025). Husain asks for "at least 50 passing and 50 failing examples" when aligning a judge (Husain, Sep 2026)	Stretching scarce labels with judge scores, as prediction-powered inference does. Trigger: a quality rate is reported from judged production samples. Never for false-PASS bounds, which rest on expert labels (6.6)
Judge bias guards	Keep the order, length and paraphrase checks (`evals/judge.py:179-216`) as a gate on every judge, prompt or rubric change, and run pairwise draft comparisons in both orders (inferred). Add a self-preference guard in code: a judged case refuses to run when the judge model wrote the output. Today both default to `claude-haiku-4-5` , so tier-2 reasoning is graded by its author (`config.py:73-75` ; repository root `spec.md:1688`)	Evaluators' preference for their own text showed a "linear correlation" with their ability to recognize it (Panickssery et al., Apr 2024). Cognition tested its graders with deliberately bad "hack reports" and with calibration solutions spanning 0 to 100% quality (Cognition, Jun 2026)	Several judges voting (6.6) or a trained debiasing model. Trigger: a bias check still fails after two prompt fixes

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Judge model and family	Choose each judge per rubric by its test-split TPR and TNR, starting with the cheapest Claude model that differs from the producer: Sonnet 5.5 for Haiku 5.5 extractions, Haiku 5.5 or Opus 5.5 for Sonnet 5.5 agents (inferred). Judges on customer data run in the zero-retention organization. A judge from another family runs only on fixture or synthetic suites, to catch blind spots that Claude models share	Husain finds one model for task and judge "usually fine" (Husain, Sep 2026); CCP keeps its stricter rule because self-preference is measurable (row 3) (inferred). On 931 labelled agent trajectories, "the LLM backbone matters most", and two backbones erred in opposite directions, one conservative and one permissive (MobileJudgeBench, Aug 2026). OpenAI picked GPT-4.1 over reasoning models as HealthBench's grader for "physician-level agreement without the costs and latency" (HealthBench, May 2025)	A fine-tuned judge. Trigger: judge volume makes cost material and no Claude judge clears the bar; it then follows the last row
Judge drift and re-calibration	Version the judge like a bundle (model id, `judge_v1` prompt, rubric hashes); a change to it, a rubric or the judged model re-runs the test split and bias checks as a pre-migration gate (5.3). Move it off `claude-haiku-4-5` soon: it relies on temperature 0 (`llm/anthropic_client.py:73`; repository root `spec.md:2233`), which newer models reject, so self-consistency over three runs becomes a real measure, not "near 1.0 by construction" (repository root `spec.md:1714`). Each month an expert relabels 20 judged production items, and agreement outside the test interval triggers re-calibration (inferred)	Anthropic lists Haiku 4.5 as retiring "Not sooner than October 15, 2026", a date now passed, with at least 60 days' notice. API version 2026-10-21 removes `temperature`, and earlier versions return a 400 for a non-default value on Claude 4.7 and later (Anthropic deprecations, undated). Husain: when gold goes stale, "Use regular error analysis to find new problems and update your examples or reference answers" (Husain, Sep 2026)	Automatic judge-drift alarms. Trigger: the monthly audit misses a drift that error analysis later finds
Trajectory judges	Grade agent paths in code first: tools, arguments, tenant scope, budgets and no write before approval (5.2; repository root `spec.md:2273`). A model judge reads the run log's typed calls and results, never the reasoning (6.6), and answers narrow questions such as "was a required evidence type never searched?". It sorts failures for error analysis and gates nothing until calibrated like any judge	Agent-as-a-Judge, an agent that evaluates other agents' work, "dramatically outperforms LLM-as-a-Judge" and was "as reliable as our human evaluation baseline" on 55 tasks with 365 requirements (Zhuge et al., Oct 2024). Sierra runs a separate agent as "an independent judge" (Sierra, Aug 2025). NIST's CAISI designs and validates transcript scanners step by step to find agents cheating on evals (NIST, Feb 2026)	A tool-using agent judge on customer data. Trigger: a path question that code cannot answer and a tool-less judge misses the bar; it then gets read-only tools under the same tenant binding (5.5)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Sampling, cadence and cost	Judges score a nightly sample per workflow, stratified by verdict, confidence band, tenant and evidence type, with AI-derived PASS results and near-threshold items oversampled. People label first where code and judge disagree, where the judge answers "unknown", and where an expert finds a needs_review decidable; beside 5.3's weekly traces, a deeper pass of 100 or more fresh traces every two to four weeks may rename failure modes. Cost comes from the usage each judge call records (`evals/judge.py:161-167`): 200 items of about 4,000 tokens a night is roughly $50 to $100 a month per workflow at 6.5's Sonnet 5.5 price (inferred); only synthetic suites use batches	Husain: review "at least 100+ fresh traces each review cycle", with cycles of "2-4 weeks" and 10 to 20 traces weekly "focusing on outliers" (Husain, Sep 2026). Decagon stratifies judge samples by issue type and customer segment, and "Human labelers audit a subset" (Decagon, Jul 2025)	Judging every production run. Trigger: a calibrated detector for a failure mode that sampling misses, such as unsupported claims in sent questionnaire answers
Capturing reviewer signal	Today an approval stores an event, the reviewer and a free-text note (`storage/migrations/0011_remediation_approvals.sql:44-53`). Add an insert-only label row per AI-read field or draft: right or wrong, the corrected value and span, a reason from a short closed list (wrong value, period or entity; unsupported claim; missing evidence), the bundle, the seconds spent and a consent scope. A plain approval is stored but counts as nothing; only a wrong mark, an edit or a rejection with a reason becomes a candidate label	Cursor trains its Tab model on accepts and rejects, needing "1.5 to 2 hours to roll out a checkpoint and collect the data for the next step"; it reports 21% fewer suggestions and a 28% higher accept rate (Cursor, Sep 2025). There acceptance is the goal; in CCP it is not proof of correctness (6.6) (inferred)	Rewards or quality metrics built from acceptance. Never for status-deciding steps; drafts may track edit distance beside expert labels
Consent and data rights	Keep three uses apart in contracts and in code. A tenant's labels and captured cases replay only for that tenant; pooling them into shared gold or judge calibration needs a written opt-in, strips identifiers and follows the evidence's erasure (6.5, U10), and training needs a second opt-in. Customer evidence never enters the eval organization, and every suite build checks each label's consent scope (inferred)	Intercom trains Fin's custom retrieval and ranking models on resolved conversations, removes identifying data with Microsoft Presidio, and lets customers opt out, who may then miss the gains (Fin, undated). Decagon lets customers "opt out of experimentation" (Decagon, Jul 2025)	Never: an opt-out default like Fin's. Compliance evidence is more sensitive than support chats (inferred)
Failures into eval cases	Each production failure opens a candidate case: a wrong mark, an auditor rejection, a Copilot thumbs-down, or a needs_review an expert found decidable. `ccp evals capture` stores the observed output as the expectation (`evals/capture.py:5-6`); a failure case takes its expectation from an expert label instead, and joins gold only after review. Each failure pattern also gets a templated synthetic twin, so batches, other-family judges and prompt optimizers can use it in the eval organization	Intercom's Monitors check "every live conversation", and flagged ones become new simulations, a loop it calls "Eval-driven delivery" (Intercom, Aug 2026). Ramp turns user-flagged errors into test cases (Ramp, undated). Husain: "Add confirmed failures as you find them" (Husain, Sep 2026)	Model-generated twins at scale. Trigger: templates cannot express a pattern; then a generator in the eval organization, with an expert checking each twin

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Prompt optimization	Allowed for any prompt, offline in the eval organization on synthetic suites, once two rounds of manual error analysis stop helping. The optimizer sees only the dev split; its output is an ordinary prompt change, measured on real gold (5.3) and ablated line by line (6.6)	Across six tasks, GEPA, which rewrites prompts by reflecting on traces, beat reinforcement learning (GRPO) by 6% on average and up to 20% with up to 35 times fewer rollouts, and beat the MIPROv2 optimizer by over 10% (Agrawal et al., Feb 2026). Husain: "Inspect a few disagreements manually before trying automated prompt tuning", which can need "hundreds or thousands of evaluations" (Husain, Sep 2026)	Optimizers on customer data. Trigger: synthetic twins stop predicting production error rates for a prompt, and the tenants involved have opted in
Fine-tuning, reinforcement fine-tuning and distillation	Train and distil nothing for now, as 6.4 decided. If CCP trains, it uses LoRA (small adapter weights) on an open-weight model in its own AWS account, for one tool-less step whose output code still checks; the trained model is a new bundle with its own retirement date to track (inferred)	Bedrock's fine-tuning list holds one Claude model, Claude 3 Haiku (AWS docs, undated), which Anthropic retired from its API on 20 Apr 2026 (Anthropic deprecations, undated); Bedrock's reinforcement fine-tuning covers Nova 2 Lite, gpt-oss-20B and Qwen3 32B (AWS docs, undated). OpenAI says that from 6 Jan 2027 "Active existing customers will no longer be able to create new fine-tuning jobs", and fine-tuned models shut down with their base (OpenAI, May 2026). LoRA matched full fine-tuning for reinforcement learning "even with small ranks" (Thinking Machines, Sep 2025)	Reinforcement fine-tuning with code-computed rewards. Trigger: the last row's test passes for a step whose reward code can compute, such as the span and value checks of 5.4 (inferred)
Embedding and reranker training	Log training pairs now, at no model cost: each query and its candidates, positives from spans that passed 5.4 and a reviewer marked right (6.6), and hard negatives from spans marked wrong, each with its consent scope. Train nothing until 6.5's triggers fire; the first candidate is a reranker, since it sits behind today's search and can be recalibrated on the golden set (inferred)	Fin's first custom models target retrieval and ranking, not reply writing (Fin, undated). 6.5 records Intercom's distilled reranker and Cursor's embeddings labelled from agent traces	A trained embedder, which forces a full re-embed and a new index version per release (6.5). Trigger: a trained reranker still leaves a recall gap on a query slice

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
When to train at all	Train only when all of these hold: one narrow, tool-less step; label definitions unchanged for two quarters, with experts at their ceiling; volume where cost or latency matters; a gap left after prompting, model choice, retrieval fixes and prompt optimization; consented data and hosting inside zero retention; and an owner for retraining when the base model retires. A trained model re-earns its autonomy level from the start (4.4)	AWS advises starting reinforcement fine-tuning with "100-200 examples" and says it "might be unnecessary" once rewards pass 95%; its claim of accuracy gains "up to 66% on average" is its own (AWS docs, undated). Fin's fine-tuned 14B model and million-example classifier (6.4) rest on support-chat volume that CCP's steps do not have (inferred)	A training pipeline or model registry. Trigger: the first step passes every test in this row

Start now, because later choices rest on it.

- **The label row and its consent scope** (rows 8 and 9). Every later judge, eval case and training decision depends on them, and labels cannot be collected after the fact.
- **A judge that differs from its author, off Haiku 4.5** (rows 3 to 5). It is the cheapest fix on this list, and it removes a known bias before more cases are judged.

6.10 Human-agent interaction and computer use

Sections 3.6, 4.4, 5.5 and 6.6 set who approves what and how reviews are checked; this subsection covers the screens around them (6.1, row 11) and computer-use agents. Decision 6 already bars agents from changing customer systems through a browser, so the last three rows cover only reading evidence from consoles that have no API. Across frontier products the pattern is the same: show records rather than narration, let people stop a run at any time, be honest about what undo cannot undo, and keep models away from browsers and passwords while an API, an export or an upload will do.

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Showing a plan before an agent acts	Show a plan only when a person starts a long run and waits for it, such as a Copilot investigation or audit prep: the questions, the read tools and the budget, which the person can edit or cancel. The plan is advice, not permission; the registry still enforces tools and budgets (inferred)	In Claude Code's plan mode, Claude "reads files and proposes a plan but makes no edits until you approve" (Claude Code docs). When Claude Code stops itself, the top reason is to offer a choice between approaches, 35% of stops (Anthropic, Feb 2026, self-reported)	Plans for scheduled verdict runs, whose path is the fixed graph (3.1). Trigger: a graph that chooses its own steps
Progress and streaming	Progress comes from the run log, not the model's narration: each recorded step (tool called, evidence read, check passed) reaches the page as it lands, over server-sent events (inferred). Stream prose for drafts and Copilot replies, never a status, and show a citation as a link only once code has checked it: the Copilot already holds back status answers (`agent/graphs/copilot.py:314-315`), and a stream that has started cannot be retried (`llm/anthropic_client.py:168-170`)	Anthropic asks product teams for "trustworthy visibility into what agents are doing" (Anthropic, Feb 2026). Claude "sometimes acts as if its actions succeeded without explicitly checking their results" (Claude docs), so narration is not proof	WebSockets or a separate streaming service. Trigger: two people working live in one space, such as the auditor workspace (LT4)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Interrupt and steer	Every run has a Stop button. Stop sets a flag that the runner checks at the next node boundary, where the run budget already stops runs (`agent/core.py:151-153`); the run ends cancelled, keeps its record and changes no status. Steering a status-deciding run means restarting it with the person's note stored as an input, so replay sees it	Experienced Claude Code users interrupt in about 9% of turns, new users in 5% (Anthropic, Feb 2026, self-reported). A message typed while Claude Code works can have no checkpoint of its own (Claude Code docs). An OpenClaw agent ignored stop commands sent from a phone (OWASP, Apr 2026), so Stop must not depend on the agent	Steering inside a running investigation. Trigger: restarts waste more than a few minutes of work per run (inferred)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
The approvals inbox	One inbox for CCP's four item kinds (`approvals/store.py:16`), each showing the code-written payload, its hash, its expiry and whether it can be undone. The choices are approve, reject, or change and resubmit, which makes a new proposal with a new hash for someone else to approve. No decision, or an expired one, means no action; today a tier-3 resume without an action approves, as reviewer "unknown" (`agent/graphs/tier3.py:231-232`)	LangChain's middleware lets a reviewer approve, edit, reject or respond, and warns that "Significant modifications to the original arguments may cause the model to re-evaluate its approach" (LangChain docs). Intercom Fin's human-in-the-loop beta sends Fin's draft "as written, without review" when no teammate answers in time, after 24 hours by default (Intercom Help, Nov 2026); CCP rejects that default	Approving from email, Slack or a phone; until then they carry a link, never an approve button or evidence text. Trigger: passkey-signed approvals (Decision 5). Auth0's agent approvals show the authorization details before a token issues (Auth0, Nov 2025)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Explanations and citations in the UI	Code writes the "why" of every status from the rule version and the facts it used, each fact linked to its checked pointer citation and shown as a highlighted span on the page image or JSON field (6.5). Model prose sits under a "summary" label and is never the reason. A citation that fails its check shows as unverified text, not a link	Ramp says "Every decision needs a 'why'", and its policy agent links to the policy sections its reasoning uses (Ramp). In a contest of 272,000 attacks, successful ones often left "no clue of compromise in the final user facing response" (IPI Arena, Mar 2026), so a fluent summary proves nothing	Model-written plain-English rule text. Trigger: auditors ask for it; then write it once per rule version and review it like code

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Showing uncertainty and abstention	Needs review and not assessed always show a reason code and the one action that clears it, such as reconnecting AWS or uploading the policy for the period, styled as normal states, not errors. Show the factors behind computed confidence (coverage, citation rate), never a bare score; today a threshold on that score decides which drafts a person sees (`agent/graphs/tier3.py:214`)	Ramp let its models say "I'm not sure", sends those cases to people, says it matters that "unsure doesn't look like an error state", and replaced numeric confidence with Approve, Reject and Needs review (Ramp). Fin says when it found no answer, and when unsure may "express uncertainty" and ask for clarification (Intercom Help, Nov 2026)	Calibrated probabilities on screen. Trigger: per-test calibration on gold sets of the size 6.6 sets

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Undo and reversal	Code labels each proposal by how it can be undone: one click for internal changes (assignments, snoozes, drafts), close or revert for pull requests, and for external sends (trust page, questionnaire) only a retraction, which is a new approved action. External sends wait ten minutes with a cancel button before they leave CCP (inferred). Undo is never a model action	Claude Code's rewind restores only its own file edits; a file that only shell commands changed "stays as it is", and checkpoints are "Not a replacement for version control" (Claude Code docs). OWASP recommends hard confirmation, an emergency stop and easy rollback for delete, send, pay and publish actions (OWASP, Apr 2026)	Undo in a customer's cloud. Trigger: the runbook role (Decision 6, LT10), with runbooks that check their own result
Copilot conversation scope and refusals	The Copilot answers only about this tenant's controls, evidence, verdicts and findings, from read tools, and says plainly when it found nothing. It gives no audit opinion, legal advice or status of its own; it points to the rollup with its as-of date or to a named owner, and logs its own refusals apart from the model's, which U5 counts. What it suggests becomes a proposal in the inbox	Fin answers "based solely on your support content", says when it found no answer, and hands sensitive topics such as refunds to people through escalation rules (Intercom Help, Nov 2026)	A Copilot for buyers on the trust center. Trigger: buyers ask for one (LT4); it would answer only from published, approved content

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Disclosure that content is AI-made	Keep the provenance badge (6.1, row 11) on every AI-read field and draft. Anything that leaves CCP names who approved it and when, and the export and audit trail keep the AI-drafted flag; 6.6 already sends the AI Act Article 50 question to counsel	After a teammate's input, Fin's reply names the reviewing teammate in web chat and in the email footer (Intercom Help, Nov 2026)	Machine-readable marking of AI text. Trigger: counsel finds Article 50(2) reaches CCP's drafts (6.6)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Reviewer ergonomics and trust calibration	Facts and source spans come first; the agent's reasoning and tool log sit behind a toggle, since people, like judges, can be swayed by a story (inferred from 6.6). New tenants and workflows get per-item review. A workflow that graduates under 4.4 swaps prompts for a daily digest of what ran, with one-click reopen, and keeps its seeded checks	As Claude Code users gain experience, full auto-approve rises from about 20% to over 40% of sessions while interrupts also rise (self-reported); Anthropic warns that rules that "prescribe specific interaction patterns" can "create friction without necessarily producing safety benefits" (Anthropic, Feb 2026). Fin's teammate view shows the draft and its confidence, with reasoning behind a "Show reasoning" option (Intercom Help, Nov 2026)	A minimum time per item, since time spent did not predict accuracy in the study 6.6 cites. Trigger: seeded catch rates fall

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Computer-use agents for evidence collection	None in v1 or v2. Evidence comes from APIs and exports, then from people's uploads, and no model drives a browser or desktop with a customer's sign-in. A console with no API gets the next two rows, not an agent	GUI computer use is the weakest surface in Anthropic's tests: the chance an attacker succeeds within 15 attempts was 12.5% for Sonnet 5.5 there against 1.0% in tool use, and 2.8% against 0.4% for Opus 5.5 (Anthropic, Sep 2026, Sep 2026, self-reported). Claude's docs advise a dedicated VM, no login details and a domain allowlist, and warn that instructions "contained in images might override your instructions" (Claude docs); Google's say its tool "may contain errors and security vulnerabilities" (Google docs). Vanta's browser evidence agent doubled its completion rate once a verifier checked its	A computer-use agent in a sandbox with a read-only account. Trigger: three customers need the same API-less system, uploads keep failing the verifier, and the attack suite (6.6) covers screenshots. Claude's tool is zero-retention eligible, except Covered Models (Claude docs); OSWorld scores, verified under one fixed setup (OSWorld), do not show that a console was read completely (inferred)

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
		work, from a baseline Vanta does not give (Vanta, Jun 2026, self-reported)	
Browser automation without a model	For a console with no API or export, CCP writes a reviewed Playwright script per page and runs it with no model: a disposable browser limited to that console's domain, signed in with a customer-created read-only account held by the credential broker (5.5). A changed page ends the run as not assessed instead of calling a model to repair it. A model may draft scripts offline for an engineer to review	Stagehand records actions on a first run and replays them "with no LLM calls", but its hosted cache calls the model again when a page changes (Stagehand docs); CCP turns that repair off. 1Password injects credentials at run time, only after a person approves, so "the AI agent and underlying LLM never need to see nor handle" them (1Password, Oct 2025)	A collector that customers run, or credentials drawn from their own vault. Trigger: a customer will not give CCP even a read-only console account

Topic	Decision for CCP	Frontier precedent	Not now, and the trigger
Screenshots as evidence	A screenshot is raw evidence: stored write-once with its hash, transcribed by Claude in the zero-retention organization (6.5, LT2), and counted only after code checks its date and subject. An upload is badged customer-attested; only a capture by CCP's own script, stored with its URL, account, capture time and page HTML, is system-observed (inferred). One screenshot shows one moment, so it never covers a Type II period alone	Oneleet's evidence guide asks for when a screenshot was taken, says to "Add URLs, report names, system IDs", and wants Type II evidence to cover the whole period (Oneleet docs)	Recorded sessions as evidence. Trigger: an auditor asks to watch a collection run; then keep the script's trace beside the capture (inferred)

Where this lands in the roadmap (Section 8):

- **With MT4 and UI v1:** the inbox rules above (a missing or expired decision never acts; change and resubmit; links, not approve buttons, in messages), run pages fed from the run log with a Stop button, reason codes with a next action, and code-written explanations over checked spans.
- **With LT2:** screenshots as raw, transcribed, badged evidence, with uploads as the only path for consoles that have no API.
- **Off the roadmap until a trigger fires:** computer-use agents and model-free browser scripts. Neither needs work now beyond keeping the credential broker (5.5) able to hold a read-only console account.

6.11 Coverage map: every agentic concern and where it is decided

This is the checklist for the agent side of CCP: each concern an agentic system has to settle, the decision in one line, and where the report makes it. Review it whenever a new workflow, model or tool arrives; a concern with no row is a gap.

Concern	Decision in one line	Where
Workflow or agent	Code fixes the path; an agent loops only where the path is unknown, and no agent sets a status	3.1, 4.2, 4.3, 6.4

Concern	Decision in one line	Where
Multi-agent work	One writer; fan out only independent reading items, each a queued run with a schema that code checks and merges; no agent-to-agent channel and no A2A for now	3.2, 6.4, 6.8
Harness engineering	Prompts, tool descriptions, graph, budgets and effort ship as versioned bundles behind per-model evals; one stop function, one code-driven correction round, and harness tests on every bundle	6.4, 6.6, 6.8
Context engineering	Each prompt is rebuilt from the run log within a budget for its call type: stable content first for caching, evidence by handle, old tool results swapped for handles	3.4, 6.4, 6.7
Reasoning settings	Effort and thinking settings are pinned per step in the bundle and changed only through the eval gates	6.4, 6.6, 6.7
Long-running work	Long tasks split into short, resumable runs linked by typed state and progress records; human waits are rows; drift, loops and early "done" end in needs_review	3.5, 6.4, 6.7, 6.8
Long context	Used only to read one document end to end, quotes first; questions across documents go through retrieval	6.5, 6.7
Memory	Per-tenant notes confirmed by a person, never counted as evidence, with poisoning tests before any write path	3.4, 6.4
Tools and MCP	One typed registry behind a default-deny policy check; a small, fixed tool set per graph; a read-only MCP server as an output	3.3, 6.4, 6.6, Decision 7
Retrieval	Layered, per-tenant hybrid search with exact-match, outline and read-span tools for agents, evaluated on its own	6.5, Decision 10

Concern	Decision in one line	Where
Structured output and citations	Typed fields with pointer citations that code checks before any rule uses them	5.4, 6.5
Models and routing	Haiku 5.5 for extraction, Sonnet 5.5 for agents, Opus 5.5 where measured; no model switch inside a run; no safety fallbacks on tool-using graphs	3.7, 6.4, 6.6, Decision 9
Verification before action	Code gates and a dry-run re-check of the rule; a clean-context model reviewer is a signal only	5.4, 6.4, 6.8
Evals and release gates	Four gates plus production sampling and a daily canary; gold sets weighted towards FAIL cases and sized to the false-PASS bound	5.1 to 5.3, 6.6
Judges	One yes-or-no verdict per criterion, calibrated on a held-out expert split, never the model that wrote the output, re-checked monthly	5.1, 6.6, 6.9
Feedback and training	Insert-only review labels with a consent scope start now; prompt optimization only offline on synthetic suites; no fine-tuning until every test in 6.9′s last row passes	6.9
Prompt injection	Evidence never enters the user turn; an attack suite built from CCP′s own evidence types; policy enforced outside the model	5.5, 6.6
Authorization and agent identity	A default-deny policy check, one principal per graph type, and credentials only in the executor	5.5, 6.6
Human oversight	Payload-bound, single-use approvals; reviews designed to catch errors; autonomy earned per workflow and demoted on any bundle change	3.6, 4.4, 6.6, Decision 5
Human-agent interaction	Progress from the run log, a Stop that does not rely on the agent, code-written explanations, and every abstention with a reason and a next step	6.10

Concern	Decision in one line	Where
Computer use	No computer-use agents in v1 or v2: APIs, exports and uploads first; a reviewed, model-free browser script only when its trigger fires	6.10, Decision 6
Durable execution	CCP's hardened Postgres queue with idempotent steps and each model call recorded as its own step; DBOS only on a trigger	3.5, 6.8, Decision 4
Observability	The run log is the record; traces carry ids and hashes, not evidence; engineers read content only through logged replay	3.7, 6.6
Cost and latency	Cache-hit alerts, cost per accepted result, and rate-limit headroom kept for interactive work	6.3, 6.6
Model lifecycle	Each new release compared with the incumbent within a month; migrations finish within six weeks; the Bedrock path tested each quarter	3.7, 6.6
Tenancy and data	Forced row-level security, per-tenant retrieval, memory and labels, and zero-retention model calls	6.3, 6.5, 6.9, Decision 8
Standards and governance	A quarterly map to the OWASP Agentic Top 10 and AIUC-1; AI Act questions go to counsel	6.6

7. Competitor-inspired improvements

The strongest ideas share one theme: keep the decision in code and make every AI contribution checkable. The approach and evidence columns are verified from the linked source; "Why it matters" is our reading; the last column is our recommendation, starting with adopt, adapt or reject.

7.1 From compliance platforms

Company	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Vanta	Every product action is one authorized service method, exposed to REST, CLI, the agent and MCP	Vanta, Aug 2026	One permission check and one schema per action, so an agent cannot do what its user cannot	**Adopt.** CCP's tool registry becomes this capability layer; REST, agent tools and MCP are generated from it (3.3)
Vanta	An evidence verifier inside the agent loop doubled the browser agent's task completion	Vanta, Jun 2026, Aug 2026	Agents improve most from a checker's feedback, not longer prompts	**Adopt.** Code verifiers first (citations, dates, scope), then a calibrated model verifier; agents call them before proposing
Vanta	Tools shaped around information, with explicit truncation and a recovery path: about 27% fewer tokens and over 45% higher task completion	Vanta, Sep 2026	A partial result shown as complete is a silent false-pass risk	**Adopt** in the tool registry: truncation is always explicit, and missing data is an error
Vanta	Auto Mode pre-approves eligible actions but never destructive ones; the event log records which agent acted, for whom	product updates	Cuts approval fatigue without giving up control of irreversible actions	**Adopt** as the approval tiers in 4.4, with an agent actor type in the audit log

Company	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Vanta	Published eval results: mapping precision from below 35% to 78%; judge agreement with experts from about 65-70% to 85-90%	Vanta, Jun 2026	The only published calibration bar in the category	**Adopt** 85% agreement with experts as the minimum before a judge gates anything
Drata	An offline LLM pipeline wrote 1,000+ deterministic tests, validated against Drata's test API and reviewed by auditors	Drata, Dec 2025	AI speeds up authoring while runtime verdicts stay deterministic	**Adopt** for connector checks: AI drafts offline; the conformance harness and a reviewer gate each check
Drata	The vendor agent asks before every write and withholds a recommendation when too many criteria are inconclusive; the mapping agent proposes with confidence	TPRM Agent, autonomy, mapping	Abstention and proposals are built into the product, not left to prompts	**Adopt** the withholding rule ("any step may abstain") and propose-then-map; **adapt** autonomy presets only once 4.4's metrics exist
Sprinto	Questionnaires run as verbatim fact extraction, answer direction, then writing, with a grounding check	Aug 2026	Separating facts from prose is extract-then-rule for documents	**Adapt.** Keep the passes; replace the model grounding check with CCP's code citation and support checks

Company	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Secureframe	Changes to built-in tests ship as versioned updates after a countdown	test updates	Customers see a rule change before it alters their status	**Adopt.** Rule versions with a notice period and a preview of the verdicts they would change
Scrut	A validation agent reproduces each finding before publishing it; every REST write needs an `Idempotency-Key`	Sep 2026, API	An AI assertion becomes a fact only after it is shown again; retries cannot double-apply	**Adopt** both: code re-checks an agent's finding before it surfaces, and every write carries an idempotency key
Thoropass	Evidence frozen per audit; LLMs draft first-pass test results for its auditors	help, blog	Auditors will increasingly consume structured evidence through AI	**Adapt.** Export hashed, as-of audit packages from bitemporal state, with each result's method

7.2 From open-source products

Project	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Comp AI	Long-tail checks as a versioned JSON DSL in the database, snapshotted before every edit and hot-reloaded	schema	Breadth without a deploy per check	**Adapt.** Declarative checks as versioned data files in the repository, shipped in the release bundle behind the same review and eval gates, not rows edited at runtime; without its unsandboxed code step; with explicit error results instead of empty ones
Comp AI	Findings reconcile only from complete, successful scans; unscanned scope stays unknown	reconciliation	Prevents a false "resolved" after a partial sync	**Adopt** as CCP's sync completeness contract
Probo	Durable runs in Postgres: leased claims, a checkpoint per tool turn, and approval decisions that must cover exactly the pending tool calls	service	A tested human-approval design on the database CCP already uses	**Adopt** the semantics, and make approval mandatory per tool class (Probo enables it in no production agent)

Project	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Probo	Field-level provenance for AI-written data (source, confidence, status); human edits are never overwritten	enrichment	Keeps AI-written values apart from curated ones	**Adapt.** Add a code check that the source supports the value before it counts as found
CISO Assistant	A read-only MCP profile that does not register write tools; an action ledger; an eval of tool selection against tool-surface size	MCP server	The boundary lives in code, not in annotations	**Adopt** all three

7.3 From AI-native entrants and adjacent agent companies

Company	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Harvey	A deterministic policy engine between agents and tools: permit, deny or ask; fails closed; tenant settings only tighten; tool definitions pinned	Oct 2026	Turns CCP's per-graph allowlists and approval stage into one policy point	**Adopt** in the tool gateway: a small Python policy module first, a policy language only if the rules multiply
Harvey	Citations point to sentence indices and are rendered from the source	Apr 2026	Fuzzy-matched quotes are exactly where CCP's checker fails	**Adopt:** cite chunk and sentence ids, render quotes from the source, check support in code (5.4)

Company	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Harvey	A benchmark of atomic, binary criteria in which a task passes only if every criterion passes; the harness is MIT-licensed	May 2026	Mirrors how auditors test a control: every attribute must hold	**Adopt** for CCP's golden sets
Sierra	Immutable release bundles of code, prompts, model versions and knowledge, with rollback; production conversations become regression tests	Sierra	A verdict can name exactly what produced it	**Adopt:** each verdict records its bundle (model, prompt, schema, rule and index versions)
Sierra	Supervisors that intercept only on high-risk topics; calibrated monitors on everything else	Sierra	Blocking checks cost latency, so they belong where output is irreversible	**Adopt** the two tiers: block on customer-visible and external writes; monitor the rest
Ramp	Approve, reject or needs review instead of model confidence; an exposure budget per agent	Ramp	Users' acceptance is a lenient ground truth, and caps bound the damage of a bad release	**Adopt** per-tenant caps on automated status changes; keep computed confidence internal
Anterior	A risk-ordered human review queue and continuous judge-versus-human agreement on live cases	talk	Offline calibration drifts once real traffic arrives	**Adopt** for production monitoring (Section 5)

Company	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Anecdotes	Models draft deterministic analysis rules; the rule re-runs after remediation	Agent Studio	Coverage grows without model verdicts	**Adopt** together with Drata's offline generation (7.1)
Decagon	Natural-language procedures backed by code, with tests generated and retired automatically	Decagon	Lets compliance staff author procedures, but auto-updated tests can hide regressions	**Adapt:** authored procedures are welcome; any change to a test needs human approval
Trustero	A repeatability metric published without a method	Trustero (vendor claim)	Stability is visible to auditors and buyers	**Adapt:** publish pass^k with its dataset and method
AIUC-1	Agent certification with 1,000 to 5,000 red-team scenarios and quarterly retests	AIUC-1	An emerging buyer requirement for AI agents	**Adapt** as a checklist for CCP's own agents; decide on certification once buyers ask

7.4 From agent engineering teams

Source	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Anthropic	pass^k for consistency; grade outcomes; start from 20-50 real failures	Jan 2026	A verdict must repeat on the same evidence	**Adopt** as the gate metric (Section 5)

Source	Interesting approach	Evidence	Why it matters	What we should adopt or adapt
Anthropic	A stateless harness with an append-only session log; credentials fetched from a vault through a proxy	Apr 2026	Recovery and the audit trail become one record; the model never holds secrets	**Adopt:** the run log is the record (3.4) and a credential broker serves the tools (3.3)
Anthropic	Tool search and usage examples raised accuracy from 49% to 74% and from 72% to 90%	Nov 2025	CCP runs on Claude, so these figures apply more directly than to other models	**Adopt** when a workflow's tool count grows; measure on CCP's tasks first
Anthropic	A separate, sceptical evaluator agent beat self-grading, at about 20 times the cost	Mar 2026	Generators rate their own work too highly	**Adapt:** use it only on high-value outputs, after code checks
OpenAI	Rate each tool's risk; approval does not guarantee exactly-once execution; a run id is not authorization	guide, SDK	Approval security and idempotency are the application's job	**Adopt** in the approval stage (3.5)
Cognition	Multi-agent designs work when "writes stay single-threaded"	Apr 2026	Parallel writers make conflicting decisions	**Adopt** as "graphs read, runners write"
Meta, Simon Willison	An agent may hold at most two of: untrusted input, sensitive data, and the ability to act externally	Meta, Oct 2025, Willison, Jun 2025	A structural rule that does not depend on detecting injections	**Adopt** as a static check on every graph's tool allowlist (5.5)

7.5 What we reject, and why

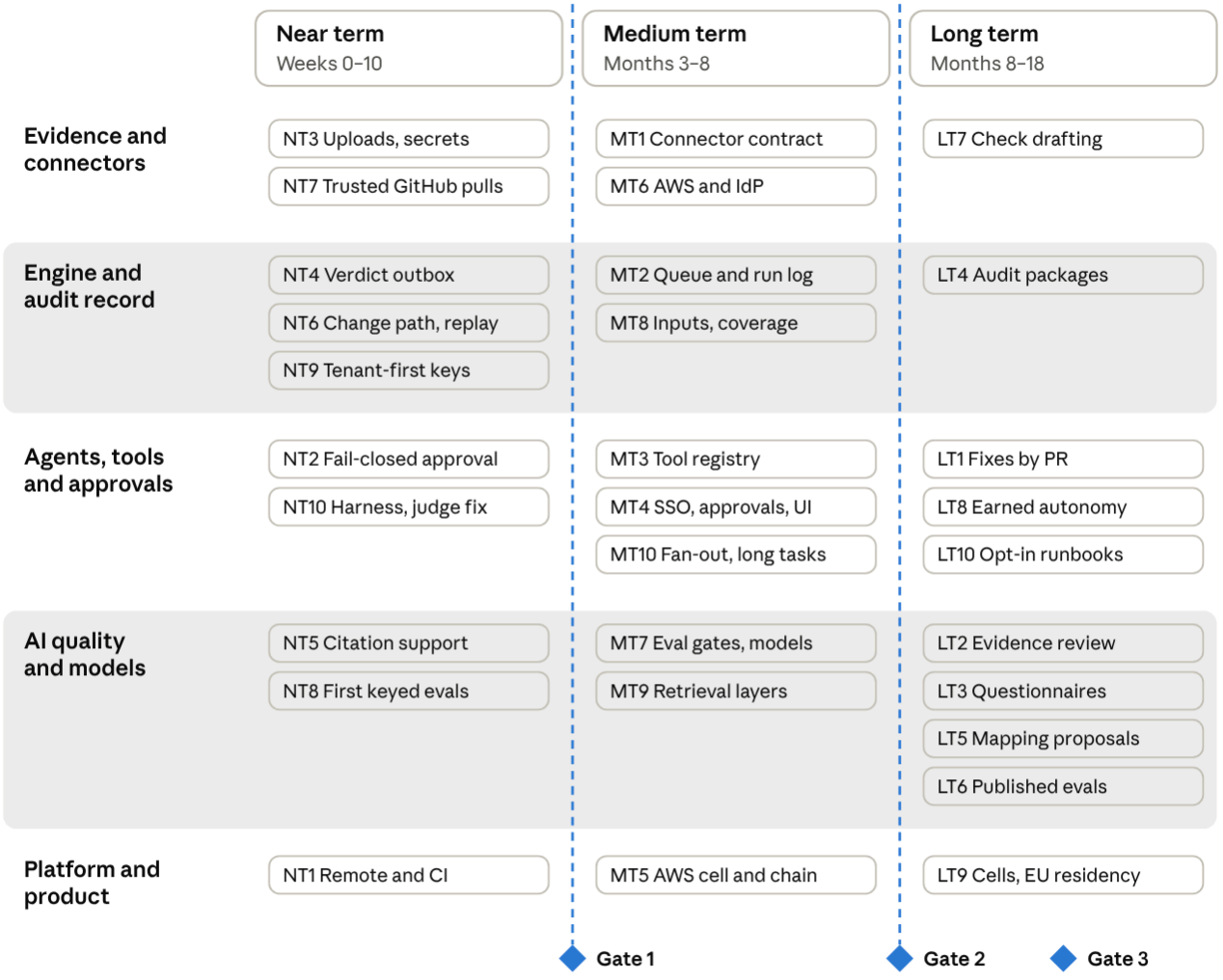
- **Model-chosen commands against customer infrastructure** (Comp AI's denylisted AWS executor) and **console changes through a browser extension** (Sprinto). In both, what a person approved and what ran can differ.
- **Full-write MCP with no read-only mode** (Secureframe, Comp AI). Writes arrive only as approval requests.
- **AI deciding whether a customer sees a failure** (Comp AI's self-heal hold). Store the raw outcome and the AI's triage as separate facts, and show needs_review.
- **Tenant ids chosen by the model** (Probo's MCP tools). The tenant comes from the session.
- **Grounding by prompt alone** (Comp AI's questionnaires) and **input classifiers that fail open** (Probo). Use structural defences and code checks.
- **Parallel writer agents, and a general agent with hundreds of tools as the first agent.** Vanta needed domain tool loading once its agent passed 280 tools (2.1). Start with narrow workflows backed by verifiers.

8. Prioritized engineering roadmap

Fix correctness before breadth, change one architectural thing at a time, and land every item with a test that reproduces its defect or a metric that shows its effect. Ids NT, MT and LT belong to this report; the "Matches" column gives the ids in the first report's roadmap (its Section F). Where a near-term row lists Q ids, its scope is the first report's scope, so the two documents agree. Sizes: S is up to about one engineer-week, M two to six, L more than six.

Fix and measure first, build the production cell, then earn the differentiators

Workstreams by horizon; horizons are not to scale. Ids match the tables below.



Gate 1, week 10: P0 fixes merged, CI green, false-PASS rate measured

Gate 2, month 8: complete syncs, signed-in approvals, release gates, audit verify green, retrieval targets met

Gate 3, about month 12: a measured false-PASS bound before any autonomy or runbook path

Efforts are estimates: near term about 21–30 engineer-weeks, medium about 54–76, long about 39–54.

Roadmap · 5 workstreams across 3 horizons, 3 gates

8.1 Near term: weeks 0 to 10, inside the current architecture

Order, following the first report: NT1 on day 1; NT2, NT3′s upload fix and NT4′s outbox fix in weeks 1 and 2; NT5 and the rest of NT4 in weeks 2 to 6, with a first NT8 baseline run in week 2 to measure NT5′s effect; NT9 in weeks 3 to 6; NT10 in weeks 3 to 8, its judge fix before the nightly NT8 runs; NT3′s secrets work and NT6 in weeks 5 to 8; NT7 and the rest of NT8 in weeks 6 to 10.

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT1	Q1	The repository has no remote, so the CI that exists has never run (`git remote -v` is empty at `3386899`); `spec.md:3` says "No implementation code exists yet"	Push to a private remote, tag `3386899` , turn on `.github/workflows/ci.yml` and `nightly-evals.yml` , fix `spec.md:3`	Every later change is tested	The workflows already exist	S	None	Keyed nightly runs need keys in a separate eval organization (NT8)	CI green on main; every NT item lands with a test that reproduces its defect
NT2	Q2	A remediation item is approved whatever `--decision` says (`approvals/cli.py:88-90` , `approvals/actions.py:55-57`); a tier-3 decision is recorded only after the resumed graph and verdict commit succeed (`approvals/actions.py:51-53`); nothing re-checks the payload hash on resume; rollback re-grants access without the gates (`remediation/runner.py:243-289`)	Accept only known decisions; record the decision before resuming; check the payload hash on every resume; run rollback through the five gates; treat a tier-3 resume with no decision as no decision, never an approval (`agent/graphs/tier3.py:231-232` , 6.10)	A typo cannot approve; only the approved payload runs; a model outage cannot lose a reviewer's decision	The fix sits in the data layer, so it holds under any engine or UI; signed-in reviewers follow in MT4	M	NT1; an additive migration (new event values)	Scripts that pass free-form decisions break, which is intended	No decision string other than `approve` approves (property test); the gate test covers rollback; a nightly query finds zero executions without a matching approved hash

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT3	Q3, Q9	A manifest entry is read from any path (`connectors/manual/uploads.py:88`); chunk text goes to the embedding and rerank APIs unredacted (`retrieval/index.py:139-151`, `retrieval/rerank.py:59-65`); the patterns miss AWS secret keys (`security/secrets_patterns.py:11-20`); GH-5 passes without saying whether it read every file (`evaluators/deterministic/gh5.py:51-57`); Voyage may train on self-serve data unless the organization opts out (MongoDB)	Refuse paths that resolve outside the tenant's folder; redact before embedding and reranking; add secret formats; GH-5 returns `needs_review` when it could not read every file; opt out of Voyage training and settle retention terms with each model provider	No upload is read into another tenant; customer secrets stay out of third-party APIs and their training data	Containment at the boundary is cheaper and more certain than detection later	S to M	NT1	Over-redaction can lower retrieval quality; NT8 measures it	Traversal cases (`../`, absolute paths, links) refused; zero secret-pattern hits in a canary of payloads sent to Voyage; the opt-out recorded

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT4	Q4, Q6, Q7	A verdict and its announcement commit in two transactions (`evaluators/runner.py:93-106`); one bad record stalls an ordered topic (`ingest/consumers.py:61-72`); retries have no backoff (`ingest/event_log.py:110-115`); tier 3 starts a new thread on every attempt (`agent/tier3.py:157-158`); the model budget is checked before the tier is known (`evaluators/runner.py:153`); unknown models cost $0 (`observability/cost.py:43-50`)	One transaction for verdict and announcement, and a check for a stored verdict before any model call; dead-letter with replay on ordered topics; backoff; tier 3 resumes its thread; the budget checked after routing, so tier 1 never parks; an unpark command; refuse unpriced models	No lost verdicts; one bad record no longer stops a tenant; deterministic results keep flowing; dollar caps trip	Each fix is needed under any engine (Section 3.5, first report D4)	M	NT1	Tests that assumed ordered Investigator behaviour change	A kill between verdict and announcement strands nothing (reconciler count 0); a simulated burst of 429s makes no dead letters; tier-1 verdicts continue with the model budget at zero

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT5	Q5, Q8	Citation checks accept an 85% word overlap (`evaluators/citations.py:36-47`) and skip the value check for booleans and "absent" values (`evaluators/citations.py:29` , `:50-52`), so an "In progress" record passed as completed (Section 1.3); tier 3's retrieval factor is fixed at 1.0 (`agent/graphs/tier3.py:189`); tool results reach the model unfenced (`llm/client.py:126-140`)	Contiguous quotes only; a value check on every deciding field, status included; measured retrieval quality; the same checker in the Copilot; zero false accepts on labelled quote cases; tool results fenced and scanned in all four agent loops, a hit lowering confidence or routing to review	Removes a verified class of false PASS; planted text cannot steer an agent unseen	Code checks are exact and cheap; model judges caught only 30% to 60% of inconsistent summaries (Section 5.1)	M	NT1; NT8 measures before and after	More `needs_review` at first, reported as coverage	Zero false accepts on labelled cases (splices, swapped dates, negations); attack success per class not higher with tool-result injection cases added
NT6	Q10, Q11	Identity resolution and index refresh run only in the full run (`compliance_state/run.py:66-67`); nothing checks the impact table's reach (`evaluators/impact.py:149-209`); verdicts carry no rule version (`storage/migrations/0005_state.sql:6-33`); replay reads chunks from the live index (`replay/rebuild.py:168-173`)	Resolve identities and refresh the index on the change path; CI fails a test with no trigger; store the rule version on every verdict; replay from the fenced text stored with each request	GH-1 and HR-1 cannot pass for an account first seen after the last full run; every verdict names its rule and survives a document change	Answers "by what method?" from stored data	M	NT4	Index refresh on the change path spends embedding calls	Every test reachable; 100% of new verdicts carry a rule version; replay passes after a cited document changes

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT7	Q12	Any non-404 error aborts the pull and a 404 reads as "nothing there" (`connectors/github/client.py:55-60`); a failed email lookup reads as no emails (`:75-85`); snapshot ids embed the pull time (`:129`, `:179`, `:189`, `:222`); live mode never sees a revocation (`normalize/github_mapper.py:303`, `:321`)	Retry with backoff within rate limits; raise a failed lookup as an error; content-derived snapshot ids; mark complete pulls, so a member missing from one becomes a revocation written as a new observation	Live results match GitHub; the remediation loop closes without the test-only overlay	Makes absence a proven fact, as Comp AI's rule of reconciling only complete scans does (Comp AI)	M	NT4 (Q6)	A wrongly inferred absence; inferred only from complete pulls, under property tests	A real collaborator removal becomes `revoked` within one sync; an unchanged re-pull lands zero rows; a 404 on a listing marks the pull incomplete
NT8	Q13	No eval has run on a real model, and the single-flip gate fires on noise (Section 5.2); models are aliases (`config.py:73-75`); `temperature` goes through `extra_body` (`llm/anthropic_client.py:62-73`), which API version 2026-10-21 rejects with a 400 (Anthropic), and `pyproject.toml:17` allows the SDK 2.x that sends that version; the judge is the tier-2 model (`config.py:73`, `:75`)	Pin dated model ids; remove `temperature`; a different-family judge on the fixture suites; real models, five runs per case, before and after NT5; gates on confidence bounds; grow the 20-case labelled set	The first measured false-PASS rate and verdict stability; an SDK upgrade stops being an outage	The harness exists; hosted eval products are churning (6.1, row 8)	M, plus labelling	NT1; NT5 for the second run	Wide intervals at small n; the nightly cap defaults to $10 (`spec.md:2212`)	A baseline per AI test: false PASS and false FAIL with intervals, pass^5, refusal rate, cost per task

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT9	None (addition)	Single-column keys let a state row reference another tenant's verdict (`storage/migrations/0005_state.sql:51`); both caches are keyed without the tenant (`storage/migrations/0008_chunks.sql:36` , `:44`); the checkpoint saver sets the tenant through startup options (`storage/checkpoints.py:64-68`); no lint checks row-level security across the catalogue	Composite (tenant, id) primary and foreign keys; (tenant, cache key) for both caches; the saver sets the tenant per transaction; a catalogue lint for forced row-level security, the tenant policy and no UPDATE or DELETE on audit tables, with a deliberately permissive policy as its negative control	The database keeps every reference inside one tenant; prerequisite for partitions, cells and per-tenant export	Unique and foreign-key checks bypass row security (PostgreSQL); cheap while the data is fixtures, costly after	M	NT1; rebuild the schema from migrations while the data is fixtures	Churn across 15 migrations	The lint covers 100% of tenant tables and its negative control fails as intended; a cross-tenant foreign-key insert is rejected

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
NT10	None (addition)	The agent loop only counts calls, and refusals and truncation are only logged; a node retry samples earlier turns again; tool descriptions sit outside the bundle; tier 2 moved to Haiku 5.5 unchanged would think inside its 2,048-token cap; the run cap counts cache reads in full and cache writes not at all; the judge shares tier 2's model and relies on temperature 0; reviewer decisions are not kept as labels (6.7 to 6.9)	One stop function with typed reasons and loop detection, every non-normal stop ending in needs_review; each model turn recorded as its own step; tool descriptions and error texts in the bundle hash; thinking and effort set per call type, with thinking off for tier 2; run budgets counted as the peak context of a call plus dollars; single-step, fuzzed-result and replay tests on the stub model and a contract test per tool; the judge moved to a different Claude model, with a self-grading block and three-run consistency; an insert-only review label table with a consent scope	Runs stop for a reason a reviewer can see, retries stop re-sampling the model, the Haiku 5.5 move cannot truncate extractions, judges stop grading their own output, and labels start accumulating for every later eval and training decision	Small changes inside today's graphs that the 2025-2026 evidence ranks above adding agents or models (6.8, 6.9)	S	NT8 for keyed evals; pairs with item 4 of 3.5 (each model call its own step); one additive migration for the label table	More needs_review at first as stops become visible; labelling time	Every run ends with a typed stop reason; no judged case where the judge model wrote the output; fuzzed tool results never yield a status; every wrong mark, edit and rejection lands in the label table

Capacity (inferred): the first report estimates Q1 to Q13 at 17 to 24 engineer-weeks, NT9 adds 2 to 3 and NT10 another 2 to 3, so ten weeks need about 2.1 to 3 engineer-weeks a week: three engineers at about three-quarters to full time, or about eleven to fifteen weeks at the first report's 2 engineer-weeks a week.

Amendments to the first report's quick wins. The table keeps the first report's scopes. This design adds five things to them, which both documents should take together:

1. **Q11:** put the rule version and a hash of tenant parameters into the evaluation key, not only on the verdict, or a corrected rule over unchanged data is still dropped as a duplicate (Decision 3).

2. **Q12:** state that a 404 on any listing means "not visible" and makes the pull incomplete. GitHub answers 404 rather than 403 to a request not authorized for a private resource (GitHub, read 9 Nov 2026), and today that path ends in "no GitHub grants on record" and a GH-1 PASS (`evaluators/deterministic/gh1.py:42-49`).
3. **Q8:** mark truncated tool results; today they are cut at 20,000 characters, possibly mid-JSON, without the model being told (`llm/client.py:136`).
4. **Q13:** the different-family judge applies to fixture suites only (Decision 9); add Haiku 5.5 with thinking off to the nightly run, to measure refusals and pass^5 before a deprecation notice forces the move.
5. **New:** add NT9 to the quick wins and its lint to Gate 1. From 6.6, three small additions ride on existing items: NT5 moves evidence and tool output out of the user turn and strips invisible Unicode; NT3's redaction also covers traces, with a CI test that fails on any evidence text; and NT8's judges get the code-verified spans, not the agent's reasoning, with empty and punctuation-only outputs added as negative controls.

Two medium-term items extend the first report: MT2 adds Decision 4's triggers to M2's harden-first order, and MT6 accepts M7's HR aggregator only behind the sync contract (U8 in Section 9).

Gate 1, week 10: before CCP takes real customer data or relies on a real model's output. NT2 to NT7, NT9 and NT10 are merged, each with a test that reproduces its defect; CI is green on the private remote; a keyed eval run has measured the false-PASS rate on labelled cases and it is under the agreed limit; production calls go through a zero-retention organization and the Voyage training opt-out is on.

8.2 Medium term: months 3 to 8

Order: MT2 and MT3 start first, because MT4 and MT5 build on them; MT5 lands before any real tenant data; MT1 comes before MT6; MT7 runs throughout; MT8 follows MT1; MT9 follows MT3 and MT7; MT10 follows MT2, MT3 and NT10.

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT1	M1	`pull()` is the whole contract (`connectors/base.py:49-54`): no connection record, cursor, completeness, rate limit or test harness; what an error means differs by endpoint	Connections as data with daily checks; sync runs of scopes that end complete, partial, failed or not visible; absence only from complete authoritative scopes; a per-endpoint error table; content-hash snapshots; backoff and rate buckets; a GitHub App; a conformance harness with recorded responses and negative controls	Every connector inherits completeness; absence becomes a proven fact	One contract beats each connector solving completeness alone; a unified-API vendor would drop provider ids and request-level receipts (inferred)	L	NT7; MT2 for schedules; fixture readers emit sync runs too	Harness upkeep; GitHub permissions differ by plan	The harness is green, negative controls included, for every live connector; at least 99% of syncs complete; an unchanged re-pull lands zero rows

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT2	M2, re-scoped	One loop visits every tenant every 2 seconds (`ingest/worker.py:67-103`); offsets are numbered at insert but visible at commit, so a second process can skip rows (first report D4); the concurrency cap is read, then claimed (`evaluators/runner.py:136-149`); graphs checkpoint asynchronously by default; tool loops keep only the first request and last response (`agent/core.py:328-330`)	Leases with heartbeats; fair claiming through a work-hint table with LISTEN/NOTIFY wake-ups; ordered offsets from a per-(tenant, topic) counter; one evaluation in flight per subject; an older verdict filed into history without displacing a newer head; a `schedules` table in each tenant's time zone; agent runs as jobs with deterministic thread ids and synchronous checkpoints; every model and tool call a recorded step; the run log in Postgres; approvals as data with an idempotent apply job. Then the DBOS spike: kill a worker mid-step, overlap two deploys, fail over Postgres, recover with three processes and no Conductor	Several workers become safe; resumes never sample a model again; every AI output can be rebuilt end to end	One queue model with transactional enqueue; after these fixes an engine adds little for minute-long jobs (Decision 4) (inferred)	L	NT4; additive columns and tables; `event_log` stays the outbox	CCP owns about a thousand lines of concurrency code (inferred); the engine triggers are reviewed each quarter	A chaos test with three workers and overlapping deploys loses or duplicates no verdict; no model call is sampled twice on resume; 100% of agent runs replay with an empty request diff

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT3	M3	Tools have no class, output schema or single policy point; agent tools use the application role, which can insert (`storage/db.py:34-36`, `storage/migrations/0005_state.sql:66`); results are cut silently (`llm/client.py:136`)	The typed registry; `policy.check` (permit, deny or ask; fails closed; logged); explicit truncation and missing-data errors; a taint flag that removes act-class tools; a Rule-of-Two lint; a read-only role for agent tools; a credential module in the worker; REST and a read-only MCP server generated from the registry, opened to customers only after MT4 and on a design partner's request; `policy.check` denies by default, with a log-only phase for new permit rules and one principal per graph type (6.6); a small, stable tool set per graph, with tool search only past about 20 tools (6.4)	One permission check per action for the UI, the API, CCP's agents and customers' agents; a tool bug cannot write	Vanta's capability layer, Harvey's policy engine and CISO Assistant's read-only profile (Section 7.1 to 7.3) in one small module	M	NT5 (Q8); existing tools wrapped, not rewritten	MCP churn; Python SDK support for the 2026-07-28 revision unconfirmed (U12)	100% of tool calls carry a logged policy decision; zero write tools in the read-only profile; the silent-truncation counter stays at zero

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT4	M4	Typed reviewer names (`storage/migrations/0011_remediation_approvals.sql:50`, `storage/migrations/0004_identity.sql:41`); nothing makes a decision unique (`approvals/store.py:98-112`); proposals can be updated (`storage/migrations/0011_remediation_approvals.sql:26`); tenants live in YAML; no API or UI	Cognito federated to each customer's SAML or OIDC provider; roles per tenant; a tenants table; the approval service (one validated decision per item; payload and precondition hashes; expiry; separation of duties; a single-use grant; an append-only execution journal; rollback as a new proposal; a code-written risk summary); identity merges, exceptions, scope changes and break-glass through it; FastAPI REST and the server-rendered UI v1, whose inbox never acts on a missing or expired decision, whose run pages read the run log and carry a Stop button, and whose abstentions show a reason code and a next action (6.10)	Every consequential decision has an authenticated owner bound to the exact payload; customers stop needing the command line	Binding payload and precondition answers Hyperproof's warning and "Possession of a run ID or decision ID is not authorization" (Section 2.3, 3.5); Cognito federates SAML and OIDC with no new vendor (AWS)	L	NT2; MT3; MT5 for the cell and IAM roles	UI scope creep (hold v1 to the listed screens); Cognito's SCIM and per-tenant SSO administration may be clumsy (inferred)	100% of approvals by an authenticated user with the right role; a concurrent-approve test yields one decision; zero executions without a grant; a design partner onboards unaided in under an hour

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT5	M5, M6, plus the chain	CCP runs on a laptop with the raw zone on local disk; the stored hash covers CCP's re-serialization (`ingest/raw_zone.py:62`); grants stop the application, not an administrator or an altered backup (inferred)	Terraform for RDS PostgreSQL 17 (Multi-AZ, pgvector at least 0.8.2), ECS Fargate `api` and `worker`, CloudWatch alarms and point-in-time restore; an S3 raw bucket with versioning, governance-mode Object Lock with event holds, and per-tenant SSE-KMS keys with Bucket Keys; pointers with version id, a SHA-256 of the bytes received and the provider's request id; a per-tenant chain over audit rows, written in the inserting transaction; hourly KMS-signed checkpoints in a compliance-mode bucket in a separate account; a scheduled `ccp audit verify`; CI/CD with the eval gate	CCP serves customers on managed services; an auditor can check the record without trusting CCP's database	Open formats CCP controls, after QLDB's retirement; compliance mode only where irreversibility is wanted (6.1, row 3)	L	NT9 and MT2; copy the fixture raw zone with recorded hashes; each tenant's chain starts at a genesis entry	Retention settings are hard to undo; delete markers are not protected (AWS); a chain nobody verifies gives false comfort	Deploy from main in under 30 minutes and roll back in under 10; hourly verification with zero mismatches; a restore drill detects a deliberately altered row; 100% of raw rows carry a version id

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT6	M7	No cloud or identity-provider evidence	AWS onboarding through CloudFormation quick-create and StackSets, with a vendor-generated external ID per account, checks that the role works with the ID and fails without it, refusal of ReadOnlyAccess or any write action, and daily re-checks; SecurityAudit plus explicit denies on secret-bearing reads; Config first, live reads as the fallback; the S3 test over the effective bucket-and-account setting; snapshots of sources AWS overwrites; an identity-provider connector keyed by immutable user ids; HR data through an aggregator only behind the sync contract	A first customer's cloud, people and access are covered, with proof of completeness	Matches the market's read path (Drata, Secureframe) and adds completeness (Section 6.2)	L	MT1, MT5	AWS edits managed policies on its own schedule; throttling is shared with the customer; some customers lack Config	At least 99% of syncs complete; onboarding refuses a role that works without the external ID; zero false PASS on the AWS golden set

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT7	M8, plus bundles	No release unit and no production quality monitoring; Haiku 4.5's commitment has lapsed (Anthropic)	Nightly gates on confidence bounds; labelled sets of 100 to 200 per failure mode, with a second annotator on about 20%; no judge gates below 85% agreement with experts; release bundles on every run and verdict; shadow diffs over stored artifacts; production checks on every run, a judged nightly sample and 50 to 100 traces read weekly; tier 2 to Haiku 5.5 with thinking off and agents to Sonnet 5.5, through the gates; a second reader on every AI-derived PASS; effort, prompt lines and tool descriptions as bundle fields, with per-model evals on every change and a daily production canary (6.6); gold sets weighted towards FAIL cases and sized to the false-PASS bound (6.6)	Model and prompt changes ship only when measured quality holds; tier-2 calls about seven times cheaper (6.3)	Sierra's release snapshots (Section 7.3); migrating under gates beats migrating under a 60-day notice	M, plus labelling	NT8; MT2 for the run log	Refusals on security evidence; flips at temperature 1.0; labelling cost	100% of verdicts name a bundle; a regression injected in shadow is caught within 24 hours; refusal rate per test under its limit; pass^5 at or above the Haiku 4.5 baseline

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT8	M9, plus pinned inputs and coverage	The impact table is code (`evaluators/impact.py:149-209`); evaluators read the latest observations (`evaluators/context.py:116-129`) and cite entity ids, so a verdict cannot be re-derived from what it saw; tenant parameters are recorded nowhere (`evaluators/context.py:55-58`); the rollup counts a test pending only when it has no state rows (`knowledge/rollup.py:76-78`)	Frameworks, controls, tests and the impact table as versioned data with effective dates; rule changes as corrections or amendments, with preview and notice; an input manifest and read watermark on every verdict, with the inputs hash in the key; `recorded_seq` and `ts_kind` on observations; tests declare which sources must be complete; an expected-subject inventory from complete scopes; the rollup's `pending` extended to unassessed, stale and parked subjects; a tier-1 replay command and a nightly sentinel; long-tail checks as YAML files	No PASS over a partial or stale population; rule fixes take effect and are traceable; every deterministic verdict can be re-derived	Rebuilding from stored inputs is the reproducibility claim that also covers model output (Decision 3)	L	NT6, MT1; one re-evaluation wave when the key changes, recording a confirmation, not a new state row, when status and valid time are unchanged	As-of queries cost more; more `pending` at first, which the UI must explain	Zero requirements pass with an unassessed expected subject; 100% of sampled tier-1 verdicts re-derive exactly; every test reachable from a change

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT9	None (addition)	Scanned pages of mixed PDFs and images drop out; chunks lack offsets and versions; the index keeps current evidence only and a model change overwrites vectors in place; voyage-3-large and rerank-2.5 are legacy; 30 golden queries and a flat 0.60 retrieval factor (6.5)	The retrieval layers of 6.5: Claude transcription of scans and screenshots, with the method in provenance; structure-aware chunks with offsets, as insert-only rows per evidence version with as-of search; whole-document context enrichment; exact-match, outline, read-span and typed fact tools for agents; voyage-4-large and rerank-3 after a paired comparison; a golden set of 25 to 50 queries per evidence type with recall, MRR and distractor gates, and a calibrated retrieval confidence	Tier 2 and agents find the right span more often, every citation stays checkable as of any date, and retrieval changes ship only when measured	Keeps retrieval in Postgres and per tenant; published results favour combining agentic and semantic search over either alone, and typed facts over a document graph (6.5, Decision 10)	M	MT3 for the tools; MT7 for release bundles and the eval gate; MT2's run log for agent tool calls. Each tenant is re-embedded into new rows in the background and switched after the gate passes; old rows stay for replay	Labelling time; transcription and enrichment cost per version; RDS may not offer pg_textsearch, so BM25 on large tenants waits (6.5)	Recall@20 and Recall@6 at or above targets set from the baseline; the distractor share in the top 6 does not rise; no scanned page is dropped without an event; replaying a past verdict returns the same chunks

#	Matches	Limitation	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
MT10	None (addition)	Breadth work (vendor reviews, unanswered questionnaire questions, audit-prep gap lists) has no safe way to fan out or run across days; a failed check gets no repair; there are no harness metrics, context ceilings or long-context probes (6.7, 6.8)	Task and item tables in Postgres, with dollar and deadline caps and at most three runs per item; a fan-out launcher over the queue (per-item keys, budgets, schemas, merge by key), shipped only after a single-run baseline per task; resumption after a human answer from stored state; one code-driven correction round after a failed check; context ceilings per call type; harness metrics and four context probes in the eval suite; nightly stratified judge samples and a monthly expert relabel of 20 judged items (6.9)	Long tasks finish item by item with a record a reviewer can replay, breadth work runs in parallel without shared context, and harness regressions are caught before release	Code-driven planners, merges and stops are what made long and parallel agent work hold up at Anthropic and Cursor; CCP keeps them in its own queue instead of adding an agent framework (6.7, 6.8)	M	MT2 for the queue and run log; MT3 for read-only tool lists; MT7 for bundles and eval gates; NT10	Fan-out cost if items are mis-sized; a correction round can hide a weak prompt, so its success rate is tracked	A bundle that raises loop or cap stops does not ship; a failed item never blocks the rest of its task; no task is marked done by a model; the first-pass check rate is tracked per bundle

Capacity (inferred): the first report estimates its M1 to M9 at 38 to 52 engineer-weeks; the run log (MT2), the chain (MT5) and pinned inputs with coverage (MT8) add about 8 to 12, MT9 adds about 4 to 6 and MT10 another 4 to 6, so about 54 to 76 over about 22 weeks. That needs four engineers at three-quarters to nearly full time; with fewer, MT6′s identity-provider connector, MT9′s model switch and MT10′s fan-out launcher move to month 9.

Gate 2, month 8: before long-term items and before a first production customer. Every live connector passes the harness and records completeness (MT1); the crash and overlapping-deploy tests pass with three workers (MT2); only signed-in reviewers with the right role can approve, and every execution consumes a grant (MT4); the raw zone is in S3, `ccp audit verify` is green hourly, and a restore drill has caught a deliberately altered row (MT5); nightly real-model evals gate every release bundle (MT7); no

requirement passes with an unassessed expected subject (MT8); the retrieval golden set meets its recall and distractor targets on the new models (MT9); and a third-party adaptive red team has tested the evidence paths (6.6).

8.3 Long term: months 8 to 18

#	Matches	Limitation or opportunity	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
LT1	L3	One live action, a collaborator removal behind five gates (`remediation/gates.py:53-84`); cloud findings have no reviewable fix path	Investigator read tools on the connector clients (Config history in 7-day windows, CloudTrail lookups under 2 requests a second, a search for the resource's infrastructure code), landing every response as raw evidence first; code-written investigations when the rule explains the failure; the model picks a catalogue action and its parameters; code renders the diff from a versioned template, validates it offline and checks its scope; approval with payload and precondition hashes; the executor re-checks and opens the pull request with a one-repository, one-hour GitHub App token; PASS only from a re-read	Shorter time from FAIL to PASS, a record linking every step, and no AWS write credential	Drata and Vanta remediate through pull requests (Drata, Vanta); model-chosen commands are rejected (Section 7.5)	L	MT1, MT2, MT4, MT6	Customers without infrastructure code get guidance only; telling which resources are in code is unsolved (U7)	Zero executed diffs that differ from the approved hash; median time from FAIL to PASS; share of pull requests merged unchanged; reopen rate within 30 days

#	Matches	Limitation or opportunity	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
LT2	L2	Uploads must be PDF or text, and anything else is skipped (`connectors/manual/uploads.py:109-132`); scans and screenshots are not read	Segment each document version (sentence ids, table cells, pages); models cite segment ids or JSON Pointers and code renders the quotes; deterministic admissibility (date, period, subject, issuer); an adequacy verifier calibrated against auditors' accept-or-flag decisions; Claude's PDF input for scans; feedback seconds after upload; screenshots stored as raw evidence, transcribed and badged customer-attested or system-observed (6.10); shadow first	Customers fix weak evidence before the auditor sees it	Vanta's verifier is the best-documented approach (Vanta, 5 Aug 2026); CCP adds deterministic admissibility and pointer checks	L	MT7; a labelled sample per evidence type	Labelling cost; refusals on security screenshots (U5)	Agreement with auditor decisions of at least 85%; a false-accept upper bound under the agreed limit; re-upload rate

#	Matches	Limitation or opportunity	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
LT3	L1	No questionnaire feature	A fixed pipeline: applicability, library match, retrieval, tool-less fact extraction, answer direction from the current verdict where a question maps to a tested control, prose, then code checks; approved answers stored with their pointers and the verdicts they relied on, and marked stale when either changes; a person sends	Answers stay true as controls change, and each traces to evidence	Sprinto's staged passes (Sprinto, Aug 2026) plus CCP's live verdicts; no incumbent page describes invalidating an approved answer when its source changes (inferred)	L	MT3, MT4, MT7	Messy formats; reviewer fatigue	Reviewer edit rate per question type; zero answers that contradict current state; zero unsupported claims in sampled audits
LT4	L4	Proofs exist only inside CCP, and nothing defines what CCP asserts outside itself	As-of packages from bitemporal state with each result's method, raw pointers, inclusion proofs and a signed manifest, sealed in compliance-mode storage, with corrections as deltas; a time-boxed, read-only auditor workspace; an attestation record of everything asserted outside CCP under a versioned publication policy keyed on provenance, completeness and freshness; a verifier customers and auditors can run	Auditors and buyers can check the record without trusting CCP	Auditor-side agents check that "every conclusion traces to its evidence" (Fieldguide); Thoropass freezes evidence per audit (Section 2.1)	L	MT4, MT5, MT8; interviews with two or three audit firms first	Auditors' acceptance is unknown (U1, U2)	Share of auditor requests answered from the package; an auditor verifies a package offline

#	Matches	Limitation or opportunity	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
LT5	L5	Mappings are hand-written YAML (`knowledge/controls.yaml`, `knowledge/frameworks/soc2.yaml`, `knowledge/frameworks/iso27001.yaml`)	Mapping proposals with computed confidence and abstention, reviewed one at a time, as an offline batch first; each accepted change makes a new catalogue version	New frameworks in days, every mapping reviewed	Drata's mapping agent proposes and applies nothing on its own (Drata)	M	MT8; a named compliance reviewer	Reviewer load	Precision and recall at 3 on held-out frameworks; days to add a framework
LT6	L6	No published quality figures	A method card on every verdict and proposal; published results per AI workflow and release: method, datasets, false-PASS rate and pass^k	Buyers and auditors can check AI claims	Only Vanta and Sprinto publish numbers, and Trustero publishes figures without a method (Section 2.1, 2.3)	M	MT7; LT2 or LT3 live	Publication exposes weak spots	A published snapshot per workflow each quarter
LT7	L7	Three connectors against the 400+ integrations Vanta claims (Section 2.4)	Models draft declarative checks and connector mappings offline; path checks against recorded payloads, the conformance harness, a shadow diff and a reviewer gate each one; verdicts at run time stay deterministic	Breadth without model verdicts	Drata built 1,000+ tests this way, at about $20 to $30 per resource (Drata, 8 Dec 2025)	M, then per connector	MT1, MT8	Reviewer fatigue; subtle errors, caught by shadow diffs and the notice period	Checks shipped per engineer-week; zero false PASS in shadow; defects in the first 30 days

#	Matches	Limitation or opportunity	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
LT8	New	Every consequential action needs a person, yet users approve 93% of permission prompts (Section 3.6), so blanket approval adds little	Levels per (tenant, workflow, action class): shadow, suggest, approve each, sampled review. Only reversible, low-impact actions may reach sampled review; promotion needs a quality snapshot (a false-accept upper bound from blind expert re-review, pass^k, a minimum count) and an administrator's opt-in; an exposure budget per tenant; automatic demotion on an alarm or a bundle change; every auto-accepted decision cites its snapshot; a simulator replays past proposals through a candidate level	Review time goes where it matters, and autonomy is auditable	Measured levels rather than presets (Drata) or user pre-approval (Vanta); Ramp caps the exposure an agent can create (Ramp); Drata simulates a policy "against a year of real historical traffic" before enforcing it (Drata, 4 Aug 2026)	M	MT4, MT7, LT6; Gate 3	Over-trust; liability split (U4)	Share of eligible decisions auto-accepted; sampled false-accept rate within its bound; zero irreversible actions without a person
LT9	New	One database in one region	PgBouncer in transaction mode; partitions with their own index for the largest tenants; a read replica; dedicated cells for regulated or very large tenants; EU cells on Bedrock EU inference profiles and Voyage's EU endpoint	Growth without a redesign; residency for EU buyers	The same code and migrations everywhere; Anthropic's own platforms offer only global or US inference (Anthropic)	L	NT9; the tenants table	Too many partitions; keep dedicated partitions to the top few hundred tenants	p95 latency per tenant flat as tenants grow; a new cell stood up in under a day

#	Matches	Limitation or opportunity	Change	Benefit	Why preferable	Size	Dependencies and migration	Risks and tradeoffs	Success metric
LT10	New	Resources not managed in code get guidance only	Only for a design partner who asks: a customer-deployed `ccp-remediator` stack; allowlisted AWS-owned runbooks; a 15-minute session with a session policy narrowed to the resource and `SourceIdentity` set to the approval id; the execution id recorded; approvals signed with a passkey over a challenge derived from the payload and precondition hashes	Fixes for resources outside code, approved and audited exactly	AWS-owned runbooks verify their own result (AWS); a session's permissions are "the intersection of the role's identity-based policy and the session policies" (AWS); no competitor documents a vendor-held write role (Section 6.2)	M	LT1; Gate 3	A crown-jewel credential path; drift from the customer's code	Zero runbook runs without a matching signed approval and pre-state; CCP's own re-read confirms the fix in 100% of runs

Capacity (inferred): the first report's L1 to L7 total 29 to 40 engineer-weeks and LT8 to LT10 add about 10 to 14; over about 43 weeks that leaves room for connectors drafted through LT7, catalogue content and customer requests.

Gate 3, about month 12: before any workflow moves to sampled review (LT8) or the runbook path ships (LT10). Blind expert re-review shows a false-PASS upper bound under the agreed threshold for the workflow; pass^5 meets its target; the exposure budget and automatic demotion have been tested; for LT10, a design partner has asked and passkey-signed approvals are live.

9. Top architectural decisions

The ten choices that shape everything else, each with the options considered, the evidence and what would change it.

Decision 1. Where model judgement may sit in a compliance decision

- **Options:** (a) a model returns the status; (b) models extract typed fields with pointer citations, and a versioned rule decides; (c) a model decides and code audits the decision.

- **Recommendation:** (b) for every status, AI-derived ones included. AI widens coverage by drafting rules offline and extracting facts, never by deciding.
- **Evidence:** since API version 2026-10-21, requests sample at temperature 1.0 and "No other setting replaces it" (Anthropic); serving can shift behaviour under a fixed model id (Anthropic); the best model scored 37.4% on one try and 20.6% across four on Sierra's τ-knowledge (Sierra, May 2026); CCP's own checker passed an "In progress" record as completed (Section 1.3).
- **Tradeoffs:** an extraction schema per interpretive test; more `needs_review` on nuanced evidence, which verifiers and reviewers absorb.
- **What would change it:** nothing in current evidence. Even a model with pass^k and a false-PASS bound at target on CCP's labelled sets would earn lighter review (Decision 5), not the right to set a status.

Decision 2. What a PASS asserts

- **Options:** (a) missing data passes (today's effect); (b) missing data fails; (c) only a complete, authoritative read may assert absence, anything else abstains, and the rollup counts unassessed and stale subjects.
- **Recommendation:** (c).
- **Evidence:** a 404 becomes "nothing there" and then a GH-1 PASS (`connectors/github/client.py:55-60`, `evaluators/deterministic/gh1.py:42-49`), and GitHub answers 404 rather than 403 for private resources a request cannot see (GitHub); on S3 Control, a 404 means "not configured", a real fact (AWS); the rollup cannot see unassessed subjects (`knowledge/rollup.py:76-78`); Comp AI reconciles only from complete scans (Section 2.2).
- **Tradeoffs:** more `needs_review` and `pending` at first; per-endpoint error tables to maintain.
- **What would change it:** a provider that offers a change feed with a completeness guarantee, for that source only.

Decision 3. What identifies and reproduces a verdict

- **Options:** (a) a trigger-time key (today); (b) rule and parameter versions in the key; (c) a content address over pinned inputs and the whole method. And: reproduce by re-running models, or by rebuilding from stored artifacts.
- **Recommendation:** (b) now and (c) by month 8. "Reproducible" means rebuilt from stored inputs and stored model outputs; agreement across re-runs (pass^k) is a separate, measured gate.

- **Evidence:** the key omits rule and parameters (`evaluators/verdict.py:72-90`, `evaluators/context.py:55-58`), full runs reuse the newest observation time (`evaluators/runner.py:200`), and evaluators read the latest observations (`evaluators/context.py:116-129`); re-running a model cannot reproduce a verdict (Decision 1).
- **Tradeoffs:** input manifests and full requests and responses stored under CCP's own retention; one re-evaluation wave when the key changes.
- **What would change it:** auditors requiring agreement across re-runs as the standard (U1); pass^k would then become a per-test release gate.

Decision 4. Which engine runs background work

- **Options:** (a) harden the Postgres queue CCP owns; (b) DBOS Transact; (c) Temporal; (d) Hatchet; (e) Procrastinate.
- **Recommendation:** (a). Run the DBOS spike in the medium term as a decision gate, and adopt DBOS only if it passes and a trigger fires: a workflow with more than three durable steps across external systems, two or more queue incidents a quarter, or more than about 20 worker processes. The audit record never lives in engine tables.
- **Evidence:** the four gaps are fixable in place (Section 3.5), and the first report's D4 recommends the same order; DBOS's pooling limit, its contradictory multi-process recovery docs, its version-matched recovery and its missing LangGraph integration (Section 3.5); Temporal workflows may do no I/O and its LangGraph plugin is a 0.1.0 preview (Temporal); Hatchet's cron is UTC-only and skips ticks missed while it is down (Hatchet).
- **Tradeoffs:** CCP owns about a thousand lines of concurrency code and its tests (inferred); there is no workflow UI.
- **What would change it:** a trigger firing, or a spike showing correct multi-process recovery without Conductor on direct connections.

Decision 5. How human approval and autonomy work

- **Options:** for authenticity, (a) typed names; (b) an SSO session plus server checks; (c) (b) plus payload and precondition binding and single-use grants; (d) (c) plus a passkey signature over a payload-derived challenge. For autonomy, presets (Drata), user pre-approval except destructive actions (Vanta), exposure budgets (Ramp), or levels earned from measurement.
- **Recommendation:** (c) for every consequential decision by month 8; (d) for the opt-in cloud write path; autonomy only as measured levels for reversible, low-impact actions, with exposure budgets, an administrator's opt-in and automatic demotion.

- **Evidence:** CCP's fail-open defects (`approvals/cli.py:88-90`, `approvals/actions.py:51-57`, `remediation/runner.py:243-289`); "Possession of a run ID or decision ID is not authorization" (OpenAI Agents SDK); users approve 93% of permission prompts (Anthropic); WebAuthn's signed client data carries the relying party's challenge (W3C, 25 Aug 2026); Drata's autonomy presets publish no metric gate (Drata, 9 Sep 2026).
- **Tradeoffs:** passkey enrolment friction; review load until levels are earned; per-tenant labels cost money.
- **What would change it:** customers who will approve only through their own IdP (label those decisions session-only and add compensating checks); a regulator requiring per-item approval regardless of measurement.

Decision 6. How CCP changes customer systems

- **Options:** a vendor-held write role by default; a pull request whose diff a model writes; a pull request whose diff code renders; an opt-in, customer-owned runbook role; model-chosen SDK commands; a browser agent in the user's session.
- **Recommendation:** code-rendered pull requests by default; the runbook role only for a design partner who asks, with passkey-signed approvals; never model-chosen commands or browser sessions.
- **Evidence:** Drata and Vanta fix through pull requests (Drata, Vanta); none of the six incumbents documents a vendor-held write role, in our reading of their docs (Section 6.2); Comp AI's denylist and regeneration after approval (Section 2.2); AWS-owned runbooks verify their own result (AWS); Config auto-remediation "can be initiated even for compliant resources" (AWS).
- **Tradeoffs:** slower closure; dependence on the customer's infrastructure code.
- **What would change it:** design partners who need fixes at volume for resources outside code (LT10 earlier).

Decision 7. The tool surface

- **Options:** per-graph tools (today); one generated capability registry; third-party MCP servers as the integration bus; an agent-gateway product.
- **Recommendation:** one registry that generates agent tools, REST and a read-only MCP server; one fail-closed policy check; provider reads on the connector clients. MCP is an output of CCP, not its internal bus. Propose-only MCP tools come later; act-class tools never appear on MCP.
- **Evidence:** Vanta exposes each action as one authorized service method to REST, CLI, its agent and MCP (Vanta, Aug 2026); Harvey's engine fails closed (Harvey); CISO Assistant's read-only profile does not register write tools (Section 2.2); Vanta's hosted

MCP has no read-only mode (Vanta); clients must treat MCP annotations as untrusted (MCP).

- **Tradeoffs:** a generator to build; MCP spec churn.
- **What would change it:** customers' own agents becoming the main way CCP is used (propose-only MCP earlier).

Decision 8. Evidence integrity and tenancy

- **Options:** for integrity, grants only; S3 compliance mode; governance mode with event holds plus a per-tenant chain and signed checkpoints; a managed ledger; a transparency log. For tenancy, pooled with row-level security; a schema per tenant; a database per tenant; pooled cells plus silo cells on contract.
- **Recommendation:** governance mode with event holds for raw evidence; a per-tenant chain over raw objects and database audit rows, with hourly signed checkpoints in a separate account's compliance-mode bucket; pooled cells with forced row-level security and tenant-first keys; silo cells on contract.
- **Evidence:** the Object Lock, QLDB, CloudTrail-digest and Postgres facts in 6.1, row 3; AWS's decision matrix (AWS, Apr 2024); Vanta's 2025 exposure (TechCrunch).
- **Tradeoffs:** the verifier must run and the keys must be managed; row-level security has planner pitfalls, such as full-text indexes going unused (Tom Lane, 30 May 2025).
- **What would change it:** buyers asking for public verification (add an RFC 3161 timestamp or a transparency log); a contract requiring a dedicated key or independent restore (a silo cell).

Decision 9. Models and data posture

- **Options:** one model everywhere; Claude-first routes; a cross-vendor router. For judges: the same model; a different Claude model; a different family.
- **Recommendation:** Claude-first routes pinned by dated id inside release bundles: Haiku 5.5 for extraction with thinking off, Sonnet 5.5 for agents, Opus 5.5 only where measured. Production evidence only through a zero-retention organization and online calls; batches only in a separate eval organization on synthetic data; Bedrock as a rehearsed, manual standby. Judges use a different Claude model on customer data and a different family offline.
- **Evidence:** prices and lifecycle (6.3, Cost); batches, the Files API, code execution and Managed Agents fall outside zero retention, and the Fable models require 30-day retention (Anthropic); Bedrock has no Message Batches and makes Amazon the processor (Anthropic); the 5.5 models' classifiers can refuse "Benign cybersecurity work" (Anthropic); Anthropic asks graders to be "a different model" (Anthropic).

- **Tradeoffs:** vendor concentration; refusals on security evidence; a manual failover takes minutes.
- **What would change it:** a 5.5 refusal rate on CCP's evidence that is too high (keep those tests on Sonnet 5, committed to at least 30 Jun 2027); a customer DPA that allows a second vendor (cross-family judges online).

Decision 10. How agents find evidence

- **Options:** (a) vector search alone; (b) agentic search alone, with the model calling search and read tools; (c) layered retrieval: hybrid full-text and vector search over chunks with added context, reranking, and exact-match, outline, read-span and typed fact tools for agents; (d) a knowledge graph built from documents; (e) long context in place of retrieval.
- **Recommendation:** (c), inside Postgres and per tenant (6.5). Tier 2 keeps one templated search that replay never repeats; agents get the search loop and the exact tools; long context serves only to read one document end to end, quotes first; typed facts stand in for a document graph. Retrieval is evaluated on its own, and a parser, chunker, model or prompt change ships only when recall holds and distractors do not rise.
- **Evidence:** Cursor reports 12.5% higher answer accuracy on average from adding semantic search to its agent's grep, and a stronger retriever lifted a research agent from 55.9% to 70.1% (BrowseComp-Plus, Aug 2025); added context cut top-20 retrieval failures by 35%, and by 49% with the context also in the lexical index (Anthropic, Sep 2024); document-built graphs kept about two thirds of answer entities and won only on multi-hop questions, by about 3 points (Han et al., Nov 2026); accuracy fell with input length across 18 models (Chroma, Jul 2025).
- **Tradeoffs:** versioned chunk rows, partitions for the largest tenants and a golden set to label and keep; transcription and enrichment cost per evidence version; agents spend more tool calls than one search.
- **What would change it:** p99 latency that partitions cannot hold, or per-tenant keys required on the index itself (a dedicated engine); a current model with no loss at 200K tokens on CCP's set (more long-context reads); corpus-wide questions that typed facts cannot answer (a trial graph index, in drafting only).

Open questions

These need a spike, a customer conversation or counsel before the decisions above are final. Section 8 refers to them by number.

#	Question	Needs
U1	What reproducibility standard will SOC 2 and ISO 27001 auditors accept for AI-derived results: rebuilt from stored artifacts, agreement across re-runs (pass^k), or both, and at what threshold?	Conversations with two or three design-partner audit firms
U2	Do SOC 2 and ISO auditors test the completeness and accuracy of platform-generated evidence as strictly as financial auditors test information a company produces? This sets the value of the coverage rollup and sync receipts	The same audit firms; a read of the AICPA SOC 2 guide
U3	What false-PASS upper bound per test makes an AI-derived PASS attestable without review, and how many expert labels per test does it take to show it?	A risk decision by CCP′s leadership, then a labelling spike
U4	Will customers opt into earned autonomy, who signs (the tenant administrator or each control owner), and how is liability split for an auto-accepted action?	Customer conversations; counsel
U5	How often do the 5.5 models refuse benign security evidence, and how often do verdicts flip across re-runs at temperature 1.0, per test on CCP′s golden set? No published figure exists	A spike within NT8
U6	Is DBOS′s self-managed recovery correct with several processes and no Conductor, with its system database on a direct connection beside PgBouncer?	The MT2 spike
U7	What share of target customers record AWS Config, in which mode, and how much of their AWS is in infrastructure code that CCP can detect reliably? This decides how often the pull-request path applies	Customer conversations; a detection spike (stack tags, Terraform state, declared mappings)

#	Question	Needs
U8	Can an HR aggregator meet the sync contract (complete syncs, raw responses, remote ids), and will customers accept another credential-holding subprocessor?	A spike against one aggregator; customer conversations
U9	Will customers enrol passkeys to approve cloud write actions, or insist on approving only through their own IdP, which cannot bind a payload?	Design-partner conversations before LT10
U10	Does destroying a tenant's key satisfy offboarding and erasure duties for WORM evidence, and which evidence classes may be kept under GDPR Article 17(3)? The EDPB holds that encrypted personal data is still personal data (EDPB, 7 Jul 2026)	Counsel
U11	Who decides whether a rule change is a correction or an amendment, and must a customer acknowledge a correction that rewrites part of an audit period?	Customer and auditor conversations
U12	Does the MCP Python SDK support the 2026-07-28 revision well enough for a stateless production server, and do customers' clients?	A spike before MT3 opens MCP
U13	At what chunk count per tenant does exact search at 1,024 dimensions exceed the latency budget?	A synthetic benchmark: 1,000 tenants with skewed sizes
U14	What volumes should planning assume at 18 months: tenants, test-subject pairs, change rates, investigation rate?	Sales and product planning
U15	Does Bedrock accept API version 2026-10-21 for CCP's models, and will customers accept Amazon as processor during a failover?	A spike; DPA review with customers

#	Question	Needs
U16	Should a new tenant's first full extraction run as a batch at half price, with its 29-day retention disclosed and agreed, given that batches fall outside zero retention (Anthropic)?	Customer conversations

Sources

Links cited in this report, grouped by kind. Dates are publication dates where the page gives one; pages without a date were read on 9 Nov 2026.

Compliance platforms

- AI evidence validation
- AI summaries: Enable ai summaries for tests
- Scrut developer API
- Create, schedule, and manage custom agents (Vanta Help) (2026-09-14)
- Design tools around information, not APIs (Chris Ng) (2026-09-02)
- Drata blog: Securing data at scale (2025-06-03)
- Drata Help: AWS Organizational Units connection (2026-08-27)
- Drata Help: Compliance as Code (2026-08-05)
- Drata Not Impacted by the May 2026 Braintrust Security Incident (trust.drata.com) (undated post; incident 2026-05-06)
- Drata: Drata extends trust management platform to continuously monitor and govern ai agents (4 Aug 2026)
- Scrut: Integrating 15Five
- FAQ: Frequently asked questions
- Fix failing monitors with Sprinto Assist (undated (screenshots ~2026-07; opened 2026-11-09))
- Framework Mapping Agent (Drata Help) (2026-09-02)
- Get started with ingestion
- Getting started with the TPRM Agent (Drata Help) (2026-09-09)
- Give your agents capabilities (Brett Bush) (2026-08-20)
- help: Why can t i reuse evidence from previous audits
- monitoring: Monitoring overview new experience
- Vanta: RCA customer story

- Remediate with MCP (Vanta developer docs) (undated (opened 2026-11-09))
- Scrut MCP tools: Supported scrut mcp tools
- Scrut September 2026 release notes; Scrut MCP tools (2026-09)
- Scrut: Getting started with Scrut Teammates (undated, read 2026-11-09)
- Secureframe MCP Server help; AI Evidence Validation (undated MCP page (opened 2026-11-09); evidence validation 2026-09-29)
- Secureframe Support: Comply AI for cloud remediation (one of several vendor pages in the notes) (2026-07-13)
- Secureframe Support: Managing your integration connections (undated (page says 'updated this week'), read 2026-11-09)
- snapshot API: Get test snapshot detail for an evidence row
- Sprinto Product Updates August 2026 (2026-08)
- Sprinto Product Updates July 2026 (2026-07)
- test updates: Test activity dashboard
- tests: Getting started with tests in scrut
- Vanta developer docs: Tests
- The cache miss that made our agent faster (Chris Ng) (2026-09-16)
- The Vanta AI Quality Eval Maturity Model (Andy Almonte) (2026-06-10)
- Thoropass MCP help
- Thoropass: MCP for audit blog; Senior PM AI Audit Automation posting (2026-09-22 (blog); job posting open Nov 2026)
- TPRM Agent autonomy settings (Drata Help) (2026-09-09)
- Using AI to Generate 1,000 Compliance Tests: What Actually Worked (Kevin Kho) (2025-12-08)
- Vanta blog, How we navigated database limits (Kannan)
- Vanta MCP (developer docs) (undated (beta; opened 2026-11-09))
- Vanta Product Updates (week of 2026-06-21) (2026-06-21 entry (page updated week of 2026-11-01))
- Vanta: Building a smarter retrieval system lessons from vanta ai (2025-02-18)
- Vanta's agent development principles (Noam Rubin) (2026-06-24)
- You can build verifiers for messy problems (Vanta Trustcraft, Zheng Tao) (2026-08-05)

Open-source compliance code

- getprobo/probo pkg/agentexecution/service.go L100-L157 (2026-11-09 (commit 7bbe06a))

- getprobo/probo pkg/coredata/scope.go L30-L98 (and pkg/gid/gid.go L79-L118) (2026-11-09 (commit 7bbe06a))
- getprobo/probo pkg/probot/channel/slack/execution_adapter.go L163-L201 (and agent.go L359-L365) (2026-11-09 (commit 7bbe06a))
- getprobo/probo pkg/server/api/mcp/v1/resolver.go L149-L230 (and schema.resolvers.go L712-L724) (2026-11-09 (commit 7bbe06a))
- getprobo/probo pkg/thirdparty/common_third_party_enrichment.go L132-L192 (2026-11-09 (commit 7bbe06a))
- intuitem/ciso-assistant-community backend/chat/ARCHITECTURE.md L49-L65, L572-L574 (2026-03-27 (last change to the doc; read at commit a867086 of 2026-11-09; code in backend/chat/tools.py L889-L892 and scoping.py L1-L92 confirms it))
- intuitem/ciso-assistant-community cli/ca_mcp/server.py L312-L370 (2026-11-09 (commit a867086))
- trycompai/comp apps/api/src/assistant-chat/assistant-chat.controller.ts L121-L129 (2026-11-04 (commit 150fed5))
- trycompai/comp apps/api/src/cloud-security/aws-command-executor.ts L121-L138, L772-L865 (2026-11-04 (commit 150fed5))
- trycompai/comp apps/api/src/cloud-security/reconciliation.service.ts L15-L29, L68-L71 (2026-11-04 (commit 150fed5))
- trycompai/comp apps/api/src/cloud-security/remediation.service.ts L512-L681 (2026-11-04 (commit 150fed5))
- trycompai/comp apps/api/src/integration-platform/utils/task-check-evaluation.ts L30-L67 (2026-11-04 (commit 150fed5))
- trycompai/comp packages/db/prisma/schema/dynamic-integration.prisma L1-L135 (2026-11-04 (commit 150fed5))

AI-native entrants and agent companies

- Agent development life cycle (Bavor, Taylor); Simulations: the secret behind every great agent (Sachi Shah) (2024-06-03 (simulations post 2025-08-19))
- Agentic Risk Operations (Ramp Builders blog) (undated on page (accessed 2026-11-09))
- AIUC-1 standard site (last updated 2026-10-15)
- TechCrunch: Delve accused of misleading customers
- Amp, Handoff (No More Compaction): Handoff (23 Oct 2025)
- Anecdotes Agent Studio page; A-CCM page; The agentic GRC era (Brittany Pereira) (undated (© 2026); data-foundation post 2026-06-09; MCP post 2025-10-16)

- AWS ML blog, "Production-grade AI agents for financial compliance: Lessons from Stripe", Phillippi, Cui (Stripe), Tariq, Musti: Production grade ai agents for financial compliance lessons from stripe (2026-06-26)
- Building AI agents in production (Miguel Rios Berrios); Agentic AI: Built with Guardrails, for Compliance (AJ Asver) (2023-10-16 (guardrails post 2025-09-19))
- Building Harvey's MCP Policy Engine (Hussain, Sudhakar, Gonella) (2026-10-07)
- Cognition, "Don't Build Multi-Agents" (Walden Yan) (2025-06-12)
- Cognition, "Multi-Agents: What's Actually Working" (Walden Yan) (2026-04-22)
- Complyance AI agents page; What is a GRC AI Agent? (Richa Kaul); Mycroft: Compliance automation for startups (Mike Kim) (2025-10-20 (agents page © 2026; Mycroft 2026-01-27))
- Cursor (Jediah Katz), Dynamic context discovery: Dynamic context discovery (6 Jan 2026)
- Cursor, Improving agent with semantic search: Semsearch (6 Nov 2025)
- Datadog, State of AI Engineering
- Factory Research, Evaluating Context Compression for AI Agents: Evaluating compression (16 Dec 2025)
- Fieldguide agents overview; What an AI workforce looks like inside an audit engagement (Amanda Waldmann) (updated 2026-08-10 (article 2026-06-04, updated 2026-08-17))
- Fin, "David vs Goliath: are small LLMs any good?", Joglekar & Yarullin: David vs goliath are small llms any good (2025-09-11)
- Fin, "How Fin Scales Customer-Defined Behaviour", Verma & Kelly: How fin scales customer defined behaviour (2026-09-25)
- From LLMs to enterprise-grade agents (Thiaga Rajan); Agent monitoring (VanLang, Hora); Confidence in every conversation (Valjean Clark) (2026-05-07 (supervisor posts 2025-10-02 and 2025-10-29))
- From SOPs to Agent Operating Procedures (Ryan Smith); The next generation of Simulations (Eric Hermann); The evaluation engine behind Decagon's AI agents (Max Lu) (2025-11-26 (simulations 2026-06-30; eval engine 2025-07-06))
- GitHub (Agarwal, Peet), How we're making GitHub Copilot smarter with fewer tools: How were making github copilot smarter with fewer tools (19 Nov 2025)
- Harvey, Murlidharan & Ge: Making review tables faster and more reliable (2026-10-22)
- How Harvey Secures Embeddings at Scale (Suha Sabi Hussain) (2026-04-30)
- How To Build Agents Users Can Trust (Ramp Builders blog) (undated on page (accessed 2026-11-09))

- Hyperproof AI validation test library; Hyperproof AI overview (help center); AI Guided Experiences (Maggie Paulk Welch) (2026-10-08 (overview ~Oct 2026; guided experiences updated 2026-04-09))
- Intercom Help, "How to roll out a release in Fin"
- Intercom: Announcing evals and releases (2026-08-13)
- Introducing BigLaw Bench: Arena (Beth de la Roche); BigLaw Bench – Sources (2025-11-07 (Sources post 2024-09-23))
- Introducing Harvey's Legal Agent Benchmark (Grupen, G. Pereyra, J. Pereyra); github.com/harveyai/harvey-labs (2026-05-06)
- Manus (Yichao 'Peak' Ji), Context Engineering for AI Agents: Lessons from Building Manus: Context Engineering for AI Agents Lessons from Building Manus (18 Jul 2025)
- May Product Update: Conveyor inside your favorite AI tools; AI Agents for Security Questionnaires (Christine Vidal) (2026-05-28 (questionnaire post 2026-06-22))
- Optro Leads the Global Audit Transformation with the Acquisition of AI-Native Midship; Optro Launches Domain-Trained Agents (PR Newswire) (2026-05-06 (agents launch 2026-10-14))
- Ramp Engineering, "How To Build Agents Users Can Trust"
- Rebuilding the Review Algorithm to Increase Accuracy and Speed (Hong, MacGregor, Rengarajan) (2026-04-21)
- Replit (Zhong, Zhao, Carelli), Enabling Agent 3 to Self-Test at Scale with REPL-Based Verification: Automated self testing (15 Dec 2025)
- TechCrunch: Vanta bug exposed customers' data to other customers (Whittaker) (2025-06-02)
- Trustero AI page; Trustero Announces AI-Powered Playbooks (Phillip Liu) (undated (© 2026); blog 2026-06-02)
- τ-knowledge: benchmarking agents on realistic knowledge (Shi, Zytek, Razavi, Barres) (2026-05-13)

Retrieval and search

- Chroma: Context rot (Jul 2025)
- Glean: Key terms
- Hugging Face: RTEB (Oct 2025)
- Timescale pg_textsearch
- pgvector CHANGELOG (read 2026-11-09 (latest dated entry 0.8.7, 2026-10-01))
- pgvector README (Filtering, Iterative Index Scans) (undated (install docs show v0.8.7), read 2026-11-09)
- turbopuffer customer log: Linear

- ViDoRe V3: Introducing vidore v3 (Nov 2025)

Agent engineering, evals and security research

- A statistical approach to model evaluations (Anthropic Research; paper arXiv 2411.00640) (2024-11-19)
- AbstentionBench: Reasoning LLMs Fail on Unanswerable Questions (Kirichenko et al., arXiv 2506.09038) (2025-06-10)
- AgentDojo results (and AgentDojo paper arXiv 2406.13352) (newest row 2025-02-24 (accessed 2026-11-09); paper v3 2024-11-24)
- AI Evals FAQ (Hamel Husain & Shreya Shankar) (2026-09-18 (modified 2026-09-21))
- Anthropic (Thariq Shihipar), Building agents with the Claude Agent SDK: Building agents with the claude agent sdk (29 Sep 2025)
- Anthropic (Zhang, Lazuka, Murag), Equipping agents for the real world with Agent Skills: Equipping agents for the real world with agent skills (16 Oct 2025)
- Anthropic Alignment blog: Bloom auto evals (2025-12-19)
- Anthropic Engineering, "Building a C compiler with a team of parallel Claudes" (Nicholas Carlini) (2026-02-05)
- Anthropic Engineering, "Building effective agents" (Erik S., Barry Zhang) (2024-12-19)
- Anthropic Engineering, "Code execution with MCP: Building more efficient agents" (2025-11-04)
- Anthropic Engineering, "Demystifying evals for AI agents" (2026-01-09)
- Anthropic Engineering, "Effective context engineering for AI agents" (2025-09-29)
- Anthropic Engineering, "Harness design for long-running application development" (Prithvi Rajasekaran) (2026-03-24)
- Anthropic Engineering, "How we built Claude Code auto mode: a safer way to skip permissions" (John Hughes) (2026-03-25)
- Anthropic Engineering, "How we built our multi-agent research system" (2025-06-13)
- Anthropic Engineering, "Introducing advanced tool use on the Claude Developer Platform" (Bin Wu) (2025-11-24)
- Anthropic Engineering, "Scaling Managed Agents: Decoupling the brain from the hands" (Lance Martin, Gabe Cemaj, Michael Cohen) (2026-04-08)
- Anthropic Engineering, "The components of agent memory" (Lance Martin) (2026-11-06)
- Anthropic Engineering, "Writing effective tools for agents — with agents" (Ken Aizawa) (2025-09-11)
- Anthropic Engineering, Beyond permission prompts: making Claude Code more secure and autonomous (sandboxing) (2025-10-20)

- Anthropic, "A postmortem of three recent issues": A postmortem of three recent issues (2025-09-17)
- Anthropic, "An update on recent Claude Code quality reports": April 23 postmortem (2026-04-23)
- Anthropic, How we contain Claude across products: How we contain claude (25 May 2026)
- Anthropic, Managing context on the Claude Developer Platform: Context management (29 Sep 2025)
- Anthropic, Mitigating prompt injections in browser use (2025-11-24)
- Anthropic, Redesigning source control at Anthropic: Redesigning source control (13 Oct 2026)
- Anthropic, The advisor strategy: The advisor strategy (9 Apr 2026)
- Anthropic: Infrastructure noise (2026-02-05)
- Anthropic: PDF support
- Anthropic: Search results
- API and data retention (Claude Docs) (undated, read 2026-11-09)
- LLM rerankers study (arXiv 2508.16757) (Aug 2025)
- Auth0 (2025-11-19)
- Batch processing (Claude Docs) (undated, read 2026-11-09)
- Beck, Eckman, Kern, Kreuter, arXiv 2509.08514 / HDSR 8.2 (2025-09; Spring 2026)
- Beurer-Kellner et al., Design Patterns for Securing LLM Agents against Prompt Injections (arXiv 2506.08837) (2025-06-27 (v3; v1 2025-06-10))
- BrowseComp-Plus (arXiv 2508.06600) (Aug 2025)
- Claude Agent SDK: Hosting; Agent SDK overview; PyPI claude-agent-sdk 0.2.180 (2026-11-09) (undated, read 2026-11-09)
- Claude blog, "Lessons from building Claude Code: Prompt caching is everything", Thariq Shihipar: Lessons from building claude code prompt caching is everything (2026-04-30)
- Claude Code docs, Orchestrate teams of Claude Code sessions (undated, read 9 Nov 2026)
- Claude docs, Compaction overview (undated, read 9 Nov 2026)
- Claude Docs: Citations (undated (status stable), accessed 2026-11-09)
- Claude Haiku 5.5 migration guide (Claude Docs) (undated, read 2026-11-09)
- Claude in Amazon Bedrock (Claude Docs) (undated, read 2026-11-09)
- Claude Sonnet 5.5 System Card §5.2.1 (2026-09-28)
- Contextual Retrieval: Contextual retrieval (Sep 2024)

- Creating a LLM-as-a-Judge That Drives Business Results (Hamel Husain) (2024-10-29 (modified 2026-09-01))
- Dash et al. (arXiv 2606.04329) (Jun 2026)
- Data residency (Claude Docs) (undated, read 2026-11-09)
- Define success criteria and build evaluations (Anthropic / Claude Developer Platform docs) (undated (accessed 2026-11-09))
- Deprecations (OpenAI API docs) (Evals entry 2026-06-03; page latest entry 2026-11-05)
- Díaz, Kern, Olive: Google's Approach for Secure AI Agents: An Introduction (PDF) (2025-05)
- Dziemian et al., arXiv 2603.15714 (2026-03-16)
- Evaluating the Effectiveness of LLM-Evaluators (aka LLM-as-Judge) (Eugene Yan) (2024-08)
- Future of Privacy Forum (2026-07-28, updated 2026-08-05)
- Gaming the Judge (arXiv 2601.14691) (2026-01-21)
- Gao et al., Enabling Large Language Models to Generate Text with Citations (ALCE), EMNLP 2023 (arXiv v1 2023-05-24, v2 2023-10-31)
- GDPval (arXiv 2510.04374) (Oct 2025)
- GitHub Docs, Risks and mitigations for GitHub Copilot cloud agent; Customizing or disabling the firewall (undated docs pages (read 2026-11-09))
- GitHub: Generating an installation access token for a github app
- GitHub: Troubleshooting the rest api
- Google ADK docs, "Workflows" (Undated docs (© 2026); accessed 2026-11-09)
- Google Developers Blog, "Architecting efficient context-aware multi-agent framework for production" (Hangfei Lin) (2025-12-04)
- Han et al., RAG vs GraphRAG
- HealthBench (arXiv 2505.08775) (May 2025)
- Hossain et al., An Analysis of Natural Language Inference Benchmarks through the Lens of Negation (EMNLP 2020) (2020-11)
- HumanLayer, "12-factor agents" (Dex Horthy), README and factor 8 (Repo page undated (© 2026 footer, 273 commits); accessed 2026-11-09)
- Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena (Zheng et al., NeurIPS 2023 D&B) (2023-06-09 (v4 2023-12-24))
- JudgmentBench, arXiv 2605.25240, v5 2026-11-06
- Kim et al. (arXiv 2512.08296) (Dec 2025)

- LangChain blog, "Context Management for Deep Agents" (Chester Curme, Mason Daugherty) (2026-01-28)
- LLM-AggreFact leaderboard (undated, accessed 2026-11-09 (metric not stated on page))
- Meta AI, Agents Rule of Two: A Practical Approach to AI Agent Security (2025-10-31)
- Microsoft (Shawn Henry), Microsoft Agent Framework Version 1.0: Microsoft agent framework version 1 0 (3 Apr 2026)
- Microsoft, Taxonomy of Failure Mode in Agentic AI Systems (whitepaper) (2025-04 (blog 2025-04-24))
- Microsoft: Lazygraphrag setting a new standard for quality and cost (Nov 2024)
- Migrating to API version 2026-10-21 (Claude Docs) (undated, read 2026-11-09)
- Mission-critical evals at scale: learnings from 100,000 medical decisions (Christopher Lovejoy, AI Engineer Summit) (2025 (AI Engineer Summit 2025; exact date not on page))
- Model deprecations (Claude Docs) (undated, read 2026-11-09)
- Model IDs and versioning (Claude Docs) (undated, read 2026-11-09)
- Models overview (Claude Docs) (undated, read 2026-11-09)
- Nasr, Carlini, Tramèr et al., The Attacker Moves Second (arXiv 2510.09023) (2025-10-10)
- NIST CSRC, COSIS project page (updated 2026-10-20)
- NoLiMa (arXiv 2502.05167) (Feb 2025)
- One Token to Fool LLM-as-a-Judge, arXiv 2507.08794, v3 2026-06-11
- OpenAI Agents SDK docs, "Agent orchestration" (Undated docs; accessed 2026-11-09)
- OpenAI Agents SDK docs, "Human-in-the-loop" (Undated docs; accessed 2026-11-09 (the JS/TS SDK guide has the same model))
- OpenAI retrieval guide
- OpenAI, "A practical guide to building agents" (Undated PDF (the document text carries no date); accessed 2026-11-09)
- OWASP Top 10 for Agentic Applications 2026 (Version 2026, December 2025), PDF (2025-12 (landing page dated 2025-12-09))
- OWASP Top 10 for LLM Applications 2025, LLM06:2025 Excessive Agency (2025 (OWASP LLM Top 10 2025 edition; page undated))
- Advisor tool (Claude Docs)
- Pricing (Claude Docs) (undated, read 2026-11-09)
- Privacy Center: How long do you store my organization's data? (last updated 2026-07-01)

- Prompt caching (Claude Docs) (undated, read 2026-11-09)
- Rashkin et al., Measuring Attribution in Natural Language Generation Models (AIS) (arXiv v1 2021-12-23, v2 2022-08-02)
- Rate limits (Claude API reference) (undated, read 2026-11-09)
- Reduce hallucinations (Claude Docs) (undated, read 2026-11-09)
- Refusals and fallback (Claude Docs) (undated, read 2026-11-09)
- Release notes (Claude Docs) (entry dated 2026-10-21, read 2026-11-09)
- Service tiers (Claude API reference) (undated, read 2026-11-09)
- Sim2Real (arXiv 2603.11245) (Mar 2026)
- Simon Willison, The lethal trifecta for AI agents (2025-06-16)
- Structured outputs (Claude Docs) (undated, read 2026-11-09)
- Task budgets (Claude Docs) (undated)
- W3C Web Annotation Data Model (Recommendation) (2017-02-23)
- W3C: Webauthn 3 (25 Aug 2026)
- τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains (Yao et al., arXiv 2406.12045) (2024-06-17)

Model platform: Anthropic and Voyage

- MongoDB Docs: Manage organization settings (Voyage AI model training data) (undated, read 2026-11-09)
- MongoDB Docs: Voyage AI model lifecycle (undated page, latest changelog entry 2026-10-20, read 2026-11-09)
- MongoDB Docs: Voyage AI models
- Voyage AI docs: Pricing (updatedAt 2026-09-29)
- Voyage docs: Embeddings

Runtime, tools and observability

- Cloud SQL extensions
- DBOS AI quickstart; DBOS blog 'durable execution crashproof AI agents' (2025-02-24) (undated, read 2026-11-09)
- DBOS docs: Architecture (undated; read 2026-11-09)
- DBOS docs: FAQ (undated; read 2026-11-09)
- DBOS pricing (undated; read 2026-11-09)
- DBOS Python: Queues tutorial (undated, read 2026-11-09)
- DBOS Python: Scheduled workflows (undated, read 2026-11-09)
- DBOS Python: Transactions and datasources (undated, read 2026-11-09)
- DBOS Python: Upgrading workflow code (undated, read 2026-11-09)

- E2B: Pricing
- FastAPI: Fastapi.tiangolo.com
- Hatchet: Cron runs; Hatchet: Concurrency; Hatchet: Durable tasks (undated, read 2026-11-09)
- Langfuse docs, "LLM-as-a-Judge", undated (mentions 2026-11-16 cutover)
- Langfuse docs, Masking
- Langfuse pricing: Pricing
- Langfuse: Open source (No date shown; read 2026-11-09 (acquisition post dated 2026-01-16))
- langfuse/langfuse-terraform-gcp README (No date shown; module ref 1.0.13, app 4.77.0; read 2026-11-09)
- LangGraph docs, "Interrupts" (also: API reference, Durability type) (Undated docs; accessed 2026-11-09)
- LangGraph Python: Persistence (troubleshooting); checkpoint serde encrypted.py; checkpoint-postgres README (undated, read 2026-11-09)
- langgraph/pregel/main.py and langgraph/types.py (main branch) (main branch, read 2026-11-09)
- LangSmith: self-hosted deployment overview; PyPI langgraph-api 0.17.1 (2026-11-03) (undated, read 2026-11-09)
- MCP Authorization (2026-07-28) (2026-07-28 (spec revision))
- MCP blog, Paul Carleton (2026-06-18)
- MCP joins the Agentic AI Foundation (MCP blog) (2025-12-09)
- MCP Specification 2026-07-28: Key Changes (2026-07-28 (spec revision))
- MCP Tools (2026-07-28) (2026-07-28 (spec revision))
- Model Context Protocol specification, Tools (latest = 2026-07-28) (2026-07-28 (read 2026-11-09))
- Model Context Protocol, Security Best Practices (latest = 2026-07-28) (2026-07-28 (read 2026-11-09))
- Procrastinate docs: Retry stalled jobs; Periodic tasks (undated, read 2026-11-09)
- Pydantic AI: Deferred tools (undated, read 2026-11-09)
- Pydantic AI: Durable execution overview and DBOS page; PyPI pydantic-ai 2.66.0 (2026-11-04) (undated, read 2026-11-09)
- Restate docs: Versioning (undated; read 2026-11-09)
- semantic-conventions-genai docs/gen-ai/README.md (No date shown; read 2026-11-09)

- Temporal blog: LangGraph plugin adds durable execution (2026-07-16); temporalio/ai-integrations python/langgraph README; PyPI temporalio-langgraph feed (2026-07-16)
- Temporal blog: Production-ready agents with the OpenAI Agents SDK + Temporal (2025-07-30, GA note); Temporal Python OpenAI Agents docs (2025-07-30 (GA note dated 2026-03-23))
- Temporal Cloud pricing (undated; read 2026-11-09)
- Temporal docs: Worker Versioning (undated; read 2026-11-09)
- Temporal Python: Workflow sandbox (undated, read 2026-11-09)
- The MCP Registry (About) (No date shown; read 2026-11-09 (preview since 2025-09-08))

AWS

- A SaaS provider's guide to securely integrating with customers' AWS accounts (Datadog Security Labs) (2024-09-09 (updated 2024-09-10))
- Amazon EC2: Request throttling for the Amazon EC2 API (undated, read 2026-11-09)
- Amazon RDS User Guide: Avoiding pinning an RDS Proxy (undated, read 2026-11-09)
- Amazon S3 User Guide: Locking objects with Object Lock (undated, read 2026-11-09)
- Amazon S3 User Guide: Object Lock considerations (undated, read 2026-11-09)
- Aurora PostgreSQL release notes: extension versions (undated, read 2026-11-09)
- AWS Audit Manager availability change (undated, read 2026-11-09 (change effective 2026-04-30))
- AWS CloudFormation: Create StackSets with service-managed permissions (undated, read 2026-11-09)
- AWS CloudTrail API Reference: LookupEvents (undated, read 2026-11-09)
- AWS CloudTrail: CloudTrail Lake availability change (undated, read 2026-11-09 (change effective 2026-05-31))
- AWS CloudTrail: Validating CloudTrail log file integrity (undated, read 2026-11-09)
- AWS Config pricing (undated, © 2026, read 2026-11-09)
- AWS Config: Monitoring AWS Config with Amazon EventBridge (undated, read 2026-11-09)
- AWS Config: Setting Up Auto Remediation (undated, read 2026-11-09)
- AWS IAM User Guide: Access to AWS accounts owned by third parties (undated, read 2026-11-09)
- AWS IAM User Guide: AWS managed policies for job functions (undated, read 2026-11-09)
- AWS IAM User Guide: Generate credential reports (undated, read 2026-11-09)

- AWS KMS pricing; AWS KMS resource quotas and request quotas (undated, read 2026-11-09)
- AWS News Blog, AgentCore quality evaluations and policy controls (updated 2026-03-03 and 2026-03-31)
- AWS Prescriptive Guidance: Implementing managed PostgreSQL for multi-tenant SaaS, Decision matrix (Ward, Davis) (April 2024 (docs.aws.amazon.com copy returned 404 on 2026-11-09; .eu mirror read))
- AWS SDKs and Tools Reference Guide: Retry behavior (undated, read 2026-11-09)
- AWS Security Blog (2026-05-20)
- AWS Security Hub CSPM: Creating and updating findings (undated, read 2026-11-09)
- AWS STS API Reference: AssumeRole (undated, read 2026-11-09)
- AWS Systems Manager Automation Runbook Reference: AWSConfigRemediation-ConfigureS3BucketPublicAccessBlock (undated, read 2026-11-09)
- AWS What's New: Amazon S3 Object Lock now supports variable retention with event holds (2026-09-08)
- AWS: API control GetPublicAccessBlock
- AWS: AWS Fargate
- AWS: Bucket key
- AWS: Cognito user pools identity federation

Other

- EDB security assessment: CVE-2026-3172 pgvector parallel HNSW index build buffer overflow (2026-03-10 (CVE published 2026-02-25))
- EDPB Guidelines 02/2025 on processing of personal data through blockchain technologies, v2.0 (adopted 2026-07-07)
- InfoQ: AWS to discontinue QLDB (2024-07-26)
- PgBouncer features and configuration docs (undated (site news through 1.26.1), read 2026-11-09)
- pgsql-bugs: Re: BUG #18941 planner does not select a multicolumn btree_gin index under RLS (Tom Lane) (2025-05-30)
- PostgreSQL 18 docs §5.9 Row Security Policies (PG 18.6 docs, undated, read 2026-11-09)
- PostgreSQL docs §5.12 Table Partitioning (undated, read 2026-11-09)
- Scoring Policy / Scoring Metrics (UK AI Security Institute Inspect docs) (undated (accessed 2026-11-09))